LyX, The Other Way of Writing

2022 Edition

One of the best tools to work with the most powerful document editor

Ricardo G. Berlasso



LaTeX and derivatives such as XeTeX have the great advantage of their power and the big problem, at least in their pure form, of their difficult *learning curve*.

LyX solves the difficulty of using LaTeX without compromising its power, providing a robust and easy-to-use program that will satisfy both, new and veteran users.

In this book we explore the power of LyX, LaTeX, XeTeX and OpenType, showing how far we can go.

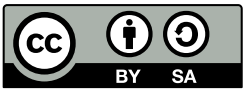
LyX, The Other Way of Writing

Speak `documentclass` And Enter

Ricardo Berlasso

2022 Edition

© 2017-2022 Ricardo Gabriel Berlasso for the original Spanish version
© 2018-2022 Ricardo Gabriel Berlasso for the English version
This book is distributed under a Creative Commons license
Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)
(http://creativecommons.org/licenses/by-sa/4.0/)





Attribution: You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.



ShareAlike: If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions: You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Any mark mentioned in this book belongs to its owner.

Links to third party websites are provided in good faith and for information only. The author disclaims any responsibility for the materials contained in any third party website referenced in this work.

The latest version of this book can be found at:
https://frommindtotype.wordpress.com/lyx-book/

Contents

I The foundations **1**

1 Introduction **3**
- 1.1 To whom is this book addressed? . . . 3
- 1.2 The structure of this book . . . 6
- 1.3 A bit of history . . . 6
- 1.4 Minimum requirements . . . 7
- 1.5 Conventions used in this book . . . 8
- 1.6 Links to have at hand . . . 10

2 Defining LaTeX **11**
- 2.1 What *is* LaTeX . . . 11
- 2.2 LaTeX as a tag language (almost...) . . . 13
- 2.3 Document classes . . . 15
- 2.4 Building the document . . . 17
- 2.5 And OpenType came to LaTeX . . . 17

3 What makes LyX so special **19**
- 3.1 The "WYSIWYM" concept . . . 21
- 3.2 Fonts and "formats" . . . 23
- 3.3 Main features of LyX . . . 23
- 3.4 What is like to work with LyX . . . 24

3.5 LyX for LaTeX fundamentalists . . . 26
3.6 When to use it, when to not use it . . . 27

II Let's start to set up things 29

4 LyX's graphical interface 31
4.1 General description . . . 31
4.2 Keyboard shortcuts . . . 34
4.3 Many documents, in many ways . . . 35

5 Building documents 37
5.1 Working with LyX . . . 37
5.1.1 General document formatting . . . 37
5.1.2 Text . . . 41
5.1.3 Lists . . . 42
5.1.4 Math . . . 43
5.1.5 Tables . . . 44
5.1.6 "Boxes" . . . 46
5.1.7 Figures, tables and other *floats* . . . 47
5.1.8 Cross-references . . . 48
5.1.9 Indexes . . . 50
5.1.10 Generate a *dynamic* PDF with the `Hyperref` package . . . 50
5.1.11 Navigate and organize the document . . . 54
5.2 Appendixes . . . 56
5.3 Bibliography . . . 56
5.4 Language . . . 57
5.4.1 "Dynamic and 'nested' quotation marks" . . . 57
5.4.2 Dictionaries . . . 58
5.4.3 Babel vs. Poliglosia . . . 59
5.4.4 Babel options in LaTeX/LyX . . . 60
5.5 Import and export documents . . . 61
5.6 Copying information from other programs . . . 62
5.7 Change tracking . . . 63
5.8 Working with *branches* . . . 63

5.9 Master documents 66

III Beyond the basic settings 69

6 Some commands 71
6.1 The LaTeX preamble 71
6.2 The TeX box 72
6.3 A matter of size 73
6.4 A matter of (typographical) shape 74
6.5 Numbers! 76
6.6 Reconfigure! 76

7 LyX, OpenType and typography 77
7.1 Introduction 78
7.2 OpenType for the default font 80
7.3 Defining a font family 81
7.4 OpenType for the entire document 82
7.5 Different fonts for different shapes 83
7.6 Different fonts for different sizes 84
7.7 Different OpenType options for each shape 86
7.8 OpenType for math: `unicode-math` 87
7.9 Line spacing 90
7.10 Hyphenation 92
7.11 Micro-typography 93
7.12 Sub- and superscripts 95

8 Footnotes and endnotes 99
8.1 Footnotes formatting and their “marks” 99
8.2 Footnotes in two columns with main text without columns 101
8.3 Footnotes without paragraph breaks 101
8.4 Restart footnote numbering on each page 101
8.5 “Numbering” footnotes with symbols 102
8.6 Return from a footnote 103
8.7 Editor notes 104
8.8 Endnotes 106

8.9 An extra: margin notes . . . 107

9 Modules, local format and math macros 109
9.1 Modules . . . 109
9.2 Creating new instructions . . . 110
9.3 Creating character styles in LyX . . . 112
9.4 Paragraph environment? . . . 115
9.5 Math macros . . . 116

10 Numbered lists 121
10.1 Modify all lists in the document . . . 121
10.2 Modify some numbered lists . . . 122

11 About the Table of Contents and other Indexes 125
11.1 Short titles . . . 125
11.2 Footnotes in headings . . . 126
11.3 Modify the name of the table of contents . . . 126
11.4 Partial tables of contents for each chapter . . . 127
11.5 Correct the spacing in lists of figures and tables . . . 127
11.6 Avoid dotted lines in indexes . . . 128
11.7 Adding a line with text to the table of contents . . . 129
11.8 Remove the page number from the entries for Parts . . . 129
11.9 Some (few) words about alphabetical indexes . . . 130

12 Headings 131
12.1 Rename the Part and Chapter environments . . . 131
12.2 Number chapters with Roman numerals . . . 132
12.3 Make the chapter start at any page . . . 132
12.4 Avoid page breaks at the beginning of a chapter . . . 132
12.5 Change the format of titles in LyX documents . . . 133
12.5.1 The packages needed . . . 133
12.5.1.1 `xcolor` . . . 134
12.5.1.2 `titlesec` . . . 134
12.5.2 A complete example . . . 134
12.6 Restart the numbering after a Part . . . 136

13 Headers and footers, and page styles 139
13.1 Preparations . . . 139
13.2 The `fancyhdr` package . . . 140
13.3 An example . . . 142
13.4 Create and use page styles . . . 143
13.5 First page of the chapter . . . 146
13.6 Blank pages . . . 147
13.7 The page style for the Part environment . . . 148

14 Figures and other floats 151
14.1 Size of inserted images . . . 151
14.1.1 Adjusting the image size in the compiled document 151
14.1.2 Size of the image in the editing window . . . 153
14.1.3 To avoid repetition: graphic groups . . . 153
14.2 Wrapped figures . . . 155
14.3 Setting the caption text formatting for all floats . . . 157
14.4 Choose page style for a particular float . . . 159
14.5 Rename floats . . . 159
14.6 Change the numbering scheme of figures and tables . . . 160

15 Drop Caps and similar topics 161
15.1 Something traditional . . . 161
15.2 Be careful with the number of lines! . . . 163
15.3 A variant: a “raised” initial . . . 164
15.4 A challenge . . . 164
15.5 An alternative: first line of paragraph in small caps . . . 165

16 Extras 167
16.1 Describe keys and menus: the `menukeys` package . . . 167
16.1.1 What is `menukeys` . . . 168
16.1.2 Using `menukeys` in LyX . . . 169
16.2 Another way to represent keys . . . 171
16.3 Side captions for figures/tables . . . 172
16.4 Adding a new float . . . 174
16.5 Colored boxes . . . 175
16.6 Things I have not talked about . . . 176

16.6.1 BibLaTeX support 176
16.6.2 Presenting code with `listings` 176
16.6.3 PDF forms 177
16.6.4 `Sweave` and `knitr` 177
16.6.5 Computer Algebra Systems 177
16.6.6 And much, much more 177

IV Appendixes **179**

A All about typeface **181**
A.1 Fonts used in this book 181
A.2 More about typefaces 182

B Local Layout **183**

C LaTeX preamble **187**

D Other formats... and a greeting **191**
D.1 Other formats used in this book 191
D.2 And yes, also direct formatting 192
D.3 Final greetings 193

Part I

The foundations

Where we define the general concepts on which all the rest is based

On the shoulders of giants

Chapter 1
Introduction

Many years ago there was a free digital magazine in Spanish about free software called "Begins." For that magazine I wrote several articles, some about Writer, some about LyX. Over time, both sets of articles became books, and now those books are being translated into English.[1]

As you will soon realize, I'm not a native English speaker. If you find any (or many!) error(s) in this text, please let me know! You can use the contact form on my personal blog for that

https://frommindtotype.wordpress.com/contact/

And of course, you can always find the latest, hopefully corrected, version of this book at

https://frommindtotype.wordpress.com/lyx-book/

1.1 To whom is this book addressed?

To say that LyX is a "graphical front-end for LaTeX" is an understatement.

LyX is a powerful document processor build *upon* LaTeX (and derivatives: in this book I will talk a lot about XeTeX) that offers its users a variety of graphical and text based tools that will satisfy even the most demanding users.

1. https://frommindtotype.wordpress.com/writer-book/
https://frommindtotype.wordpress.com/lyx-book/
There you'll find links to the original works.

In principle (life is always complicated) it would be possible to use LyX without knowing anything about what is underneath, but of course this would be a serious limitation because, as I will show you throughout the book, this program offers much more than a series of menus and buttons.

LyX can help both, new users who have never used LaTeX before as well as experts that have no problem "coding" their documents.[2] New users will find that their introduction to the LaTeX world will be smoother compared with other alternatives while advanced users will find it incredibly easy to implement their old tricks in LyX.

And here is the main point of this discourse, since sooner rather than later you will have to learn some LaTeX, whether you like it or not.

In fact, many LyX tools, such as the creation of character styles or new environments, require a fairly in-depth knowledge of LaTeX. So, is it worth the effort?

As always in life, *it depends*.

LaTeX is an incredible powerful system, but "great power carries great responsibilities." And in this case, the responsibility of the user is to learn how to use the system.

If someone provides you with the base *documenclass* (the document *template*) and you just have to fill the content, LaTeX is *really* easy to use: the base commands are simple and easy to understand, the document structure is logical and clear, when you get used to writing mathematical expressions they flow naturally, numbering and cross-references are simple to perform and maintain, and so on.

And all this becomes idyllic in a program like LyX where you don't have to worry about forgotten parentheses and the code doesn't get in the way of reading what you are writing.

If nobody provides you with a *documenclass*, but you are happy with one of the predefined ones and don't need to change anything, LaTeX is still quite simple. Again, having a well thought-out graphical interface will help you with many of the heavy details, such as defining the page and its margins, fonts, languages, and so on. A novice user might have some difficulties

2. TeX, the foundation on which LaTeX is built, is a *Turing complete* programming language: basically, any procedure that can be written as an algorithm can be implemented in TeX. I mean, nobody says it is easy, but at least it is *possible*.

when creating special pages such as the dedication or copyright, but you can probably solve this in a few minutes by asking your favorite search engine.

If you have to go beyond the basic configuration? It all depends on how far you want to go. Certainly LyX will help you along the way, but it's also clear that it can't do everything for you: LaTeX is *too* big.

To give you an idea, the LaTeX preamble for this document has more than 120 lines of code, the "local format" (style definitions) has more than 90 lines and many places in this book needed the use of code (through the famous "TeX boxes" that we will use repeatedly in this book).

It is also necessary to pay attention to the packages used because some of them may conflict with each other: for instance, `titlesec`, a package used to format headings has problems with `tocloft`, a package to change index formatting.[3]

In short, LaTeX is so powerful that it becomes a universe in itself, a sometimes contradictory universe in which it is not always easy to know its rules.

LaTeX allows you to do anything you want in terms of formatting. Not only does the ease of writing mathematical expressions make it indispensable in the writing of technical documents, its versatility allows you to seek not only the beauty of the content but also that of its presentation. Advanced typographic options, simplicity when handling figures and tables (yes, that which causes so many headaches in Writer or Word is incredibly simple in LaTeX), everything is within our reach.

So the question of whether it is worth learning LaTeX is not correct, you should ask whether you are interested in achieving the results that LaTeX can offer you and whether you are willing to make the effort that learning this system requires.

And that is something that only you, reader, can answer.

I invite you therefore to join me in this typographical adventure, so that perhaps your ideas on these issues will become clearer. Don't be afraid to experiment: LyX being free software, just install it and see if it meets your needs.

3. Most likely, this is because `titlesec` comes with `titletoc`, another package that can be used to change index formatting. The problem is that `titletoc` is not as powerful as `tocloft`.

1.2 The structure of this book

The book is divided into three parts plus some appendixes. I will take you from the simplest to the more complex, starting with a light introduction to what LaTeX is, followed by a quick description of the LyX interface and how to work with this program, before ending with advanced topics about document building.

As I already mentioned, some of the graphical tools in LyX require to know something about LaTeX code before you can use them, so their explanation will be delayed to the last part. An example of this is the creation of character styles: only after learning how to apply different typographical options by hand, of being able to create new instructions and LaTeX commands, of being confident with the LaTeX preamble, only then you'll be able to go to the "local layout" to create your styles, which will then be available with a simple right click.

All this will make some topics seem more difficult of what they really are, but there is no alternative: before you can run, you have to learn how to walk without stumbling.

1.3 A bit of history

Continuing with the joke on the book cover I could write

> *The doors of LyX, Lord of Texts. Speak, documentclass, and enter.*
> *Matthias Ettrich made it, the community made it grow.*

But to get here many things have been necessary. I will now give an exaggeratedly quick and absurdly oversimplified review of the main ones.

1977 When Donald Knuth received the printing proof of the second edition of his classic book, *The Art of Computer Programming*, he though it was horrible. At that time he had already become interested in digital typesetting systems, so the poor presentation of his book prompted him to create his own typesetting system, TeX. Many years were needed before the system was ready for general use (more than he originally anticipated), but everything started at that very moment.

1985 LaTeX was created by Leslie Lamport as a way to simplify the use of TeX through a series of *macros* that perform specific tasks, such as the creation of footnotes, headings, etc., along with tools to compile documents.

1995 Matthias Ettrich began to develop an user interface to LaTeX called Lyrix. Shortly thereafter the project was renamed to avoid conflicts with a commercial word processor now discontinued. The new name, LyX, comes from the file name extension used by Lyrix. The following year, Matthias Ettrich would create the project for which he is best known: KDE.

2004 XeTeX, a LaTeX-based system with Unicode support and the ability to use modern typographical technologies such as OpenType, AAT and Graphite, is released.

2011 Starting with version 2.0.0, LyX offers direct support for XeTeX, LuaTeX and XHTML.

And the rest is history.

More information:
https://en.wikipedia.org/wiki/TeX
https://en.wikipedia.org/wiki/LaTeX
https://en.wikipedia.org/wiki/LyX
https://en.wikipedia.org/wiki/XeTeX
https://en.wikipedia.org/wiki/Donald_Knuth
https://en.wikipedia.org/wiki/Leslie_Lamport
https://en.wikipedia.org/wiki/Matthias_Ettrich
http://www.lyx.org/misc/archaeology/
http://tolkiengateway.net/wiki/Doors_of_Durin

1.4 Minimum requirements

I will assume, reader, that you have at least the same tools that I have used to write this book. These tools are:

- LyX 2.4 or later
- A LaTeX distribution equivalent to TeXLive 2021 or later
- (Optional) Pandoc 2.17 or later

I will also assume that you know, or at least have someone to ask, how to install those programs and how to add packages to that LaTeX distribution. This clearly implies that I take for granted that you know how to use your computer.

And most importantly, I will assume that you are willing to try new things and delve into the topics introduced in this book on your own.

In this book you will *not* find detailed guides on how to do this or that, or exercises to get used to the program, for that there is already excellent documentation included in LyX and found under the Help menu. Because of this and even if many topics will be covered more or less exhaustively, many others will only be *presented*, leaving it to your curiosity to familiarize yourself with the documentation and to try to see beyond the predefined options.

The best way to *learn* is to *do*!

1.5 Conventions used in this book

The paths to the different menus will be indicated with a special format.[4] For example, the path to reach the "LaTeX preamble" (menu *Document*, then *Settings* and finally *LaTeX Preamble*) will be indicated as:

Document » Settings » LaTeX Preamble

Elements on the destination of those menu paths will be indicated with Small Caps.

Keyboard shortcuts will also have a special formatting.[5] For example the key combination "Ctrl" and "E" will be presented as

4. Courtesy of the `menukeys` package. I will come back to this this package in section 16.1.2.
5. Using a character style with the Libertinus Keyboard font.

Ctrl E

while the sequence "Alt" with "P", then "2" will be presented as follows:

Alt P, then 2.

I am aware that Mac's operative system requires adapting those shortcuts, but I do not have access to that system so I cannot provide the equivalences.

To indicate code or package names a fixed-with (typewriter) font will be used:

```
\usepackage{emptypage}
titlesec
```

To indicate addresses, either on the internet or in the file system (mostly to indicate the location of a package documentation) a sans serif font will be used:

https://www.lyx.org/
/usr/share/texmf/doc/latex/fancyhdr/

About that last point, I'm writing from a Linux[6] system and that means the addresses that I will provide are those of my system. If you use a different operating system or something other than TEXLive, you will have to adapt the addresses, but it won't be difficult to do so.

For those with the PDF version of this document, the internet links will be blue while the internal links (table of contents, cross references, etc.) will be shown in red. Each link will be "active" on the PDF: one click on them will take you to the linked location, either by opening the web browser on the linked page or by scrolling within the document.

6. Or GNU/Linux, if you prefer. Or GNU/BSD/MIT/Apache/MPD/etc./Linux, to be more precise. I use openSUSE Leap.

1.6 Links to have at hand

LyX project page is in

https://www.lyx.org/

The wiki

https://wiki.lyx.org/

To get support on LyX you have the mailing lists. To subscribe it is enough to write an e-mail to

lyx-users-subscribe@lists.lyx.org

After confirming the subscription, it is enough to write to

lyx-users@lists.lyx.org

It is important to keep in mind that the mailing list is set to "reply to sender," so to keep a conversation there you must remember to add the address of the list. I forgot it many times...

The traffic on the list is low, but answers are fast!

Just remember: it is a public mailing list, so your e-mail address will be known to everyone!

The bug tracking system is

https://www.lyx.org/trac/wiki/BugTrackerHome

As we progress through this book, we will need more and more information about LaTeX and the best place to obtain documentation for any package (other than the local installation) is the CTAN *(Comprehensive TeX Archive Network)* repository:

http://ctan.org/

And, of course, the place where every answer about LaTeX can be found is *TeX-LaTeX Stack Exchange:*

https://tex.stackexchange.com/

If nobody there knows how to solve it, it is impossible. And since almost nothing is impossible in LaTeX, there you will find the answer.

Chapter 2
Defining LaTeX

As we already saw in the introduction, *in principle* it is not necessary to know LaTeX (or XeTeX) to use LyX: the latter offers a really nice graphical interface that simplifies the use of the former. But it is also true that knowing *something*, that being aware of what LaTeX *is* and how it works will give you the power to control the appearance of your documents to a level that is unreachable for other word processors.

And that is the reason why this book begins with a light introduction to LaTeX, that being clear about what is going on "under the hood" will allow you to go further.

In the third part of this book, starting with chapter 6, I will come back to these topics, that to dive into the advanced part there is no alternative but to write some code. But for now it will be enough to go through some basic concepts in a relaxed and superficial way.

2.1 What *is* LaTeX

The fundamental idea behind LaTeX and derivatives such as XeTeX (which is not always achieved, everything must be said) is to separate *content* from *formatting*.

To indicate a chapter heading you just say "chapter" (using the LaTeX language, of course) without considering typography, size, numbering... All those details are automatically handled by LaTeX. You only have to worry about the text itself!

The *format* is defined separately in a special file called *documentclass* that is called in the document, along with other formatting indications, on what is known as the "LaTeX preamble."

In this sense, LaTeX works as an interpreted programming language in which instructions are given to the system, which then "builds" the document. These instructions can be written in a conventional text file, so, *in principle*, you do not need a special program to write a LaTeX document, although it is highly recommended to use one. And for that we have the protagonist of this book: LyX.

This double structure, *file with content + file with formatting definitions* is at the same time the point of greatest strength and the main problem of LaTeX and derivatives.

The greatest strength because it is possible to change from one *documentclass* to another without major problems, because having the styles separated from the content makes the formatting of the document consistent, because if someone else provides you with the *documentclass* your only task will be to write since the document structure is standard and easy to understand.

The biggest problem because defining a *documentclass* it is incredible difficult, someone else must provide it, because the use of a different typeface needs some work, because trying to change the format from what is indicated in the *documentclass* could result in a lot of extra work.

But don't worry, fortunately, any LaTeX distribution such as TeXLive (for Linux) or MiKTeX (for the "small window"[1] system) is packed with so many *documentclasses* that almost any imaginable need will be covered out of the box. And of course, with a little of extra work it's always possible to customize everything.

The problem for many new users is that LaTeX is closer to a programming language than to a traditional text composition system: the common user (i.e., the non-academic who does not need to write complex mathematical formulae) tends to feel intimidated by LaTeX. After all, even if LaTeX instructions are simple and intuitive, they are still *instructions*.

But don't worry, as I will hopefully show you in the next pages, this problem is not entirely real.

1. I don't want to infringe copyright.

2.2 LaTeX as a tag language (almost...)

The most famous tag programming language is perhaps HTML, so if you know something about it you will quickly get an idea of what we are talking about: each element of a LaTeX document is wrapped in tags that define the properties we want to assign to it.

For example, if you want to write a famous mathematical expression such as[2] $E^2 = \left(mc^2\right)^2 + \left(pc\right)^2$, to do that "in-line" (that is, within a paragraph), in traditional LaTeX (with LyX is way easier) you need to write:

```
$E^{2}=\left(mc^{2}\right)^{2}+
    \left(pc\right)^{2}$
```

whereas if you want the expression in its own paragraph, with a number that automatically changes when other expressions are included, such as

$$E^2 = \left(mc^2\right)^2 + \left(pc\right)^2 \tag{2.1}$$

then you need to write

```
\begin{equation}
   E^{2}=\left(mc^{2}\right)^{2}+
      \left(pc\right)^{2}
\end{equation}
```

On the other hand, if you are not interested in the number you can use

```
\[
   E^{2}=\left(mc^{2}\right)^{2}+
      \left(pc\right)^{2}
\]
```

LaTeX also uses instructions that are more similar to "functions" (hence the "almost"). For example, to indicate the title of a section (I'll come back to what is meant by section later) you need to write

2. No, it is not wrong: *the other one* is wrong (or rather, incomplete).

```
\section{This is a section title}
```

Many instructions can be nested, so if you want to insert a mathematical expression within the title of the section you simply write something like this:

```
\section{The relativistic energy:
   $E^{2}=\left(mc^{2}\right)^{2}+
       \left(pc\right)^{2}$}
```

It is also possible to "give a name" to a numbered equation in order to reference it later with a cross-reference. For this purpose you can use the instruction

```
\label{name}
```

With all this, a typical "pure" LaTeX document would look similar to the following example (plus some other things that define the document class and that will be considered later):

```
\begin{document}
\section{The relativistic energy:
   $E^{2}=\left(mc^{2}\right)^{2}+
       \left(pc\right)^{2}$}
The famous relationship between energy,
mass and the speed of light squared,
$E=mc^{2}$ is just a particular case
of a more general and important
expression, which can be seen,
numbered and in all its glory, here:
\begin{equation}
   E^{2}=\left(mc^{2}\right)^{2}+
      \left(pc\right)^{2}
   \label{eq:Einstein}
\end{equation}
The equation \ref{eq:Einstein} gave
many headaches to the fathers of
```

```
quantum mechanics.
\end{document}
```

Let me insist that with LyX it won't be necessary to write all this, I'm just showing you how everything works "under the hood."

As you can see, within the above code a cross-reference (like this one: 2.1) has been made linking to the numbered equation using the following command

```
\ref{name}
```

It is important to keep in mind that both `\label` and `\ref` can be used for far more than just calling equations. These instructions are extremely flexible and if you place `\label` next to a section title (or a figure or table), `\ref` will understand where the referenced it's calling is located and will display it as it should.

From this simple, and clearly incomplete, example it is evident that while writing a LaTeX document you deal only with the content, not with its form; even the numbering of the sections and equations (and figures and tables) is automatically performed by the system.

Another thing that is done fully automatically in LaTeX and that is a real blessing in very complex documents is the placement of figures and tables: not only they will change page as needed, but also the text around them will flow automatically so as to not leave those ugly gaps on the page so common on other word processors.

Yes, you read that right, without effort on the part of the writer[3] disappear in LaTeX the eternal struggles with blank spaces that remain when a figure moves to the next page, because the system, using criteria that can be easily customized, makes sure that everything is perfect.

2.3 Document classes

What I said in the previous section, you may object, about thinking more about the content of a document than its form is all very well, but sooner

3. We will see later the limits of all this, that in life nothing is perfect, but do not worry, they are easy to overcome.

or later I will also have to worry about how what I am writing will look like. How do I decide what typeface to use, its sizes and all those things that are always necessary?

As already mentioned, LaTeX takes the definition for each style from a *documentclass*, which is nothing more (and nothing less) than a special file in which all the formatting stuff is set.

The formats defined are of different types. You have *environments* that define how particular paragraphs, or text fragments, behave; you also have page formats; footnote formats; all the formatting that makes up the document.

So, with the article document classes you can separate the content into sections, subsections, subsubsections, paragraphs, subparagraphs, and, of course, the body of the text. In book document classes you also have, above these levels, the chapters, and above the chapters, the parts. Thus, what size and font feature should be used when the `\section` instruction is used in the document is determined by the definitions within the *documentclass*.

To select a particular *documentclass*, such as *article*, and a language in which we want to write, such as English, in a pure, old style LaTeX document you need to write at the very beginning (before the `\begin{document}` introduced in the previous section):

```
\documentclass[english]{article}
```

Creating a *documentclass* from scratch is really difficult and requires advanced knowledge of LaTeX, but fortunately you do not need to do so since every modern LaTeX distribution offers a huge number of ready-to-use *documentclasses* and in the CTAN repository you can find many, many more. This fact, together with the possibility of fine-tuning the default values of any *documentclass* will allow you to format your documents almost without any problems.

For example, if instead of using A4 paper size which is the default format of the documentclass article you want to use A5, the instruction above changes to the following:

```
\documentclass[a5paper,english]{article}
```

In this part of the document before the `\begin{document}`, a part called LaTeX *preamble*, it is possible to define many things. From chapter 6 onward we will make extensive use of the *preamble*, so for now I will stop talking about code. But just for now.

2.4 Building the document

As this book is an introduction to a graphical tool to manage LaTeX, I will not enter the jungle of technical details on how a document is built: LyX does everything for us, so I will only give you the basic ideas.

Your document is so far just a collection of LaTeX instructions that must be compiled (usually, more than once) to obtain the final result. This compilation is performed by calling a program called "latex" or another closely related to it (pdflatex, xelatex, etc.) with a series of parameters among which must be, obviously, the name and location of the file you have written. LaTeX then generates a .dvi or .pdf file that can be visualized or printed with Okular or any other similar program and where all the definitions of the *documentclass* are implemented. It is also possible, by using the `hyperref` package, to obtain a dynamic document in which links, either internal to the document or external to another documents or websites, will be active (I will come back to this point bellow).

2.5 And OpenType came to LaTeX

Something I have not talked about so far is how to select fonts. This omission is due to two reasons, the first of which being that making that selection from within LyX is *far* easier than doing it manually in a pure LaTeX document.

The second reason is somewhat more cumbersome: LaTeX uses its own font system, which is not only completely incompatible with any other other systems, but is also really complicated.

LaTeX typefaces are not called by family or font style, but by file name.[4]

4. Before some "LaTeX purist" protest to me, yes, I know there are *packages* that allow you to select groups of fonts more easily. But in truth this *reinforces* what I'm saying: handling *metafont* fonts by hand is not an easy task.

And to complicate matters further, LaTeX in its traditional form has only recently acquired Unicode support. This means that getting support for languages other than English required some detours.

There are LaTeX variants that solve both problems, specially that of Unicode support. In this book I will focus on the project that is perhaps the most mature of all, XeTeX, but what I discuss here will be mostly applicable to LuaTeX as well.

XeTeX, initially developed by the people of Sil International precisely to provide support for non-Western languages, is a LaTeX variant that not only provides Unicode support, but also allows us the use system fonts. In fact, XeTeX is one of the few free systems providing full OpenType support,[5] allowing the use of all the advanced features of this technology, such as automatic glyph substitution (typographical ligatures, for example), the use of true Smal Caps, true subscripts and superscripts... whatever the font provides.

XeTeX calls fonts on a different way than LaTeX, but since I haven't gone into details about how the latter one does it, I won't enter in details about how the former does it either.

At least not at this point: I'll come back to this topic briefly in chapter 5 and in great depth (load your oxygen tanks) in chapter 7.

But hey, enough of theoretical underpinnings and this story of seeing what's under the hood: this book is about LyX, so in the next chapter I'll start to talk about its possibilities.

Pour some more coffee, dear reader, we are on track to this adventure.

5. Both LibreOffice and Scribus offer fairly comprehensive support for this technology.

Chapter 3
What makes LyX so special

There is a long war between those who believe that the best way to create documents is with a text editor, as is done with pure LaTeX, and those who believe that nothing beats a good graphical interface with its drop-down menus and buttons all over the place like... well, surely you know *at least* one example of such program.

Proponents of text-based editing say, and they are often right, that when you know the right codes it is far easier to just type than to go clicking from one place to another looking for the right button to use, and that in graphical editors you spend more time looking for the right function than using it.

To this the defenders of graphical editing tools say, and they are often right, that remembering an infinitude of commands, however simple they may be, implies an effort that most of the time cannot be justified and that the learning curve in a graphical program is always gentler than in a text-based system.

In this war there is a third faction that claims that both groups are just a bunch of idiots and that nothing is better than a good keyboard shortcut. To which the two previous groups respond, and they are often right, that doing everything with those cumbersome keyboard shortcuts require, both in the fingers and in the memory, the skills of a virtuoso pianist.

Well, LyX has resolved this age-old conflict in the most elegant way possible.

On the one hand those heavy actions that need to be carried out only

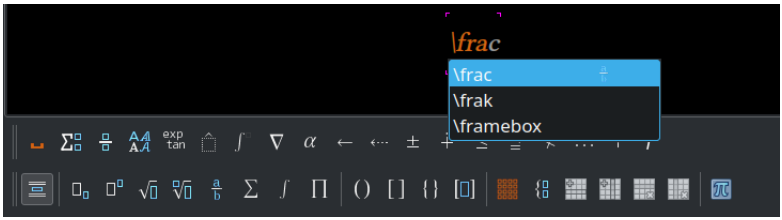


FIGURE 3.1: *The math editor in action*

once per document (selecting the *documentclass*, typefaces, page size and margins, etc.) are handled through menus.

On the other hand those particular actions that need to be carried out every now and then in particular documents (change the numbering for the index pages, set up styles for special pages, restart page numbering at the first chapter) can be handled through code.

Finally, and here lies the great beauty of the program, *for everything else you can use either code, menus/buttons or keyboard shortcuts*.

Indeed, LyX offers the best of *all* worlds, providing happiness to every kind of user.

Consider for example the figure 3.1.

Here you can see LyX's math editor in action, but the concept is valid for other functions as well. As you can see there is a traditional toolbar that allows you to insert a fraction just by pressing a button as in all graphical editors, but if instead of looking for buttons or menus you start *typing* the corresponding instruction, `\frac`, LyX not only understands what you are typing but also offers a contextual menu with all possible instructions that can be completed from what you have typed so far. Continuing with the example: `\frac` for writing fractions, `\frak` to use the *Fraktur* typeface over a character or `\framebox` to insert a box around some text.

But let's focus for a moment on `\frac`: if after finishing typing the instruction you press the space bar, the command will be replaced by a real fraction with two input fields ready to receive both numerator and denominator. Now, if instead of typing `\frac` (or using the corresponding button) you press Alt M, followed by F you will get the exactly same result.

And that is how everything works: if you type `\sqrt`, or use the button

with the square root, or the combination Alt M followed by R, you will always get an empty square root ready to receive an argument.

As I already mentioned, this versatility is not limited to equations. For example, in normal text you can use the box in the upper left corner of the toolbar area to select the *Section* "environment" (or Chapter, or special paragraphs... the available environments depend on the selected *documentclass*) or use the shortcut Alt P, followed by 2.

As you can see, the LyX interface gives us the possibility to work in all three modes: text, menu/buttons and keyboard shortcuts. Whatever makes the user happy.

3.1 The "WYSIWYM" concept

What LyX displays during document editing is *not* exactly the same you will get when the document is finally compiled, but it gives you an accurate idea of what the document *means*. For example, LyX does not show page breaks since they are completely irrelevant for working with the *content* (but it does, obviously, generate them when compiling).

Another thing that is *not* visible in the editing window are the LaTeX codes: all the `\documentclass`, `\begin{document}`, `\end{document}`, etc., commands are "hidden" and managed by LyX without the writer having to intervene directly. Even the titles of chapters and sections are presented as real titles (albeit in a different format from the final result) and not with the `\section` instruction.

Other things that new LyX users have to get used to, but which have their logic:

- There are no page rulers.
- If you press the space bar twice in a row, the second time will be ignored. The same applies with the ↵ key: spaces are automatically handled by styles, the author does not need to worry about them.
- The ⇥ key is only used to move between parts of the document (parts of a mathematical expression, table cells...).

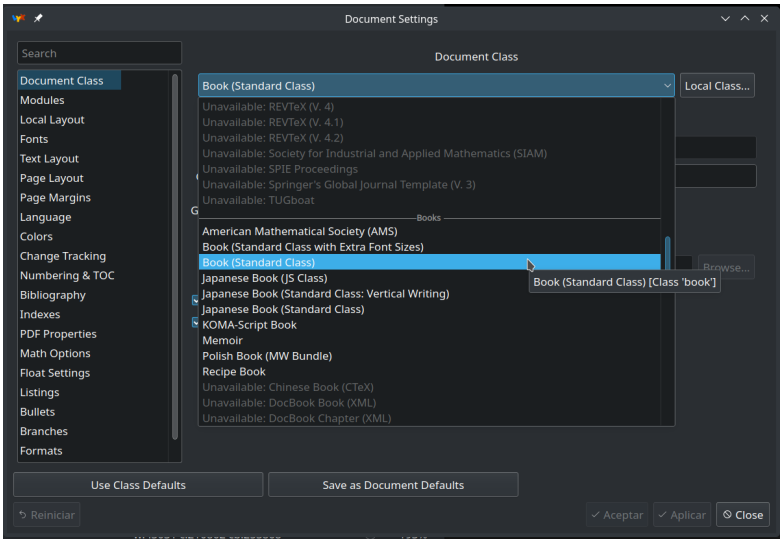


FIGURE 3.2: *Choosing the* documenclass

- Everything related to spacing, or paragraph indentation, or text formatting, headers and footers content, positioning of figures and tables... *everything* is managed by LyX in a completely automated way according to the context and the *documentclass* selected in Document > Settings > Document Class.

As you can see in figure 3.2, there are many document classes available and you can always get more through your LaTeX distribution. You also have, in other menus, many ways to customize the choice by changing, for example the way paragraphs are separated (indentation or vertical spacing), the size of the page and its margins, the location of floats (I'll tell you later what they are), numbering, and so on. The available options will depend on the document class chosen, but they are always so numerous that even the most detail-lover users will be satisfied.

Moreover, the way in which each part of the document is constructed is not only governed from there, since beyond those general options it's always possible to make fine adjustments to particular parts of the document. For example, in the Edit > ... menu it is possible to change the settings for a

particular paragraph, a single word or a special page, while from the Insert > ... menu it is possible to insert manual page breaks, special spaces between words or paragraphs, whatever is necessary.

Ah! Ctrl B toggles bold while Ctrl E toggles italics: at least that is (almost) the same as always.

3.2 Fonts and "formats"

Among the many things you can find in Document > Settings (Page Layout, Margins, Language...) there are two entries of great interest for this book: Fonts and Formats.

These two options will allow you to configure LyX to use XeTeX (or LuaTeX, but I won't discuss that option here) as the text engine when building your documents.

If you select Fonts > Use non-TeX fonts (via XeTeX/LuaTeX) it is possible to choose system fonts instead of LaTeX fonts. With this option enabled, LyX will use by default XeTeX to build the document, but you can also go to the menu ... > Formats > Default output format to make this choice explicit by selecting "PDF (XeTeX)." As you can see there are other options there, but I won't go into them.

3.3 Main features of LyX

LyX can load several documents simultaneously, opening them in "tabs" in the same way Firefox opens several web pages. It is also possible to use a "split screen" to show documents side by side (or one on top of the other, I'll come back to that later).

The program remembers in which part of the document you were working on when it was closed, so it will go back to that point when it is reloaded. It also provides a panel that shows the document structure, which can be activated from View > Outline pane or with the corresponding toolbar button (Toggle outline). This pane works as a document index, which allows you not only to quickly reach a point but also to easily change the order and "level" of any heading ("demoting" a section to subsection, for example). The Keep check box is interesting to prevent the "tree" of levels from

closing when you double-click a higher level. I will come back to this point in the section 5.1.11.

The program also offers a system to track changes, which allows you to register, accept or reject changes, thus facilitating teamwork on a single document (I will briefly talk about this in section 5.7). It can use Enchant (default option) or Hunspell for real-time spell checking and has a thesaurus that can use dictionaries from LibreOffice (section 5.4.2). It gives you the ability to work in several languages within the same document. It offers an incredible simple and robust way to work with master documents (section 5.9)...

Something important to remember is that LyX works without problems with Unicode so, even in the case you decide not to use XeTeX, you won't have to worry about inserting special symbols or diacritics because LyX will translate everything so that LaTeX understands it without trouble.

Keep in mind that the images are not embedded into the LyX files, they are only linked, and the original files remain in their original locations. So if you have to work on the same document from several systems you will have to do some extra work to keep everything synchronized. To simplify the task of moving a project to a different system, it is possible to export a compressed package as a tar.gz file containing the LyX document and every picture and related files: File » Export » LyX Archive (tar.gz). At destination, just unzip the package in an empty folder on the target system and you'll be ready to continue working.

3.4 What is like to work with LyX

I will come back to all this in great detail from next chapter, but let's now make a quick review of the "working method" you need in LyX.

I have already pointed out that figure and table placement is done automatically. For this to work you must first insert an appropriate "float" (Insert » Float) and inside it place the figure or table together with its caption. These floats, as well as all footnotes and other special elements, are present in the LyX editing window as a red box, an *inset*, with a label indicating what kind of element they are. These insets have the particularity of being able to be closed leaving only the label visible when you do not need

to see their contents: this can be achieved for a particular inset with a simple click over its label, or globally with View » Close All Insets. Many insets offer a contextual menu to set up their behavior.

At this point it is important to keep in mind that LaTeX can work mainly with encapsulated postscript images (EPS), while XeTeX or pdfTeX only with PDF images... but do not worry, LyX uses different external libraries to accept almost any type of image, converting them to the appropriate format when needed. In general, if high quality graphics are needed, using vector formats such as EPS, PDF or SVG will provide by far the best results, but any type of bitmap image (png, jpg, whatever) will work flawlessly.

A fundamental concept in LyX is that of *environment*. Environments are used to distinguish between different parts of a document, for example, separating the title of the chapter from the normal text that follows it, and can be assigned with the menu that can be found on the left of the first toolbar (or with the corresponding keyboard shortcut), just below the File menu.

The available environments depend on the selected document class. For example, when writing an article or a letter, it makes no sense to have *Chapters*, but in a book, it does.

We have already discussed the different keyboard shortcuts offered by LyX. These keyboard shortcuts are of two types: direct ones such as the combinations Ctrl M or Ctrl ⇧ M used to introduce a mathematical expression, or the keystroke sequences such as the already mentioned Alt M followed by F to introduce a fraction, or Alt M, then G followed by a letter (a, b...) to insert a Greek character (α, β...). As we already saw the assignment of the main environments can be done from the keyboard through the sequence Alt P followed by a number from 0 to 6 (or b, or e, try to see what you get!). It is important to note that some of these shortcuts may not do anything depending on the *documentclass* chosen.

The last important step in LyX's workflow is certainly the document *compilation*. Surely you have already seen the toolbar just below the *environment* menu: from left to right we have View, Update, View master document... the first two buttons will be the most used by far, at least for those users who are new to LyX.

3.5 LyX for LaTeX fundamentalists

Even if LyX uses its own XML-based file format to store the work, do not worry, LaTeX fans, that the conversion to LaTeX format is always perfect and for a good reason: LyX *always* converts to LaTeX format every time the document gets compiled so that is something that *must* work. That means that if for any reason someone needs to obtain an human-readable LaTeX file, it is enough to go to File > Export.

As we have already seen, in LyX, it is possible to insert code not only in mathematical mode. To work with code outside the math mode we have the TeX button (keyboard shortcut: Ctrl L) that opens a special box where you can write your gorgeous code. For example, if using one of the book document classes (all variants of book that LaTeX gives us) you insert with the red box the command

```
\frontmatter
```

just before the index (Insert > List/Contents/References > Table of Contents) and similarly insert a second box with

```
\mainmatter
```

right after it, you'll have the pages where the table of contents is located numbered with Roman numerals, while the following pages will have Arabic numerals starting with 1 in the first chapter.

I'll come back to these (and other) topics later.

LyX also allows you to see the LaTeX code in real time, either for the current paragraph or for the entire document, simply by enabling the option View > Code Preview Pane.

Last, but not least, LyX offers a really easy way to edit the LaTeX preamble, that is, (almost) all the LaTeX code that can be written between the `\documentclass` and `\begin{document}` instructions, code that affects the overall document behavior: Document > Settings > LaTeX Preamble.

The possibilities are almost unlimited and in the following pages I'll give you several examples.

3.6 When to use it, when to not use it

There is no such a thing as a perfect tool capable of doing everything, and clearly you do not use a screwdriver to nail a nail. LyX is powerful for an incredible variety of jobs, but it could be troublesome with others.

If you want to write a book, specially a technical one with math formulae, graphs, tables, etc., LyX is perfect (I am tempted to say *essential*), but if you need to create a poster, or a presentation, or design a magazine, or any other document that needs manual control of all its elements, it would be better to look for an alternative. Yes, it is *possible* to build a poster or a presentation with LyX (LaTeX has many document classes for presentations), but it would not be difficult to find arguments to show that it is not the right tool for the job.

Despite being one of the oldest systems for text editing, LaTeX and its derivatives continue to be one of the most powerful.

Its limitations are few while its benefits are enormous, specially thanks to LyX that facilitates all those annoying LaTeX tasks like the document class selection, the creation of figures and tables, among other things.

While some programs—for example LibreOffice Writer with its style-based philosophy and extensive OpenType support—are getting closer to the features offered by LaTeX, they still have a long way to go, specially when it comes to complex technical documents.

The broader audience for this system is certainly among people in science (physicists, mathematicians, astronomers...) and engineering, but it is not limited to that specialized group: LaTeX or XeLaTeX + LyX is open to anyone who has something to say through text and who wants to obtain beautiful documents with (relatively) low effort.

And here we come to a philosophical point: *is there any reason why we should care about the beauty of our documents?*

In fact, if you go to a bookstore and pick a random book off the shelf you will almost certainty find a really poor presentation. And I am not only talking about a bad binding: even at this point of the century and of technology many of the popular publishers still do not even bother to use typographical ligatures, let alone contextual alternates or micro-typography.

I've even have in my hands books with an horrible kerning![1]

Each person will have their own reaction to this sad reality. Mine is, *so what?*

Is there any reason for me to worry about what other do *not* do?

Of course not. I like a good looking and well presented text. I love typographical ligatures. I am a micro-typography fan. Why should I deprive myself of using the powerful tools provided by LyX, XeTeX and OpenType?

Why should you, dear reader, deprive yourself?

If you do not want to give up on dressing up your document's content with beauty, come with me, this journey has just begun.

1. https://en.wikipedia.org/wiki/Kerning
https://www.xkcd.com/1015/

Part II

Let's start to set up things

Where we start working on LyX, basic to medium level

Don't Panic!

Chapter 4
LyX's graphical interface

LyX presents a typical interface of menus, toolbars and panels. But there is much more than meets the eye.

Let's see how it is built.

4.1 General description

The graphical interface of LyX is fairly traditional in appearance, with a series of menus at the top followed by a couple of toolbars. With a right click on the toolbars you can change the size of the icons and in the Tools > Preferences menu you have several options to modify the overall appearance of the program.

But of course this is only the beginning, as can easily be seen in the figure 4.1.

A very important space that tends to be overlooked is the thin bar bellow the editing area. Here LyX presents really important information about what's going on while working:

- the function of the toolbar button on which the mouse pointer is located
- the formatting of the text on which the cursor is positioned
- the options available to complete a keyboard shortcut

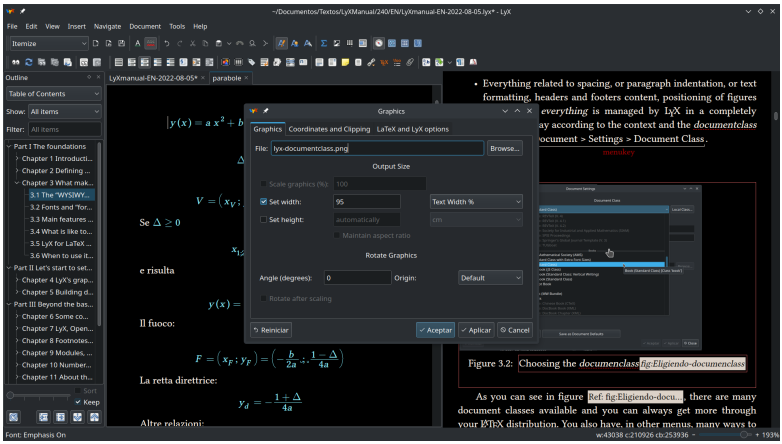


FIGURE 4.1: *LyX and its multi-document interface, showing the Outline pane and a dialog with the settings of an inserted image*

- the command associated to the function just activated from a menu
- compiler notifications (warnings, errors, etc.)

Some of these features can be seen in figure 4.2.

The Ctrl F combination will give the basic search & replace tool, while Ctrl ⇧ F offers a panel with advanced search options and the possibility to use regular expressions, search for math expressions, and so on.

> NOTE: I will not describe the advanced search & replace tools. If you want to dive into this topic you can refer to section 6.14 of the manual found under Help » User's guide.

The View » ... menu provides access to several options.

I will talk about the OUTLINE PANE in section 5.1.11.

The CODE PREVIEW PANE shows you in real time the LaTeX code generated by LyX.

The MESSAGES PANE shows the messages issued by LyX while working. You will only need it if you encounter many errors when trying to compile the document.

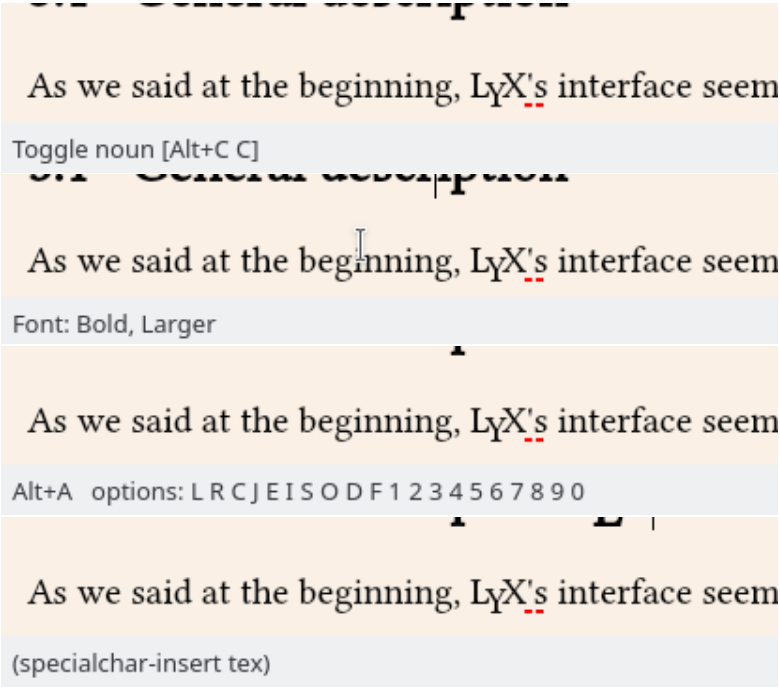


FIGURE 4.2: *From top to bottom, the first screenshot presents the status bar showing the function of a toolbar button over which the mouse pointer is located (if the action has a keyboard shortcut associated with it, this will be displayed). The second screenshot presents information about the format on which the cursor is located. The third screenshot shows how by activating the first part of some keyboard shortcuts, LyX presents all available options. The last screenshot shows relevant information of an action performed through the menus and that does not have (yet, see next section) a keyboard shortcut assigned to it.*

Under TOOLBARS it is possible to activate or deactivate different toolbars. It is important to know that toolbars marked as "auto" are activated only when needed. An example of "auto" toolbars are those related to the equation editor.

4.2 Keyboard shortcuts

As already mentioned, there are different kinds of keyboard shortcuts available in LyX:

- The *traditional* ones, like Ctrl B to toggle bold.
- Those that are sequential, where you use an activation shortcut which is then followed by other keys or combination of keys. For example, Alt P followed by 2 to activate the section environment.

As shown in the figure 4.2, when you start typing a sequential shortcut, the status bar offers the available options to complete it.

Clearly, not all LyX functions have a keyboard shortcut assigned to them. To create a new shortcut for an option you use a lot, go to Tools » Preferences » Editing » Shortcuts. There you choose the function you are interested in from the list, click on MODIFY, press the chosen combination and then accept. And do not worry, LyX will kindly warn you if the selected shortcut is already in use so you only have to change the selection.

Now, not all LyX functions are in this list, is it possible to create a keyboard shortcut for those missing functions? Of course! Here you have to look, again, to the status bar that I mentioned at the end of the previous section, because as you can *also* see in the figure 4.2, when an action is performed from a menu, there you'll find the code for that action.

> Those codes are called LyX functions. It is possible to find a complete list of those functions in the menu Help » LyX Functions.

Suppose you want to create a sequential keyboard shortcut to insert the LyX logo. This logo is found under Insert » Special character » Logos » LyX logo. When you use that menu you will see the following code in the status bar:

```
specialchar-insert lyx
```

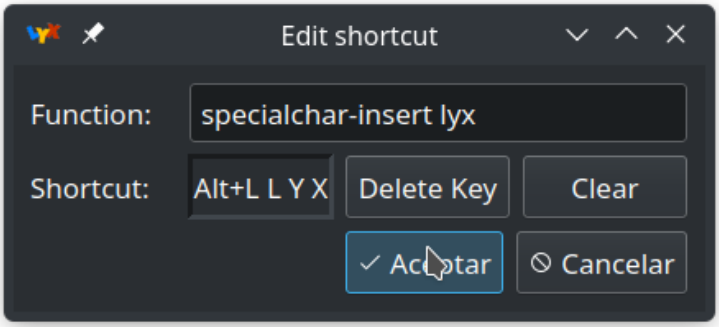


FIGURE 4.3: *Creating a new keyboard shortcut*

This is what you need to create the new shortcut. Be careful, however, that you have to respect the capitalization!

Go back to Tools » Preferences » Editing » Shortcuts, but now click on NEW. In FUNCTION type the code you found in the status bar, and in the corresponding box enter the keyboard shortcut, as you can see in the figure 4.3

This new shortcut is a tribute to the sequential shortcuts: Alt L and then the sequence L, X, and Y.

But you can do more: imagine you want a keyboard shortcut to insert the text, complete with italics, "*typography rulez!,*" the command would be

```
command-sequence font-emph;
self-insert typography rulez!; font-emph
```

and that's it!

4.3 Many documents, in many ways

LyX allows you to work simultaneously with several documents, or multiple instances of the same document, either as tabs in the same window or in a split-screen scenario.

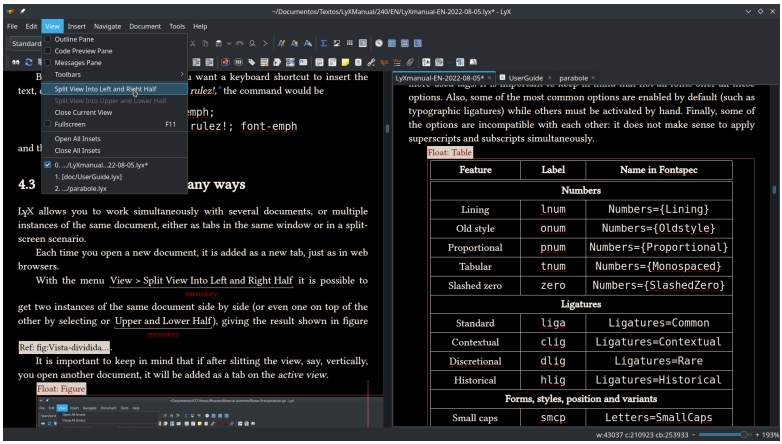


FIGURE 4.4: *Split screen + tabs: for all tastes*

Each time you open a new document, it is added as a new tab, just as in web browsers.

With the menu View » Split View Into Left and Right Half it is possible to get two instances of the same document side by side (or even one on top of the other by selecting or Upper and Lower Half), giving the result shown in figure 4.4.

It is important to keep in mind that if after splitting the view, say, vertically, you open another document, it will be added as a tab on the *active view*.

I will let you play with the options. Just do not forget the right click on a tab!

Chapter 5
Building documents

In this chapter, I will show you very quickly (after all, the program documentation is there for a reason) how to create a simple document from scratch, select its type, browse it, and modify its content. I will introduce the basic tools LyX offers and explain how to work with them. For all this I will assume that you have already read the previous chapter.

I won't go into the "LaTeX details" yet (after all, for that we have the more advanced chapters in the last part of this book), but what is presented here is essential to understand how the program works.

5.1 Working with LyX

Creating a new LyX document is not different than in other programs: you can use either File » New, Ctrl N or the corresponding toolbar button. But as always, that is just the beginning.

By default, a new LyX document uses the *Article* document class, which is one the simplest document class offered by LaTeX.

5.1.1 General document formatting

Everything (OK, yes, it's *almost* everything, but I will come back to that later) you need can be found in the Document » Settings menu, as you can see in figure 5.1.

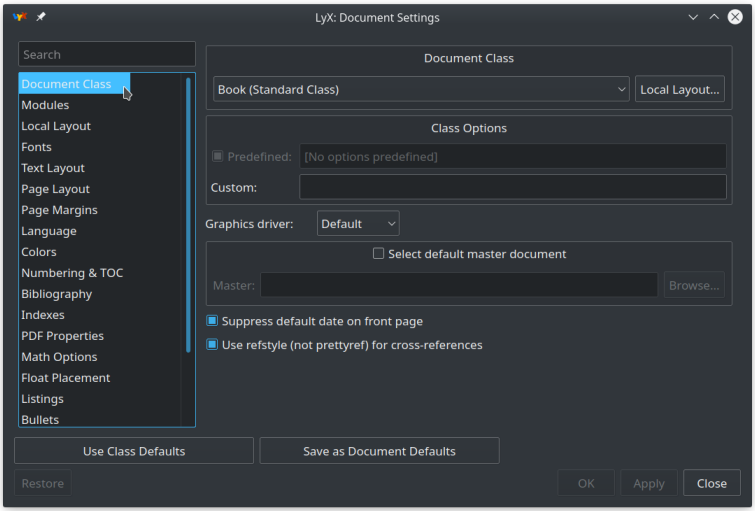


FIGURE 5.1: *The Document > Settings menu*

In the first entry you can choose the document class. Unfortunately, the only way to know which particular document class you actually need is to create a test document using the different options and see if you like the result, so I will not enter in the details about how to choose the right one. But rest assured that not only you can change the selected document class later, you can also adjust any detail of any document class!

However, here is a general tip: there are a couple of document classes that are repeated, offering a standard version and a second one with "extra font sizes." For example, "Book (Standard Class)" and "Book (Standard Class with Extra Font Sizes)." Just forget the second one.

While I'll come back to size issues in section 6.3, let's see how these classes differ.

If you go to the FONTS entry, under BASE SIZE you will find a number of possibilities. In addition to the DEFAULT option, the normal classes offer three font sizes for text: 10 points (usually, the default option), 11 and 12, while the classes with "extra sizes" offer, well, more options.

For this document I have used "Book (Standard Class)" choosing 11

points for the base font size, but the other options are perfectly valid and some offer an almost unlimited number of possibilities, all of them clearly explained in the documentation that comes with each package. Remember, CTAN is your friend.

Back to the topic. In this first entry *(Document Class)* you also have the SUPPRESS DEFAULT DATE ON FRONT PAGE option that does... exactly that. By default in all book document classes LaTeX adds on the front page the date on which the document is compiled, something that is not always desired. In fact, it is almost never desired. Well, by just checking that option as you can see in the screenshot, the date will not be added.

Something very important that is also found there is the CLASS OPTIONS section. With this box you can pass any parameter to the document class to change the document behavior.

For example, in the book document classes, when you choose in the page setup to build a two-sided document, each chapter will start in an odd (right) page: if you want the chapters to be able to start on any page you can add in CUSTOM the instruction `openany`. As we already said, the available options will depend on the document class in use, so you need to check the corresponding documentation.

I will talk about MODULES and LOCAL LAYOUT in chapter 9.

The FONT entry is, as it is easy to guess, the main place where you can select the typefaces used by the document. The most important element there is the checkbox USE NON-TEX FONTS (VIA XETEX/LUATEX) because with this option you will be able to use the system OpenType fonts instead of LaTeX's owns. I will come back to this complex, but interesting, topic in chapter 7.

In a LaTeX document there are three main font families (I'll show you how to define more later): the *Roman*, which is the base font used for the text body, *Sans Serif* and *Typewriter*. As we have seen previously, the base size box gives you the possibility to choose the size for the base font, which is the Roman one. The size of the other two families is defined relatively to that base size: for example, in this book I have used a scale of 80% for the typewriter font.[1]

The entries TEXT LAYOUT, PAGE LAYOUT and PAGE MARGINS are

1. DejaVu Sans Mono has a "generous" size.

easy to use and understand... except for the *Line spacing*. That topic *seems* innocent, but has its twists and turns so I will discuss it later, also in the chapter 7.

Of all this I want to emphasize only one point, specially important for people coming from other programs such as LibreOffice Writer: in a LaTeX/LyX document margins indicate the distance from the page border to the effective text area, *but headers and footers do not belong to the text area and therefore they are introduced into the margin*s. For this reason, in Page Margins we have specific options to configure the position and height of headers and footers.

I will talk more about Language below.

The Colors entry allows you to select, well, the document colors (text, notes, page background).

Numbering & TOC refers to how far we want to number the document structure. By default in the book classes, *Parts*, *Chapters*, *Sections*, *Subsections*, and *Subsubsections* are numbered, but nothing else. In addition, by default the table of contents only includes the numbered levels. Here, by sliding the controls on top, you can set what is numbered and what is not, or what is in the TOC and what is out of it.

And so on...

It is important to note that you can use templates in LyX: as you can see in the menu File » ... you have the option to both *load* and *save* a template. If you want to reuse the options selected for a particular document you have two options:

- The Save as Document Defaults button that you can find in the document settings menu will apply the selected options to *all* new documents.

- Save an empty document as template for each group of options (one for books, one for articles, etc.) and then call any of those templates from File » New from Template so that an unnamed copy of it is created from which to start working.

LyX already comes with several predefined templates that you can use for the creation of special documents (specific publications such as *Springer* and *Scientific American*, for example). One of these templates, called PDF

(CROPPED) under GENERAL, can be used to create a PDF from a mathematical formula with the size of that mathematical formula, which you can then use as an image in other applications.

In future chapters, I will talk about some of the other entries in the document settings menu: I will discuss the tools on PDF PROPERTIES and BRANCHES at the end of this chapter, FORMATS in the chapter 7 and the LaTeX PREAMBLE... in many different places.

5.1.2 Text

So, you have chosen fonts, page, margins and so on, let's write!

A fundamental concept when using LyX is that of *environment*. With the menu (or keyboard shortcuts) on the left side of the main toolbar, just below the FILE menu, you can assign each paragraph the *function* you want to give it: chapter title, section title, normal text, numbered list, bullet, and so on.

Everything is perfectly classified and understandable, so I will leave it to you to play a little with these options to get used to their functions. The only important thing to note is that the unnumbered elements (those with the asterisk) will not be included in the table of contents: I will come back to this in section 13.4.

In this way the document will show a well-defined and coherent structure.

LyX offers some predefined character styles: Ctrl E toggle the *italic* shape (the LaTeX command `\emph{}`), Ctrl B toggle **bold** (the command `\textbf{}`) while the sequence Alt C, then C does the same for SMALL CAPS (`\noun{}`). These three options can also be used from the toolbars.

> NOTE: In order to return to the default formatting, in addition to *reapplying* the style (think of the act of formatting as a switch that turns it on or off) you have the combination Alt C, then [ ].

For those rare situations where a bit of direct formatting may be necessary you have two menus with lots of tools: Edit › Text Properties › Customized and Edit › Paragraph Settings, both accessible from the right click too.

The way to introduce *n-dashes* and *m-dashes* is quite simple: two dashes -- provides an n-dash –, while three dashes --- gives an m-dash —.

5.1.3 Lists

Directly from the toolbar or from the *environment* drop-down menu you can choose between the *Enumerate* (Alt P, then E), *Itemize* (Alt P, then I), *Labeling* (Alt P, then L) and *Description* (Alt P, then D) environments.

The first two are the classical numbered list (the first one) and "bullets" (the second) that other word processors insist we should use. These environments can be nested to show different levels. To move from one level to the next you can use either the toolbar or the keyboard shortcuts Alt ⇧ → to go up one level and Alt ⇧ ← to go down one level. With those tools you can get something like this:

- First level
    - Second level
    - Another second level
        - Third level
    - A new second level
- A new first level

To exit the environment it is enough to select the *Standard* environment or press ↵ twice.

Both, *labeling* and *description* provide an environment that separate a "word to be defined" from the text that defines it, changing only in the details of the paragraph formatting and how the description is separated from the keyword. I'll leave you to play with these options. Just one point: if you need to have more than one word as "label" you will have to join them together with a non breaking space (Ctrl ␣), so LyX considers them as a single word.

In LyX it is quite easy to configure the bulleted lists *(itemize)*. To do this you simply go to Document » Settings » Bullets where you can select the symbol you want for each of the four possible levels, either from the six

proposed fonts or by inserting your own symbols. You can also change the size of the symbols by selecting one of the many options ranging from *tiny* to *Huger*.[2]

However, setting up numbered lists is not so immediate, so I will leave it for the last part of this book, in chapter 10.

5.1.4 Math

There are two styles for displaying mathematical expressions: *in-line* and *displayed*, with the last one that can be *numbered* or not.

The main difference between both styles is that in-line expressions provide smaller symbols (integrals, sums) to better integrate as part of a paragraph. Compare how this in-line expression is presented $\int_a^b f(x)dx$ with its *displayed* counterpart:

$$\int_a^b f(x)dx$$

The *in-line* style is available from the main toolbar, while both styles can be inserted with the menu Insert › Math. But the easiest way to insert math expressions is for sure the use of the keyboard shortcuts Ctrl M for in-line expressions and Ctrl ⇧ M for displayed ones. It is also possible to use these shortcuts to convert one kind of expression to the other: by pressing Ctrl ⇧ M inside an in-line expression, it turns it into a displayed one.

It is possible to create numbered expressions using the menu Insert › Math › Numbered Formula, or you can turn a displayed expression into a numbered one with Alt M, then N (this combination also removes existing numbering):

$$\int_a^b f(x)dx \tag{5.1}$$

Latter in this chapter I will show you how to cross-reference numbered mathematical expressions (and other items).

2. I will return to the meaning of those names in chapter 7 where I will talk about typography.

By default, *displayed* expressions are centered in the text area, but LyX offers native support for the `mathindent` package, with which it is possible to set the math expressions with indentation instead of centering them. To do so, it is enough to go to Document › Settings › Math Options to change the corresponding option. There it is also possible to select on which side of the equation the number will be displayed.

In the figure 3.1 on page 20 we saw the equation editor at work, showing how menus, buttons or pure code can be used. Tons of keyboard shortcuts are predefined for the equation editor. Many of those keyboard shortcuts start with the combination Alt M: Alt M, then G, then a character gives us a Greek character, Alt M, then F a fraction, Alt M, then (the scalable round brackets, Alt M, then L moves the limits of an integral from the right to top-bottom (the command `\limits`), etc. By hovering the mouse pointer over the toolbar buttons you can see the corresponding shortcuts so that you can learn them easily.

The possibilities offered by the LyX equation editor are almost unlimited and it really makes no sense for me to duplicate here what is already magnificently explained in the documentation that comes with the program, so after this brief introduction it only remains for me to invite you to consult the manual found in Help › Math: do not worry about its more than 80 pages, it is an easy and quick read.

5.1.5 Tables

Both, from the standard toolbar or from the menu Insert › Table you will find the classic options to choose the number of rows and columns that work for you.

With the cursor inside the table you will find at the bottom of the editing screen the table toolbar, which will allow you to insert or remove rows or columns, join cells (multi-column or multi-row, the last two buttons on the right), select different borders, rotate a cell or the entire table by 90°, and so on.

Tables can be *nested:* by placing the cursor inside a cell, it is possible to insert another table.

With a right click › Settings you have many more options such as rotating the content of a cell at an arbitrary angle or setting up a multipage table

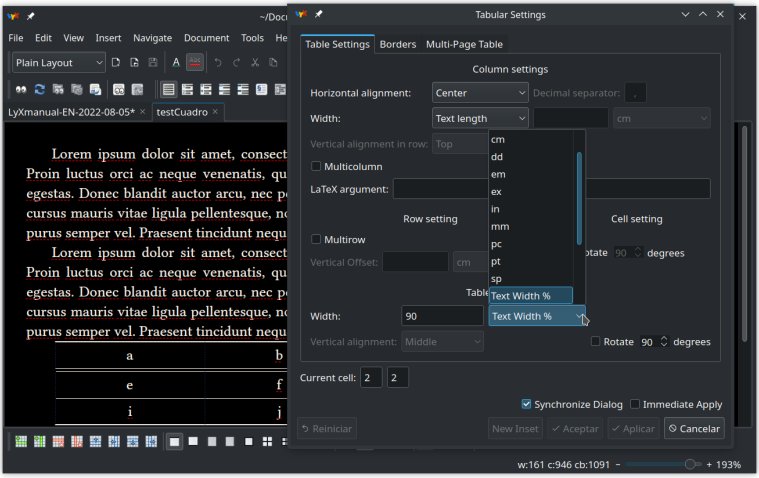


FIGURE 5.2: *Setting the table width*

whose content can be split into several pages automatically.

Other than manually setting the borders of your table, in the BORDERS tab there are several table styles. It's possible to set the default style on Document » Settings » Text Layout.

You can certainly create any formatting you want for your frame, and from 2.4 onwards it is possible to define new styles.

Something important to remember is that, from the table settings, you can also modify the width of the table. By default, the table takes the minimum width needed to accommodate the content, but you can easily make it occupy a certain percentage of the text area, for example, as shown in the figure 5.2. In fact, by setting the width of the table it will be possible to then set the relative widths of each individual column.

Now, how do we change the table *color*?

You can change the *text* color via direct formatting with a right click » Text Properties » Customize or by using a custom character style (I will return to that in chapter 9). But what about the cell *background*? Well, for that a special LaTeX package (`colortbl`) is needed, together with a bit of coding work: everything is explained in section 2.11 of the manual found

in Help » Embedded objects. But before trying to follow that path, I recommend you to carefully read the third part of this book, because in this topic you will not be able to escape from pure LaTeX code.

5.1.6 "Boxes"

In the menu Insert » Box you have several options (frameless, simple frame, oval thin/thick... there are at least seven of them). The question, of course, is what each of them does.

Basically, by default the options presented in that menu insert something that in LaTeX is called a `minipage`: a small "page within the page" floating on top of the *real* page. This small page can have a border or even a background color. It is important to keep in mind that these mini-pages cannot be divided into several *real* pages, therefore, if the content does not fit on a single page, you will have some problems. Their use is therefore the insertion of notes, for example, or of small "clarifications" that complement the main content.

Now, if what Insert » Box does is to insert a mini-page, why the menu is not called "insert minipage"? The answer is, clearly, that these "boxes" do much more, that there are more options.

With a right click on the box label you get a menu that not only allows you the switch between the different available options, but also gives you access to the box settings.

In this configuration menu it is possible to change many things, such as the box width. But most importantly, it is possible to change the "inner box," as you can see in figure 5.3.

We have three options there:

None For short texts within a single line

Parbox To "frame" a particular paragraph

Minipage What was discusses above

All the options in this menu are clear, hence, I let you play a bit with them.

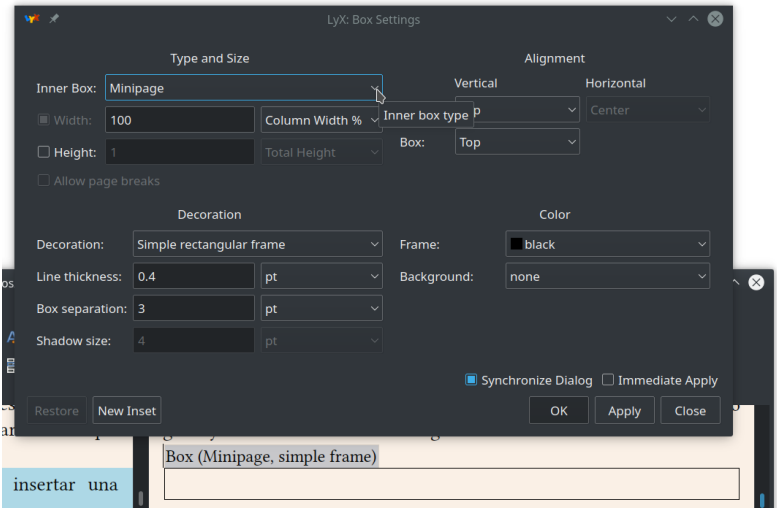


FIGURE 5.3: *Setting a "box"*

5.1.7 Figures, tables and other *floats*

In traditional word processor such as LibreOffice Writer, when you need a figure or table with its caption, you usually insert the picture first and then add the caption around it. Well, in LyX this is done exactly the other way round.

Either with the corresponding button in the toolbar, or from the menu Insert » Float » ... it is possible to insert a "float" in which to write the caption and then insert the image or the table.

These floats do exactly what their name say they do: they *float* on top of the page. Depending on the available space, after compiling the document these floats will either be placed at the top of the current page, at its bottom or they will go to the next page (only when there is no space left) allowing the text to flow comfortably around them without any additional work.

The *float* behavior can be set either for each particular float (right click on its label) or in general for the document as a whole from Document » Settings » Float Settings. Generally speaking, letting LaTeX to do the job on its own *(Class default)* is a good idea: it does it *really* well.

Now, LyX gives you the possibility to easily choose if you want the caption to go above or bellow the figure or the table. You just have to move the cursor up or down the caption, as you can see in the upper part of figure 5.4.

Another thing that is shown in figure 5.4 is the possibility of working with *subfloats:* figures (or tables) within figures, numbered as "figure 3A," etc. It is enough to insert a float inside the main float to meet your needs. These "subfloats" can be placed in any position: one above and one below, one next to the other... again, I leave it up to you to explore this possibility.[3]

It is important to keep in mind that if you want a float *without* caption, it is enough to delete the corresponding box. If you later want to reintroduce it, with the cursor inside the float go to Insert > Caption.

I will talk more about figures and other floats in chapter 14.

5.1.8 Cross-references

LyX does not have an automated tagging mechanism. For this reason, to insert a cross-reference to a chapter, equation or figure, a simple two-step process is needed.

First you have to create the label, either with the Insert Label button or from Insert > Label, next to (or, in the case of floats, inside of) the element you want to reference. As soon as the label is ready, you can now call it with the Insert Cross-Reference option, again either from the menu or from the corresponding button.

As already commented in the general introduction, the labels can be placed in any type of content: figures, tables, numbered equations, chapters, parts or sections, footnotes, whatever is needed. LyX will understand without problems what type of content is being labeled so that in the cross-reference the numbering will always be correct.

It is important, when inserting a label, to give it a name that you can later recognize without problems. For this reason it is a good idea to accept the suggested pattern that LyX offers by default, prefixing the label with a few characters that help to understand if we are talking about a chapter, figure, or anything else.

3. Get used to it, this is the trend.

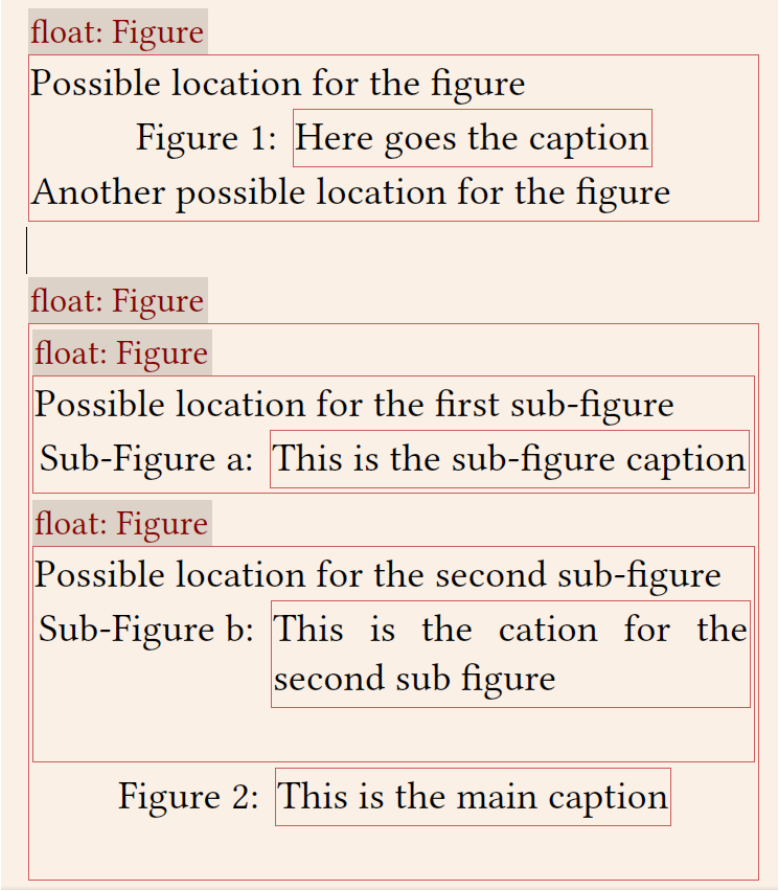


FIGURE 5.4: *Working with floats. In the upper part of the figure you see how the float looks in the editing window, indicating with text where you can place the image. In the lower part of the figure you see that it is also quite easy to work with subfigures by simply inserting the necessary floats* inside *the main one and letting LyX do its magic. As you can also see in* this *figure, I do not like subfigures.*

Now, suppose you have changed the title of a previously labeled chapter, or that you promoted a subsection transforming it into a section, *is it possible to change the name of the label?* Of course! *And without fear of ruining the cross-references to it?* Without a shadow of a doubt! With a simple click on the tag, you can later change its name to a more appropriate one, and the cross-references to it will be automatically updated without extra work.

Speaking of cross-references, the menu you get when you *insert* them is very clear, so I will just point out its most interesting features:

- In the upper part of the window you can choose between the open documents, something that makes the insertion of cross-references between documents incredible simple.
- On top of the list there is a filter to facilitate the search for the label you are looking for.
- In the lower part of the window there are different options for the cross-references, such as number or text, page, both...

In figure 5.5 you can see both menus, the one used to create the label and the other to introduce a cross-reference to that label.

5.1.9 Indexes

Changing the default settings of an index, either a table of contents (TOC), word index, any kind of index, is one of the most cumbersome tasks in LaTeX. There are several ways to set up indexes: in fact, there are several ways to *create* an index.

Fortunately, in the bast majority of cases the default index is just perfect, so just go to Insert » List/Contents/References » ... to create an index, whether it is the general one, figures, tables, etc.

Since index configuration is one of the advanced LaTeX topics, I will leave it for the final part of this book, in chapter 11.

5.1.10 Generate a *dynamic* PDF with the `Hyperref` package

The `hyperref` package can be used to set up the PDF output of LaTeX documents so they have active links, either within the document, to web-sites

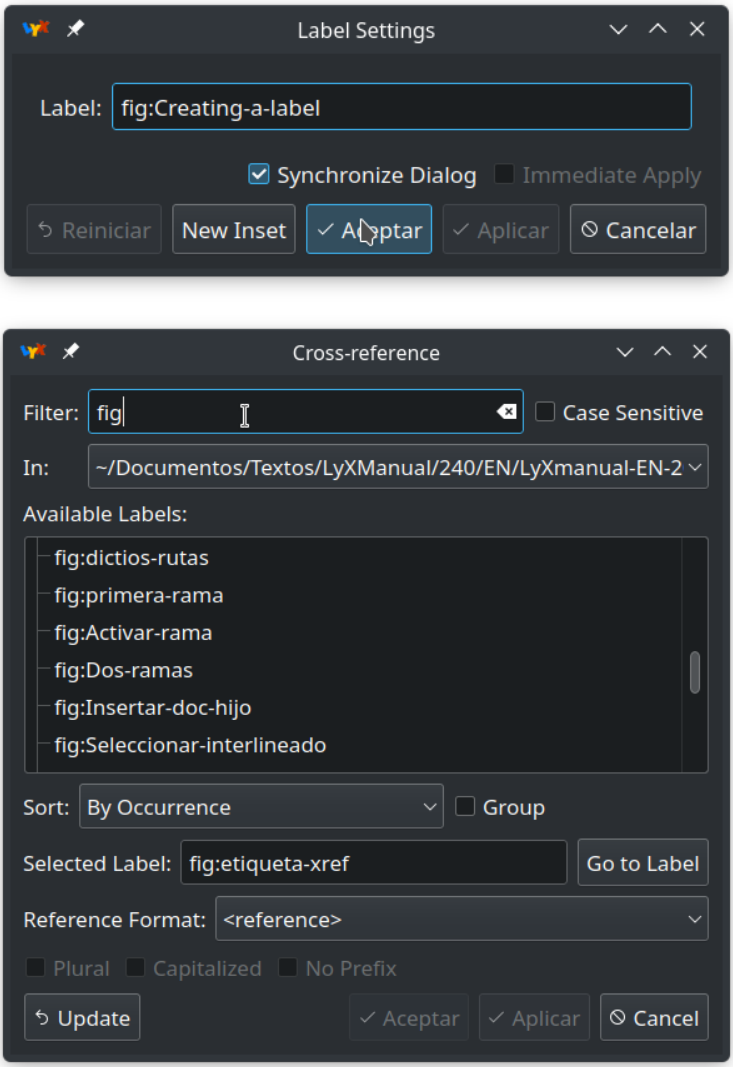


FIGURE 5.5: *Creating a label (above) and inserting a cross-reference (below). And yes, the names of the label are in Spanish, so what?*

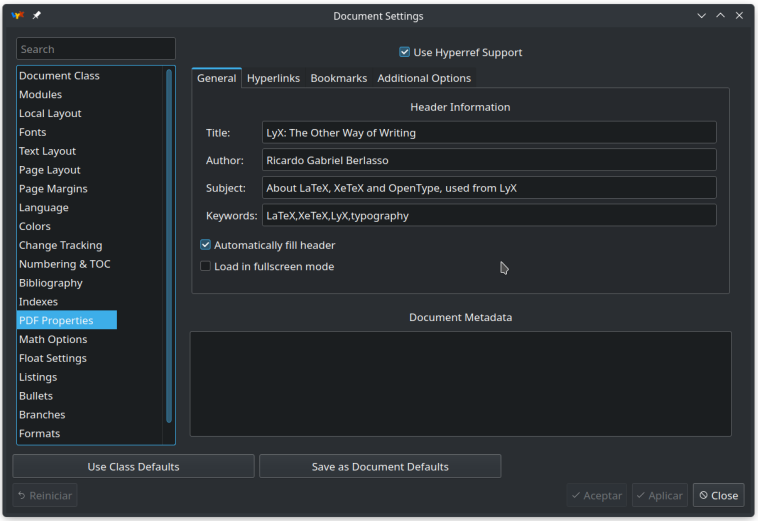


FIGURE 5.6: *Setting the* `Hyperref` *package*

or to external files.

LyX allows you to configure several options of the `Hyperref` package by simply selecting some check-boxes, while other options (such as choosing colors for the different kinds of links) can be configured by typing some parameters in a text line.

Opening Document » Settings » PDF Properties you will find something like what is shown in figure 5.6.

As you can see, in the GENERAL tab it is possible not only to enable support for the `Hyperref` package for that document, but also to set the general properties of the PDF that will be generated, such as the name, author, whether the PDF should open full screen, and so on.

From the HYPERLINKS tab you can access some options of the `Hyperref` package, such as coloring the links... but you will not see a graphical option to select the colors.

In the tab called ADDITIONAL OPTIONS is where you should write, separated by commas, those options of the `Hyperref` package that cannot be set graphically.

For example, after enabling Color links on the Hyperlinks tab you can determine the colors of each type of link using the following instructions:

`citecolor` sets the color of the links to the bibliography

`filecolor` sets the color for links to files

`linkcolor` sets the color for internal links (cross-references, table of contents)

`urlcolor` sets the color for web links

Colors can be indicated either by name (*red*, *green*, *cyan*, *magenta*, *black*) or by RGB values with three numbers between zero and one. I generally use the option No frames around links under Hyperlinks tab because I hate having links inside boxes, but if someone prefer to use those boxes their color can be set with instructions such as `linkbordercolor` and similar.

The `Hyperref` package documentation is usually installed in
/usr/share/texmf/doc/latex/hyperref/manual.pdf
By writing something like

```
citecolor=cyan,filecolor=green,linkcolor=red,
urlcolor=blue
```

you will obtain a PDF not only "linked," but also "colorful."

An important option to check in the Hyperlink tab is Break links over lines, otherwise a text containing a link could go on and cross the right margin with impunity.

In the Bookmarks tab you should never forget two really important options, necessary to obtain properly numbered PDFs: Generate bookmarks (TOC) and Numbered bookmarks.

Another important detail to note is that with the configuration indicated so far, in the table of contents only the titles will be converted to links, not the page numbers. By adding the `linktocpage` instruction in the Options box, the numbers will be converted to links, *but the titles will no longer be links*. To have both titles and page numbers as links (but not the "guide dots"), the option to use is `linktoc=all`.

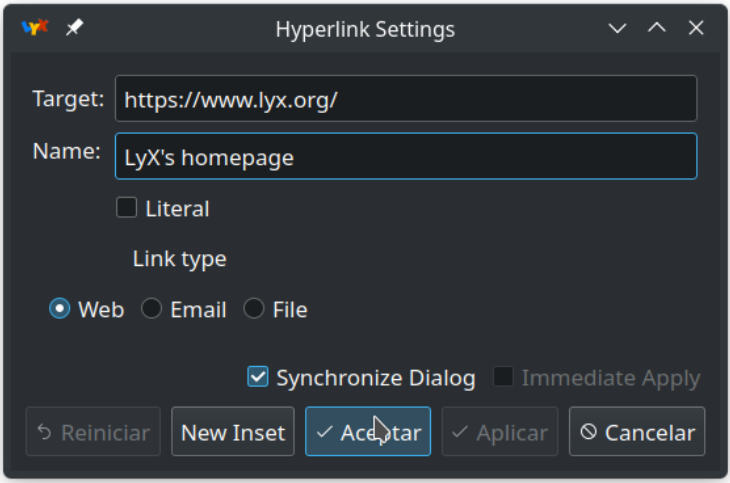


FIGURE 5.7: *Creating a link in a document*

To have *the whole* line as a link... well, it's not that simple. The possible "solutions" (let's say, dirty tricks) depend on the type of document used, so I will not talk about them in this book.

I told you, the configuration of indexes is *complicated*.

As last comment, remember that in LyX there is a graphical tool to insert external links (see figure 5.7), so there is no need to use TeX boxes for that.

5.1.11 Navigate and organize the document

From the Navigate » ... menu you have access to the whole content of the document: you can go from there quickly to any chapter, figure, table, footnote, equation...

The menu Navigate » Other lists » Labels and References is something you should not overlook. There you will have not only the list of all the tags used to mark something in the document, you will also have the list of all the *references* to those tags, allowing you to quickly see from where you are calling them.

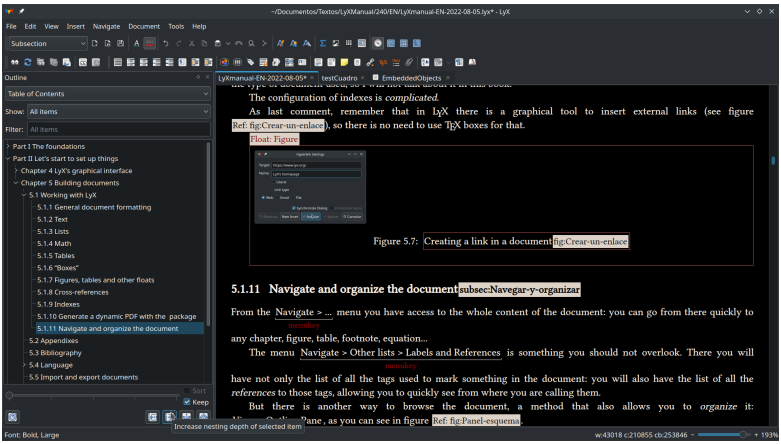


FIGURE 5.8: *The Outline Pane during the edition of this book. The KEEP checkbox is really useful when you need to go back and forth through the document while the four buttons at the bottom right allow you to quickly modify both the order and hierarchy of the chapters, sections and other elements of the document.*

But there is another way to browse the document, a method that also allows you to *organize* it: View > Outline Pane, as you can see in figure 5.8.

As the screenshot shows, at the top you can choose what type of information you want to browse or modify (the general table of contents, the list of figures, etc.). Below this option you have a field that allows you to filter the list that follows, which is organized hierarchically in a tree view.

It is important to note that when you click on an item in this list, the edit point moves to that item, *but the click also closes that branch in the hierarchy*. To prevent this from happening you have the KEEP button, which makes the branches only open/close by clicking on the arrows to the left of the corresponding entry.

The slider bar below serves to globally choose up to which level the branches will be open.

Finally, the buttons at the bottom right (which will only be available for the general index) allow you to change the level of a heading or to move it above or below another of the same level, allowing you to change the order

of the document entries quickly and easily.

5.2 Appendixes

With Document > Start Appendix Here you create a new part of the document, indicated in the LyX editing window with a red box saying "appendix" in the top line, which will contain the appendices.

To insert an appendix or a subappendix you simply use the Chapter and Section settings, respectively.

Cross-references to appendixes and everything related to them work without problems in the usual way.

It is important to keep in mind that, by default *(or by definition?)*, the appendixes go to the end of the document. It is *possible* to create appendixes "by chapter" or "by part," but I have not yet found a good enough reason to bother doing so.

5.3 Bibliography

In a LyX document there are two ways to work with bibliographic references: internal to the document or through the use of an external bibliographic database such as BibLaTeX.

LyX offers support for BibLaTeX, but going into such topics is really beyond the general purpose of this book so I will only give a brief introduction to the internal method. To learn about the other method, you can refer to section 6.6 of the user's guide.

Everything starts with the "bibliography" environment. After inserting it, you have a new section called *References* and the first empty entry with a gray label called `key-1[]`. With a click on that label you can change the key to something more descriptive or give it a name that will be used to display the citation in the document: if the key is left blank, the citation will be numbered.

It is important to keep in mind that this dialog does not accept spaces nor non ASCII characters.

Following the label is where you type the text of the reference, formatting it by hand (italics for the title, bold for the page number, etc.).

Finally, it is enough to go to Insert > Citation to select the citation(s) needed, their formatting, add text to the citation (e.g., [5, chapter 6]), and so on.

Two important things to note about references inserted in this way:

- Being an environment you can always leave the environment to add another environment as a chapter or section *after* it. This means that you could create a new reference group for each chapter or section of the document.

- The reference list will remain in the exact order in which it was created, so you need to be really careful and insert the entries in the correct order. Remember that you can always move paragraphs up or down with a simple right click. This problem is solved by using a database such as BibLaTeX, but as I said before, I will not go into this topic.

5.4 Language

In Document > Settings > Language you will find a few options, but all of them are really important. In this section I will show you what each of them does.

5.4.1 "Dynamic and 'nested' quotation marks"

Many languages, and English is no exception, can use nested quotation marks like the ones you see on this subsection heading. Each language has its own rules to establish which are the correct combinations[4] and from this menu LyX allows you to configure these combinations easily.

The external quotation marks are inserted as usual (in my Spanish keyboard, it is ⇧ 2 while in the English one is to the left of the ↵ key, I think), while in order to insert the inner marks you only need to add Alt to the key combination. By selecting Use dynamic quotation marks, the quotation marks will adapt automatically if you then change

4. https://en.wikipedia.org/wiki/Quotation_mark

the selected style globally: without checking this option, they will remain fixed.

With dynamic quotation marks active you can also change the quotation mark style manually for a particular case with a simple right-click.

5.4.2 Dictionaries

For all of us, Linux users, the dictionaries needed for spell checking are probably already installed (they will have been carried over from LibreOffice), but the thesaurus dictionaries may be missing. Or you might want to use other, more up-to-date dictionaries than those provided by your distribution. It may even happen that someone has to use LyX on other operating systems.

In this subsection you will see how to get updated dictionaries and how to use them in LyX.

First of all, you have to download the dictionaries, for example, from LibreOffice's extension repository,[5].

The next step is to rename these extensions from <dictionary>.oxt to <dictionary>.zip and then unzip them, which will give you several files and possibly some folders. From all this you should worry only about files with the following extensions:

- **.dic** and **.aff**, which are the files containing the spelling correction dictionaries
- **.dat** and **.idx**, the files containing the thesaurus.

Moving those files to an appropriate folder, now you need to open in LyX the menu Tools » Preferences » Paths as you can see in figure 5.9.

Now LyX will use the dictionaries you copy to the chosen directories. In the screenshot I have only changed the thesaurus folder, but changing the spelling dictionaries directory works the same way.

In the menu Tools » Preferences » Language Settings you can choose to have the spell checker mark the errors found in real time, while the keyboard shortcut ⇧ F7 will open the thesaurus on the word on which the cursor is located. It is important to note that if you have the .aff files, the thesaurus

5. https://extensions.libreoffice.org/extensions?getCategories=Dictionary

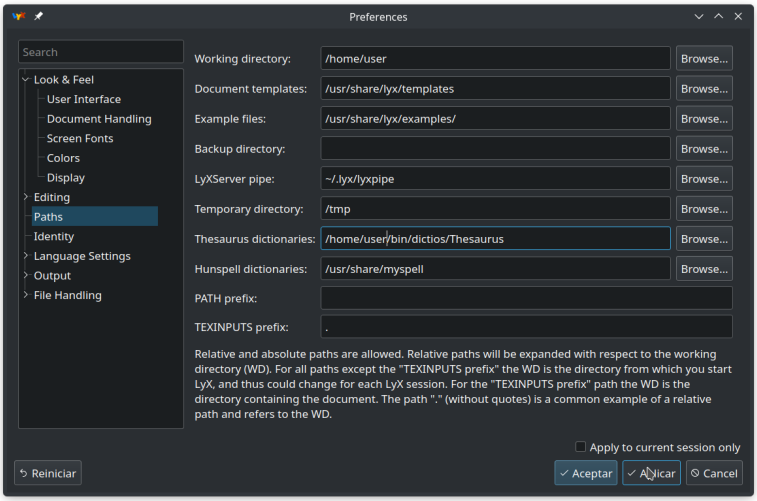


FIGURE 5.9: *Changing the paths to the dictionaries*

will recognize modified words (plurals, conjugated verbs, etc.), but will only give unmodified suggestions (it will offer singulars, infinitive verbs, etc.).

5.4.3 Babel vs. Poliglosia

Language definition affects much more than spell-checking. From simple things like translating "Chapter" into "Capítulo" for a document in Spanish, up to the difficult decision of getting rid of the first line indent after a heading (no, it is not only Americans: Spaniards also keep that first line indent) the language settings are responsible for many things in your documents.

Historically, LaTeX has always used an external program called Babel to make these decisions. The problem was that, also historically, Babel used to have problems with languages using non-Latin writing systems.

This situation worsened with the advent of Unicode and the need to support out of the box different scripts, which led to the emergence of an alternative project called Polyglossia. In fact, in documents using system fonts, Polyglossia is used by default instead of Babel.

However, not only in the latest versions Babel has solved almost all of its old problems, but obviously for English these problems never existed. As never existed for Spanish or Italian, for instance. If we add to this the fact that for some languages Babel offers many more options than Polyglossia it will be clear to you why in the following I will talk mostly about Babel, although some tricks that I will show you later will also work with Polyglossia.

5.4.4 Babel options in LaTeX/LyX

> NOTE: It is important to keep in mind that the English version of Babel will automatically recognize whether your document is set to British English, American English, Australian English, etc., so first line indentation and (if the corresponding dictionary is available) hyphenation will work automatically, smoothly and effortlessly.

Not of real use for English speakers, but for some languages Babel offers several options that can be set-up through parameters or additional instructions.

In particular, the Spanish version offers a lot of options, the Italian version a few less, and so on.

In Document › Settings › Language, it is possible to select CUSTOM as LANGUAGE PACKAGE and, in the box, write an instruction such as:

```
\usepackage[language,option1,option2]{babel}
```

where `option1`, `option2`... are the options we want.

For example, to set a document in Spanish in which the first line indent is dropped after a heading and where Roman numeral are written in small caps, you need to write

```
\usepackage[spanish,es-noindentfirst,
   es-ucroman]{babel}
```

Notice that's perfectly possible to select NONE instead of CUSTOM under language package and type the code directly in the LaTeX preamble of the document.

Most of the time you'll not need to touch those options, generally speaking the defaults are just perfect, but it's possible to use commands that call Babel to modify some characteristics of the document, such as the names of some elements, but we'll talk about those commands later, when they are needed.

You can see the documentation for the English version of the package at:

/usr/share/texmf/doc/generic/babel-english/english.pdf

5.5 Import and export documents

One of the most enduring myths in modern times is that of the *interoperability:* the compelling but crazy idea that you can work without trouble in different systems and different programs, but in a single document.

I won't go into why this interoperability thing is practically impossible outside the web world (it would take me too far away from the content of this book), but the sad reality is that if you work on a text document with one program and then want to switch your work to another system you will *always* have problems.

But, of course, that does not mean it is impossible.

LyX offers integration with `Pandoc`, a program which allows you to convert between different document formats. With `Pandoc` and other filters already present, it is possible to import and export to and from LyX all kind of documents formatted as odt, doc, docx, xhtml... Everything is under File » Import / Export.

In version 2.4 an option to export documents to ebook format has been added.

But don't expect miracles: if the documents are complex, some extra work will probably be necessary.

Obviously importing and exporting LaTeX documents works perfectly. In fact, if you copy a fragment of a LaTeX document to the clipboard you will have the option to do a special paste that will convert everything to LyX format effortlessly.

5.6 Copying information from other programs

Copying and pasting information between different programs has always been, is and will always be problematic and with LyX this is no exception. But difficult is not synonymous with impossible, so here are some little tricks to simplify the thankless task.

Pasting with Ctrl V in LyX from, for example, LibreOffice Writer means that in addition to losing all formatting, any paragraph or line breaks will be ignored.

With Ctrl ⇧ V paragraph breaks are instead retained.

Under the Edit › Paste Special menu you have a couple of other options, but beyond PASTE FROM LATEX (provided you *copied* from a LATEX document, of course) you should not put much hope in them. In fact, copying images or objects is almost impossible and you will always have to recreate them.

Something that *can* be copied (mostly) without problems are tables, either from HTML documents displayed in the web browser (easy), LibreOffice Calc or Calligra Sheets spreadsheets (just as easy) or even tables in word processors such as LibreOffice Writer (with some extra work).

Let's say that in an HTML document or in a spreadsheet you have a table of, say, 6 rows by 4 columns. After copying, you need to go to LyX and create an empty table with the same dimensions. Finally, with the cursor in the first cell in the upper left corner you just need to press Ctrl ⇧ V. And that's it. Formatting will be lost, but the *content* will go in the correct cells.

Copying a table from Writer has more problems. As in the previous case you have to create a table in LyX of the correct dimensions. Now you need to go back to the Writer document, click on a cell of say table and do Table › Convert › Table to text, selecting "tabs" as separator. Now it is enough to copy the resulting text, to go to the first cell in the LyX table and to press Ctrl ⇧ V.

Paradoxically, copying from LyX *to* Writer is simpler: environments are automatically mapped to appropriate paragraph styles (sections become "Heading 2," for example) while bold and italics are respected without problems. Even tables are copied correctly!

Now, sadly this will also generate lots and lots of sections (in Writer's sense of the word, of course[6]) that you will want to delete and the links to the images will be copied badly. Nothing that can't be fixed with a few clicks.

5.7 Change tracking

LyX provides a change tracking system that allows you to track the different stages of writing a document, which can be particularly useful when several people are working on the same file.

It all starts with Document » Change Tracking » Track changes, or with the keyboard shortcut Ctrl ⇧ E. Changes will be indicated depending on their type: underlined text to indicate what is new, strike-through to indicate what has been deleted, and so on.

When you activate version control you will have a new toolbar that allows you to accept or reject changes, navigate between them and all that stuff that's usual with such systems.

The system is quite easy to use and it is perfectly documented in the section 6.17 of the user guide found under Help » User's Guide.

Also, I must admit, it's not something I use a lot. You could say that as a writer I am a lone wolf... and disorganized.

5.8 Working with *branches*

Suppose that you want to create a document with variants that can be quickly selected, either to hide them or to show them in the final version. The simplest example is an exam with several questions and their corresponding answers: you want the students to see only the questions, but you also want to give the teachers who must correct the exam the complete document with all the questions and answers.

Or maybe you have to write a manual for a product with three similar but not identical models, so by selecting to display only what corresponds to

6. You can learn more about this topic here: https://frommindtotype.wordpress.com/writer-book/



FIGURE 5.10: *Our first branch. We will discuss the green color later.*

a certain model when you create a PDF you will get three different manuals from a single document.

LyX offers a very powerful tool to achieve this goal: *branches*.

In any document, it is possible to create a branch from the menu Insert › Branch › Insert New Branch. Some branch has to be the first one.

If you call the first branch "Questions," after accepting you will find something similar to what is shown in figure 5.10.

Do you notice the "✖" to the left of the label? I'll come back to that later.

In the red box you can now insert any kind of content: normal text, headings, figures, tables, equations...

If you now go back to Insert › Branch › Insert New Branch you will see that in addition to the possibility of creating another branch you also have the previously created branch.

And that is how everything works: you can have several separate blocks of the document, all of them grouped under the same branch so that when you hide that branch all those blocks are hidden simultaneously. In addition, you can have several branches that can be hidden or shown independently.

However, by default all the branches are disabled so if you compile the document they will not be shown. In fact, if all you have in the document are branches and all of them are disabled, *the document will not compile at all*. This is shown with the "✖" symbol to the left of the label I was talking about before. To activate or deactivate a branch, just right click on the tag of any of the blocks corresponding to the branch of interest, as shown in the figure 5.11.

Now the "Questions" branch shows a "✔" on the left indicating that it has been activated and will appear in the compiled document.

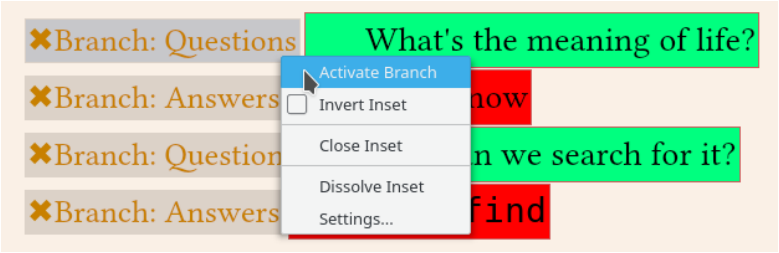


FIGURE 5.11: *Activating a branch*

In the menu just shown you have four more options:

INVERT INSET offers the possibility of "inverting" a particular inset belonging to a branch so that it is displayed only when the branch is *hidden*.

CLOSE INSET this option hides the content of that block, only the label will be visible.

DISSOLVE INSET the content of the box will be moved to the main document, cutting it from the branch.

SETTINGS takes you to a simple menu where it is possible to assign the block to another branch.

Another way to control the branches, not only to activate or deactivate them, but also to merge them, change their name, set a color (only seen during editing[7])... is to go to the menu Document » Settings » Branches.

And in this way, with a couple of clicks, you can go from the situation shown at the top of figure 5.12 (to the left the LyX document, to the right the compiled PDF) to what is seen at the bottom of the figure.

Cool, isn't it?

7. Now you know where the green color comes from

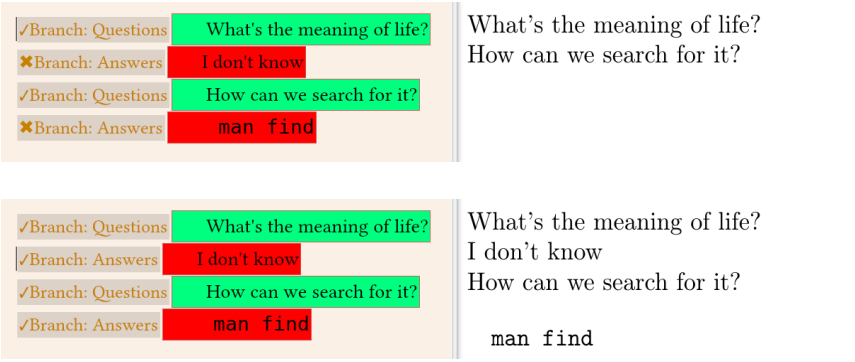


FIGURE 5.12: *Two branches in action, with only one (top) or both active (bottom). In both cases you have the LyX editing window on the left and a fragment of the PDF on the right.*

5.9 Master documents

In LyX it is incredible simple to split a document into sub-documents and to combine them into a master document. Working on each document, cross-referencing between them, it's all a breeze. I'll show you the available options below.

First of all it should be made clear that, unlike other applications such as LibreOffice Writer, there is no real difference between a master document and a child document: you could even have several levels of child documents, with a first document that functions as master, a second that is the first child and in this one a *grandchild* document. The important thing to note is that the *documentclass* that prevails will be that of the document at the highest level, so if in a sub-document you use some special feature (some module, for example, or a particular preamble code) you have to make sure that it is also enabled in the main document.

OK, let's get to work.

In the main document, with the cursor at the point where you want to insert the child document, you need to go to Insert » File » Child Document to get the menu partially shown in figure 5.13 on the next page.

This menu offers several ways to include a child document. With the

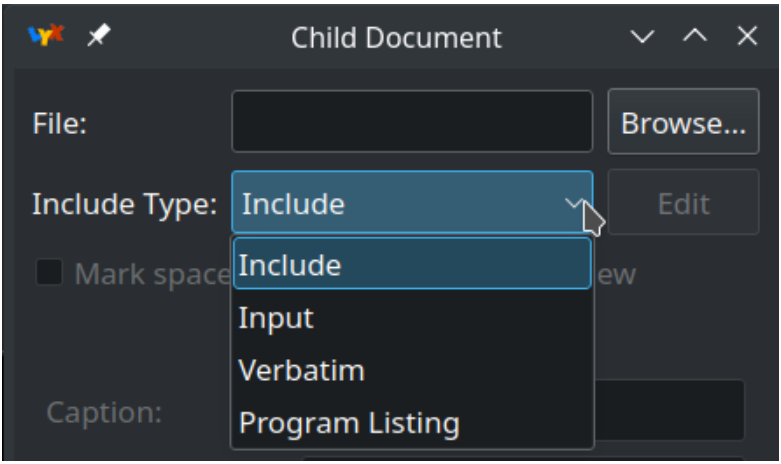


FIGURE 5.13: *Insert a child document*

first one, INCLUDE, the document will be represented in the editing window by a button centered on its own line: click on this button and you will return to the properties of the child document, where you have the EDIT button that will open it for editing. When you compile the master document, the file thus inserted will start from its own page and after it you will have another page break.

If you use INPUT instead, the document will go on the same page where it was inserted, starting on a new line.

The two last options, VERBATIM and PROGRAM LISTING, introduce the *code* of the child document. I leave them to your curiosity.

One really interesting thing if you have all documents open simultaneously, each in its own tab, is that the numbering of sections and chapters in the LyX edit window will follow the actual order of the whole document, giving you a clearer idea of the structure of the finished document.

Cross-referencing between different documents and sub-documents is incredibly simple if you have all documents open simultaneously: *all* bookmarks will be available from *all* documents.

And that's it. No, really, that's all there is to it: working with master and child (or grandchild) documents in LyX is just as easy as working in a

single file that has everything.

Super cool.

There is more information in section 3.2 of the manual found under Help 〉 Additional Features. Also here:

https://wiki.lyx.org/FAQ/Multidoc

Part III

Beyond the basic settings

Where the full power of LaTeX, XeTeX, OpenType and LyX is unleashed

It's full of stars!

Chapter 6
Some commands

LyX cannot provide support for each and every aspect of LaTeX in its graphical interface, so to get past certain formatting boundaries there will be no choice but to roll up your sleeves and start giving instructions in pure LaTeX, either in the preamble or in the body of the document.

This sound more difficult than it really is, so do not panic just because you have to write some lines of weird looking code. But of course do not relax too much either, because doing things without understanding how they work is an invitation to disaster.

In this chapter I'll show you how to insert code into the document as well as give a quick overview of some of those instructions that have a habit of being necessary many times.

6.1 The LaTeX preamble

We have already talked discussed the importance of the LaTeX preamble in the introduction of this book: is the place where you can define things that can be used throughout the document.

It is accessible from Document > Settings > LaTeX Preamble, and there you can write all the code we will see in the following chapters.

An instruction that will be repeated many times is the one that activates a particular package from the LaTeX distribution. For example, in order to insert the XeTeX logo I had to activate the `hologo` package, for which I wrote in the preamble:

```
\usepackage{hologo}
```

As you can see, an instruction with a clear name (`\usepackage`) is used to tell the system to use a particular package (`hologo`).

It is interesting to note that *comments*, i.e. texts that are inserted in the code that should not be executed, are marked with a `%` sign.

Thus, if you write

```
\usepackage{hologo} % Activate the package
```

everything to the right of the `%` sign will be ignored.

Some packages offer the possibility to adjust different parameters to configure their behavior. This can be done directly when calling the package or by using special instructions that will be available only if the package has been activated, while other packages offer, or even require, *both* possibilities. In some cases you will have to *patch* a package with the `\renewcommand` instruction and will even be forced to use logical statements to determine when (or if) a change should be made.

Unfortunately, not all instructions are so easy to understand: for example, `\makeatletter`, can easily create confusion in someone who is new to all this.

To simplify things a bit (or a lot), the character @ *(at)* is special in LaTeX... but only sometimes. When redefining a command that uses @ in its name, `\makeatletter` *(make @ a letter)* transforms @ into a "normal" (that's just one way of putting it) character, while `\makeatother` *(make @ other)* turn it back to "what LaTeX says it is."

Anyway, nothing to worry too much about since I won't go *that* deep into these things, but as you can see we are facing a really complex world in which it is necessary to move with a firm foot.

6.2 The TEX box

Either with the corresponding button on the main toolbar or more simply with the keyboard shortcut Ctrl L you will find yourself in front of a red box waiting for code.

Continuing with the `hologo` example, to make the XeTeX logo appear you will have to type inside one of these boxes the command:

```
\hologo{XeTeX}
```

which in the LyX edit window would look something like this:
`\hologo{XeTeX}`

> NOTE: In this book I have used a character style instead of writing the instruction each time. I will come back to this in the chapter 9 and in the appendix B.

This boxes can accept several lines of code and all the necessary instructions, so you are not limited to small expressions.

6.3 A matter of size

There are LaTeX instructions for defining text size. These instructions can be seen, ordered from the smallest to the largest, in the table 6.1 where the size values for the different document classes and for some values of the base text size are presented.[1].

As you can see, all these sizes are fixed, proportional to the base size defined for the document. And that means that although you have a wide *variety* that will cover almost any imaginable situation, it does not give you absolute *freedom*. To obtain arbitrary font sizes there is the command

```
\fontsize{font-size}{line-heigh}\selectfont
```

If you don't specify a unit, the command assumes that you are talking about "points." Now, I will come back to line heights in section 7.9, but to make it clear that this has its twists... imagine that in any given paragraph you write

Normal text `{\fontsize{15}{30}\selectfont` Big text `}`

That is, you apply the instruction to only a small part of the text (hence the extra opening and closing braces). Well, no matter what you put in the line height (as long as you put something in), *the number will be ignored*.

1. A technical detail: LaTeX uses a definition of "point" (the "TeX point") different from the standard one (or "PostScript point"). The difference is really small, below the 0,4%, but exists:
https://en.wikipedia.org/wiki/Point_(typography)#American_points

Size	Standard classes, *beamer*			$\mathcal{AMS}$ classes, *memoir*		
	[10pt]	[11pt]	[12pt]	[10pt]	[11pt]	[12pt]
`\tiny`	5	6	6	6	7	8
`\scriptsize`	7	8	8	7	8	9
`\footnotesize`	8	9	10	8	9	10
`\small`	9	10	10.95	9	10	10.95
`\normalsize`	10	10.95	12	10	10.95	12
`\large`	12	12	14.4	10.95	12	14.4
`\Large`	14.4	14.4	17.28	12	14.4	17.28
`\LARGE`	17.28	17.28	20.74	14.4	17.28	20.74
`\huge`	20.74	20.74	24.88	17.28	20.74	24.88
`\Huge`	24.88	24.88	24.88	20.74	24.88	24.88

TABLE 6.1: *Size values used by the different instructions as a function of the base size for the text. The classes for generating slides has been ignored in this table. Source:* https://en.wikibooks.org/wiki/LaTeX/Fonts#Sizing_text

Now, if you make it clear that the instruction applies to the whole paragraph by putting each thing on separate lines

```
{\fontsize{15}{30}\selectfont
A very large and really, really, really, really, really, really, really, really, really, really, really, really long text!
}
```

then your choice of line spacing will be taken into account!

To know more:

https://en.wikibooks.org/wiki/LaTeX/Fonts#Sizing_text

6.4 A matter of (typographical) shape

I will talk *a lot* about typography in the next chapter, but sometimes it is not necessary to go into so many detail. Many times it is enough to indicate the *variant* of the predefined fonts that you want. This will be really useful in chapter 12.

As we have already seen, in a LaTeX document there are three font fam-

ilies: serif or "roman," sans serif and "typewriter."

Each of these font families can have a "weight" that is indicated by a "series" (bold, semi-bold, light) or a "shape" (normal, italic, slanted, small caps, etc.).

Each of these families and each of these shapes can be called with a specific LaTeX instruction. Thus you have three types of instructions,

```
\XYfamily
\XYseries
\XYshape
```

where `XY` are two characters that represents the feature you are looking for.

The families are represented by

- `rm` for Roman: `\rmfamily`
- `sf` for sans serif: `\sffamily`
- `tt` for typewriter: `\ttfamily`

The "series" are represented by

- `bf` for bold: `\bfseries`
- `md` for medium: `\mdseries`
- `lf` for light: `\lfseries`

And last, the "shapes" (I'll show you only a few of them) are represented by

- `it` for Italic: `\itshape`
- `sl` for slanted: `\slshape`
- `sc` for Small Caps: `\scshape`

For a complete list of these commands:
https://en.wikibooks.org/wiki/LaTeX/Fonts#Font_styles

6.5 Numbers!

You will undoubtedly be faced with the situation of having to choose between numbering with Arabic numerals, Roman numerals or letters. Each type of numbering is associated with an instruction:

- `arabic` for Arabic numerals: 1, 2, 3, 4...
- `Roman` for uppercase Roman numerals: I, II, III, IV...
- `roman` for lowercase Roman numerals: i, ii, iii, iv... (poor Romans!)
- `alph` for lowercase letters: a, b, c, d...
- `Alph` for uppercase letters: A, B, C, D...

I will show you in the examples below how all this is used, don't worry.

6.6 Reconfigure!

In the following chapters I will introduce several LaTeX packages that will be necessary to carry out all the tricks that will fill the remaining pages of this book. Many of these packages come preinstalled in LaTeX distributions, but not all of them.

However, it is not enough to install a package to be able to use it from LyX: you have to tell the later that the package has been installed. To do this, after modifying anything in the LaTeX installation you should always go to Tools » Reconfigure, give LyX a few seconds to take a good look at things and restart the program when it asks you nicely to do so.

Chapter 7

LyX, OpenType and typography

Before starting, an important note: in this chapter and those that follows I will make extensive use of the "TEX boxes" and the LATEX preamble since there will be no alternative to using direct code. But don't worry, it won't be that difficult. In fact, later in this book (chapter 9) I will show you how to simplify everything even more with the creation of styles through the use of what LyX call *local format*.

OK, back to the main topic of this chapter: you already have an OpenType font defined for the document, but you want to go beyond the default options of the chosen typeface and activate some specialties.

This can be done at different levels:

1. few and small text fragments,
2. many and perhaps not so small text fragments,
3. the entire document,
4. mathematical formulas.

In this chapter I will talk about these possibilities, but also about some more general topics such as line spacing, hyphenation, and microtypography.

7.1 Introduction

In a LaTeX/XeTeX document three main font types are defined: the actual main font (called Roman in LyX), a sans serif font and a Typewriter font.

Even if LyX hides all this behind the Document > Settings > Fonts menu, in a LaTeX document the main font is defined in the preamble of the document with the `\setmainfont` statement while the sans serif font is defined with the `\setsansfont` statement and the typewriter font is defined with `\setmonofont`.

Remember these instructions, later it will be necessary to call them up explicitly.

For now, go to Document > Settings > Fonts and check the Use non-TeX fonts (via XeTeX/LuaTeX), picking the base size and the three main fonts as we've seen in previous chapters. Don't worry about the other options available there, we'll come back to them later.

To use OpenType features you have to know how to call each of its substitution tables. In what follows in this chapter and in the rest of the book I will take as reference Dario Taraborelli's page:

http://nitens.org/taraborelli/TeXOpenType

and the wikipedia page:

https://en.wikipedia.org/wiki/List_of_typographic_features

but you should be aware of the fact that these pages are missing some information that will be presented later on.

As an example, in the table 7.1 you can see some of the more used tags. It is important to keep in mind that not all fonts offer all these options. Also, some of the most common options are enabled by default (such as typographic ligatures) while others must be activated by hand. Finally, some of the options are incompatible with each other: it does not make sense to apply superscripts and subscripts simultaneously.

Now, this is not the whole story: some of these tags also accept parameters. But as this topic has its twists and turns, I will come back to it later.

Feature	Label	Name in Fontspec
Numbers		
Lining	`lnum`	`Numbers={Lining}`
Old style	`onum`	`Numbers={Oldstyle}`
Proportional	`pnum`	`Numbers={Proportional}`
Tabular	`tnum`	`Numbers={Monospaced}`
Slashed zero	`zero`	`Numbers={SlashedZero}`
Ligatures		
Standard	`liga`	`Ligatures=Common`
Contextual	`clig`	`Ligatures=Contextual`
Discretional	`dlig`	`Ligatures=Rare`
Historical	`hlig`	`Ligatures=Historical`
Forms, styles, position and variants		
Small caps	`smcp`	`Letters=SmallCaps`
Superindex	`sups`	`VerticalPosition=Superior`
Subindex	`subs`	`VerticalPosition=Inferior`
Scientific inferiors	`sinf`	`VerticalPosition=ScientificInferior`
Ordinals	`ordn`	`VerticalPosition=Ordinal`
Stylistics sets	`ss01...ss20`	`StylisticSet=1...20`
Stylistics alternates	`salt`	`Style=Alternate`
Swash	`swsh`	`Style=Swash`
Fractions		
Numerator	`numr`	`VerticalPosition=Numerator`
Denominator	`dnom`	`VerticalPosition=Denominator`
Fractions	`frac`	`Fractions=On`
Alternate fractions	`afrc`	`Fractions=Alternate`

TABLE 7.1: *Some OpenType tags and their use*

7.2 OpenType for the default font

Let's say you want to activate a special option for the default font but only at a particular point in the document. For example, and to impress your readers, you might want to write a short text using *discretionary ligatures* (actual historical standard instead of actual historical standard), or perhaps you need to write a fraction such as $^{123}/_{37}$ but don't want to use the equation editor.

Let me insist that later on you will see you how to define character styles to simplify the use of these instructions. But you still have to learn a couple of things (well, more than a couple) first, so there's no alternative, for now you'll have to write some code.

So, the idea is to combine the `addfontfeature` instruction with the name of the OpenType feature you need, for which you can use two TEX boxes surrounding the text to be modified. For example, discretionary ligatures can be set with

`{\addfontfeature{RawFeature={+dlig}}` historical `}`

(the plus sign "+" *activates* the option `dlig`) or

`{\addfontfeature{Ligatures=Rare}` historical `}`

to obtain, in both cases, the same result: historical.

The brace that opens the first TEX box and the lone one on the second are both important because they ensure that the code will be applied only to the text you are interested in and not to everything that follows.

Clearly, it is possible to activate several options at the same time: just separate each of them with commas. For example

```
\addfontfeature{RawFeature={+dlig,+onum}}
```

will simultaneously enable discretionary ligatures and old style numerals.

It is important to note that these instructions can also be used to *disable* features that are enabled by default. For example you can disable contextual ligatures, either with

```
\addfontfeature{Ligatures=NoContextual}
```

or with

```
\addfontfeature{RawFeature={-clig}}
```

(the minus sign "-" *disable* the option "clig").

From now on, to identify any OpenType feature I'll use the corresponding tags and not the Fontspec names. Just to write a little less.

Coming back to the topic, here we get into a rather technical problem that in most cases will be of no interest to anyone, but for a couple of particularly complete fonts may generate a bit of confusion: the use of parameters to OpenType options.

As far as I know what I'm going to explain bellow applies to very few fonts (that I know of, fonts from SIL International and Junicode), so the last part of this section is a bit more *specialized*, but you know, to get the full picture. I mean, it's awesome, you *must* know about it.

According to the standard, you can add an equal sign to a label and after it a number. So, always according to the standard, you can have `+etiq=0`, `+etiq=1`, `+etiq=2` and so on. These labels can activate different options, when available. Always according to the standard, the value "zero" *disables* the option indicated by the label (it does the same as the minus), while the 1 applies the first option, and so on.

The problem is that `fontspec` does not follow the standard as closely as does, for example, LibreOffice Writer: with `fontspec` you can only disable the option with the minus, while the zero applies the first option, the one the second, and so on.

It is clear that the people who did this were programmers. Anyway.

In the table 7.2 I propose you an example with options applied to the font Gentium Plus and the character Ĳ (U+014A), which in this font offers four variants, all applicable with option `cv43`.

Let me insist on the fact that there are very few fonts that offer multiple parameters for a tag, and even within those few fonts there are only a few tags that give more than one variant, but you never know when you might need them.

7.3 Defining a font family

What I told you in the previous section is relatively good only if you have to do it a couple of times: writing all that code over and over again is certainly

Standard	Result	fontspec
cv43=0	ŋ (no change)	-cv43
cv43=1	ŋ	+cv43=0
cv43=2	ɴ	+cv43=1
cv43=3	ŋ	+cv43=2

TABLE 7.2: *Example of the use of parameters in OpenType options with the Gentium Plus font. With fontspec, you must place the corresponding option inside the "RawFeature," while with Writer, which follows the standard... well, I'll tell you about that elsewhere, this is a book on LyX after all.*

annoying.

For this reason, and others that we will see later, it is a good idea to define an extra font family that has the desired feature. In addition, you will not be limited to the predefined font of the document.

As an example I will now define a new font based on EB Garamond, but with discretionary ligatures enabled. In the menu Document › Settings › LaTeX Preamble it is enough to write the following code:

```
\newfontfamily{\discrlig}[RawFeature={+dlig}]
   {EB Garamond}
```

To use this definition with pure code you have to do something similar to what I discussed in the previous section (clearly, writing less), that is, use two TEX boxes surrounding the text to be modified:

`{\discrlig{` historical`}}`

Don't forget the double braces at the end!

And well, yes, this is simpler than the above, but still not ideal. Don't worry I'll come back to this in chapter 9 where everything will become almost idyllic.

7.4 OpenType for the entire document

Let's suppose that you want your document to use the EB Garamond font by default, but with the stylistic set 6 (`+ss06`) enabled.

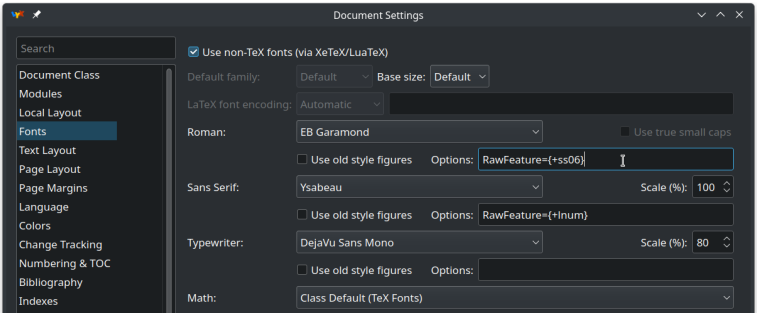


FIGURE 7.1: *Setting OpenType features for the whole document*

Since version 2.4, LyX offers the possibility to pass OpenType options to the three main fonts directly from the menu where you select said fonts: just bellow each of the fonts you can choose not only to use old style numerals, but also you have a box called OPTIONS where it's possible to write the needed instructions. For example, writing there

```
RawFeature={+ss06}
```

for the Roman font, everything will be ready.

If you want another feature simply change `+ss06` to whatever is appropriate within the braces following RawFeature. If you want several features simultaneously, it is enough to write them all inside the same braces, separated by commas.

7.5 Different fonts for different shapes

From this section on, the font configuration menu will be too small for us, so we will now write everything in the preamble.

You will hardly ever need what I tell you in this section, but since we are here I'll tell you anyway, so you can get an idea of the power of the system.

Imagine that you have a font that you like a lot, but that only offers the "regular" version, no italic or bold are available. For example, it could be Doulos SIL. Imagine also that you realize that the Stix Two Text font is quite similar —they are both Times style fonts— and that for some reason

instead of using Stix Two directly as a normal person would do, you want to use Doulos SIL for the regular and Stix Two Text for everything else. I don't recommend you to do it, remember that this is just an example, but it serves to see how to define a "virtual font" that joins several different fonts for the different shapes. To the point. After checking the use of non-TEX fonts in the fonts menu, and leaving the Roman font in DEFAULT, go to the preamble to write the following:

```
\setmainfont[BoldFont = STIX Two Text Bold,
             ItalicFont = STIX Two Text Italic,
             BoldItalicFont = STIX Two Text
                              Bold Italic,
             Mapping = tex-text]{Doulos SIL}
```

7.6 Different fonts for different sizes

Attention, the example in this section might even be useful!

Suppose we have a document configured with a font size of 10 pts. As you can see in table 6.1, the font size used for footnotes will be 8 pts. Now, reducing a font also reduces how "thick" its outline is, resulting in text that appears lighter than the body text, which is not great, to be honest.

To solve this problem some fonts come with "optical sizes," versions of the typeface specifically designed for the different sizes. Fonts of this type are, for example, Latin Modern, TeX Gyre, and so on. XƎTEX supports this type of fonts, automatically choosing the appropriate size in each case, which is very good... The problem is that only a few fonts provide this option.

Do not despair. While fonts with optical sizes are few, fonts with many variants of weight are more common and it is possible to use those variants to simulate optical sizes.

Let's take an example. EB Garamond, the font used in this document, in its previous incarnation offered a font designed for 12 pts and another designed for 8 pts, which is fine, but it did not come with neither bold nor bold italic. Octavio Pardo improved the font by adding not only bold, but also a medium, a semi-bold and an extra-bold variants, all with their corresponding italics, which is great; problem is, this new incarnation of EB Garamond

no longer gives the version intended for 8 pts, the "regular" form is based on the old font for 12 pts. and there is no more.

Now, it turns out that the medium weight version of the new EB Garamond has the same "color," or stroke width, as the old 8 pt. version, so you might ask, as I have done in constructing this document, is it possible to decide which font to use for each size range?

Of course! Again you choose to use non-TEX fonts, but leave the Roman one in DEFAULT and type the following in the preamble:

```
\setmainfont{EB Garamond}[
   Mapping=tex-text,
   RawFeature={+ss06},
UprightFont = EB Garamond Regular,
BoldFont = EB Garamond SemiBold,
ItalicFont = EB Garamond Italic,
BoldItalicFont = EB Garamond SemiBold Italic,
UprightFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Medium},
 {Size = {10-}, Font = EB Garamond}}},
BoldFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Bold},
 {Size = {10-}, Font = EB Garamond SemiBold}}},
ItalicFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Medium Italic},
 {Size = {10-}, Font = EB Garamond Italic}}},
BoldItalicFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Bold Italic},
 {Size = {10-}, Font = EB Garamond SemiBold
                                 Italic}}},
]
```

In this code example (by now, you know where to write it), after saying which font to use for each shape I decide which one to use for the different size ranges: up to 10 pts and from 10 pts onward.

If you want an even more detailed setting, you can always use an intermediate range, for example from 10 to 14, by saying `Size = {10-14}`.

The range includes the initial value, but not the final one. Mathematically speaking, if x is the size, then $10 \leq x < 14$ (see section 6 of `fontspec` manual)

7.7 Different OpenType options for each shape

I'm back with another crazy example to show the power of it all. I am now going to activate some OpenType features for normal font, some different ones for italic, some for bold... For this, to the instructions introduced in previous sections you have to add a bit of code.

To make things clear I show you a completely arbitrary and absurd example where for the Vollkorn font I activate the first stylistic set only for normal text, the stylistic set 17 only for italic, 14 only for bold and the "zero" option (the crossed out zero) only for bold italic:

```
\setmainfont{Vollkorn}[
   UprightFeatures={
      RawFeature=+ss01,
   },
   ItalicFeatures={
      RawFeature=+ss17,
   },
   BoldFeatures={
      RawFeature=+ss14,
   },
   BoldItalicFeatures={
      RawFeature=+zero,
   },
]
```

Pay attention to the commas!

I know, I know. It's just for the sake of a clear example!

To add more options for each face, simply add them separated by more commas:

```
ItalicFeatures={
```

```
        RawFeature=+hlig,
        RawFeature=+swsh,
        Numbers=OldStyle,
    },
```

You might even decide that a certain shape should have a different color... but for that I'll let you explore section III-6.1 of the `fontspec` documentation, which is installed at

/usr/share/texmf/doc/latex/fontspec/fontspec.pdf

7.8 OpenType for math: `unicode-math`

To complete the migration to OpenType fonts, it is of course necessary to change the fonts used in mathematical formulas as well.

This is simple to accomplish in LyX thanks to the `unicode-math` package and several projects that offer fonts specially designed for this purpose, such as STIX Two Math, Garamond Math, Libertinus Math, Asana Math, etc.

It is important to note that these special fonts are not intended for writing normal text, they can only be used to create mathematical expressions. They achieve this by including additional OpenType information that can be exploited by some equation editors.

If you wish, you can check the unicode-math tag of my personal blog, where I review all (well, not all: only those I liked) the OpenType mathematical fonts I have found so far:

https://frommindtotype.wordpress.com/tag/unicodemath/

To use these fonts, and this package, in LyX it is necessary to follow two small steps, one graphical and the other "through code."

First of all you need to go to Document > Settings > Fonts > Math where in the drop-down menu you need to select the cryptic Non-TeX Fonts Default. Now, in Document > Settings > LaTeX Preamble you have to write the following to use the STIX Two fonts:

```
\setmathfont{STIX Two Math}
```

For Libertinus Math

```
\setmathfont{Libertinus Math}
```

Or for DejaVu (maybe someone like it...)

```
\setmathfont{DejaVu Math TeX Gyre}
```

If any of these fonts offer stylistic sets or other OpenType specialties it is possible to activate them when calling the font. For example to use the alternate glyphs of some letters and get more vertical integrals with STIX Two Math (stylistic set 2 and 8, respectively, see the font documentation for more information) you can type

```
\setmathfont{STIX Two Math}[RawFeature={+ss02;+ss08}]
```

Another thing you can do is to choose different fonts for different *ranges*. For example you could insist on using Latin Modern Math for the calligraphic characters even though the main math font is another one. Simply type

```
\setmathfont[range={\mathcal,
            \mathbfcal}]{Latin Modern Math}
```

One point about this: by doing this you *may* encounter spacing problems between the characters in the formula. To solve this it is sufficient to add another line by setting an "empty range" for the base mathematical font:

```
\setmathfont[range={}]{STIX Two Math}
```

That's it, even our equations now use OpenType fonts!

Just remember to install said font first!

On Linux and with TEXLive, the documentation for `unicode-math` is installed in

/usr/share/texmf/doc/latex/unicode-math/unicode-math.pdf

Now, the number of mathematical fonts is large, yes, but not huge. What if we want to use for the document a font that does not offer a mathematical version? No problem, since it is possible to combine mathematical and normal fonts when writing our equations.

In this the `unicode-math` documentation is not exactly user friendly, so let's see how to do it with a concrete example.

Imagine you have a document that uses Cochineal[1] as its base font (the one called *Roman* by LyX). This is a "normal" font without the OpenType math table that allows you to write equations. Well, using the `range` option discussed above, it is possible to choose a mathematical font for the symbols (integrals, summations) and a normal font for the text (variables, functions, Greek characters). By typing in the preamble

```
\setmathfont{STIX Two Math}         % the symbols
\setmathfont[range=up]{Cochineal}     % the text
\setmathfont[range=it]{Cochineal Italic}
```

equations will use the mathematical font STIX Two Math for symbols, the font Cochineal for normal text and Cochineal Italic for italic text.

It is possible to do even more complicated things, such as choosing different fonts for different alphabets, but in most cases the above is more than enough.

For more information you can consult these threads

https://tex.stackexchange.com/questions/619785/what-do-the-unicode-math-range-labels-mean

https://tex.stackexchange.com/questions/58098/what-are-all-the-font-styles-i-can-use-in-math-mode/437882#437882

And while we are at it, one last surprise of this `unicode-math` package: the instructions discussed in this section can be used not only in the preamble, but also in the body of the document.

This means that you can change the math font used at any time and as many times as you want. For example, for a particular section you could switch from Garamond Math to STIX Two Math by simply adding the corresponding code in a TeX box. I'll let you explore the details.

1. http://www.ctan.org/pkg/cochineal

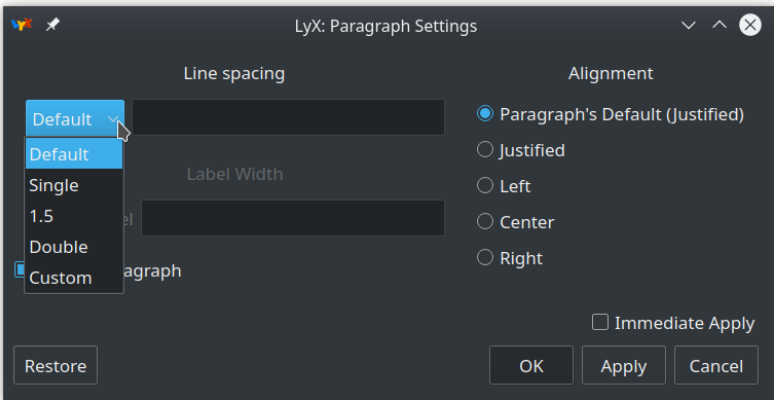


FIGURE 7.2: *Selecting the line spacing in a particular paragraph*

7.9 Line spacing

Here is a dilemma: both for the document as a whole, as well as for a particular paragraph, LyX offers several line spacing options. The main three ones (Single, OneHalf in the document settings or "1.5" in the paragraph settings —this double naming for the same function is crucial— and Double) seem to be clear and understandable enough, but only to the point in where we realize that we have no idea what "Single" actually means.

The issue is not minor and has given and will give many headaches to most LaTeX users, and not only to newcomers.

Let's perform some tests to understand how all this works.

If on a paragraph you do a right click » Paragraph Settings you will get the menu shown in figure 7.2.

Well, now try the following: create a text with line spacing of "1.5" and another with a *custom* spacing of 1.5. After compiling the document, can you see the same thing? Do not? *Custom* at 1.5 gives a bigger line spacing than the "1.5" option! Now, go back to the paragraph settings, select CUSTOM again and type 1.25: you will see how when you press APPLY the option changes by itself to 1.5. Finally select CUSTOM again and type 1.667: now it changes to "double." What about this?

Modern fonts include the definition of what a "single spacing" actually

is: the person who designs the font chooses the minimum space between lines so that they do not collide with each other. This optimal size depends on the design of the characters (their height, the way in which accents are stacked —yes, there are languages in which the same letter can have several accents simultaneously), the author's personal preferences... Word processors such as LibreOffice Writer use this internal font parameter as the basis for line spacing.

Now, the thing about LaTeX is that it was created decades before Unicode, OpenType and all that. And that is why LaTeX calculates the "single spacing" in its own way.

The thing goes more or less like this: LaTeX defines the basic line height as the size of the font. That means that a 10 pts font will have a 10 points line height, and that is why "one and a half" (or 1.5, depending on which menu we look at) will be 15 pts, while double will be 20 pts. BUT, the single line spacing is 12 pts, not 10!

Yes, that's right. Since having a line spacing equal to the font size will give text that is too compact and unreadable, where the taller characters of a line will collide with the lowest letters of the previous one, LaTeX automatically adds a 20% to that base height, *but only to the single line spacing*, "one and a half" and "double" are calculated *without* this increase in size! And to make matters even worst, the custom line spacing is a factor over the single line spacing, which has been increased!

Now I will ask you to make the effort to remember the time at school when you worked with proportions and percentages. If you know that a quantity (the distance between two lines with "single spacing") is the 120% of an unknown value and you want to calculate the 150% of that unknown value, what is the factor you need? Well

$\frac{1}{1,2} \cdot 1,5 = 1,25$

And if what you want is the double of that value, then

$\frac{1}{1,2} \cdot 2 \approx 1,667$

I can tell you that LaTeX knows how to be strange...

As you can see, the confusion arises from the fact that the options "one and a half" (or "1.5") and "double" are defined from a different value than the one used for "custom."

And before you ask, yes, all this affects XeTeX and LuaTeX too.

More information about this can be found at:
http://wiki.lyx.org/FAQ/Spacing
http://tex.stackexchange.com/questions/13742/what-does-double-spacing-mean
https://en.wikipedia.org/wiki/Leading

7.10 Hyphenation

When in a document you make the text "justified" (i.e., you force both margins, left and right, to form a straight line) you face a problem: if the text is distributed in columns or the font used so as to have few words per line, the software you use will be forced to vary the distance between words to fit the margins, which will give white spaces of unpredictable size that will make reading uncomfortable.

To avoid this, the fist line of action (we will see others in the next section) is to rely on the hyphenation of words at the end of the line in order to reduce those spaces... at the price of "cutting" many words.

In general, LaTeX do a great job finding a balance between the spaces and the number of hyphenated words, but only when the text area is "broad enough." Is it possible to manually change how LaTeX controls this feature? Well, yes, but that does not means you should try.

Anyway, let's see how LaTeX works here.

LaTeX uses a complex algorithm to decide how to distribute the text on the lines of a paragraph. Simplifying the process a little *(or a lot?)*, we could say that a parameter that measures "how bad a text looks like" is calculated and then LaTeX tries to minimize that parameter in two attempts. In the first attempt it is calculated how bad the text looks like without hyphenation and if the value is good, then everything is fine. If the result is not good (and usually is never good), it repeat the process allowing hyphenation at the end of the lines.

The "limiting value" that separates the "looking good" from the "looking ugly" is called *tolerance*. And since the process runs twice, we have two tolerances, one for the first attempt and the other for the second.

These tolerances are controlled through the following instructions, to be written in the LaTeX preamble:

```
\pretolerance=<value>
\tolerance=<value>
```

where <value> is an integer between 0 (aggressive hyphenation) and 10000 (no hyphenation).

And is in that "value" where the whole process became a sort of "magic:" the correct values for `pretolerance` and `tolerance` will depend on the page size, the font, margins, number of columns... In short, finding them is a matter of trial and error. Lots of errors. The narrower the text is, the less stable these parameters are, which means that the more we need them, the more difficult it will be to find them.

Fortunately, most of the time LaTeX hyphenates well, so don't worry about this, it is not worth the effort.

By the way, if in the PDF output you find a word wrongly hyphenated, it is possible to manually select the hyphenation point with Insert > Formatting > Hyphenation Point or Ctrl - . This is particularly important to know when writing in English, because LaTeX tends to not hyphenate words that contain dashes: you need to manually hyphenate them.

7.11 Micro-typography

First of all, a warning: the subject of this section is purely about aesthetics. What I'm going to talk about is how to make text created in LyX, either with LaTeX or XeTeX, look more beautiful. It is of no use by itself and in fact most readers will not notice what we will do... which will mean that we have done it right!

The idea is precisely to subtly modify the text so that it "looks better," but so that whoever sees it will not be able, in a first analysis, to tell what was modified. And how is this achieved?

Well, first of all we must make an extremely strong statement: it is terribly simple to fool our brain. For proof, it is more than enough to review the article on optical illusions in wikipedia:

https://en.wikipedia.org/wiki/Optical_illusion

And it is precisely a kind of optical illusion what we face when we see a normally justified text: every now and then you get the impression that the

right margin is wavy, with entries. Just where you have punctuation marks and hyphens it looks as if the line is receding, even if this is not the case (you can check it with a ruler).

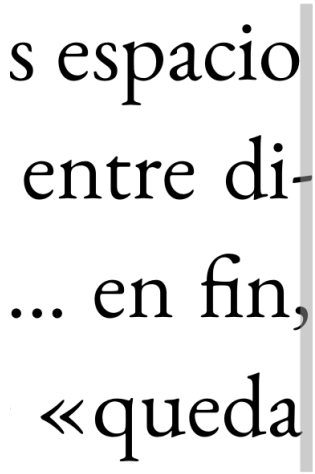


FIGURE 7.3: *Protrusion example*

What happens here is that the periods, commas and hyphens have *less weight* than the characters before them, so our brain interprets the image as if there were gaps there.

One way to *solve* this problem is to make those more ethereal characters go slightly across the right margin, so the edge of the lines will appear fuller and your brain will interpret the right margin as more even... even when it's not (again, you can check it with a ruler). The change is really minimal, but it greatly improves the appearance of the text.

This micro-typography technique is called *protrusion* and you can see how it works in the figure 7.3: the gray line clearly shows how the hyphen and the comma go beyond the margin.

Another micro-typography technique is to deform the characters so that they occupy more or less horizontal space to avoid the spaces between words being different on different lines. The distance between characters can also be modified... in short, there are several techniques. But unlike protrusion, which always looks good, these other techniques are important mainly when you have narrow columns of text, that is, when the *macro*-typography begins to show its limitations.

Do not even think about trying to apply any of these techniques in a normal word processor like Writer or Word: it is simply not possible. Scribus allows limited work with some of these techniques, but I have never used this program so I can't comment on that.

The only thing left in the FLOSS world is LaTeX and its derivatives with

the `microtype` package, whose documentation is installed, on Linux, at

/usr/share/texmf/doc/latex/microtype/microtype.pdf

The manual is over 250 pages long, but don't worry that in normal use the default options are simply perfect and you don't have to worry about how it works.

To enable the default microtypography options just go to the font settings menu and check the corresponding box. That's it, that's all there is to it. This works either with normal LaTeX, XeTeX or LuaTeX, although in the latter two only in a limited way: for example, at the time of writing this book, XeTeX only offers protrusion support.

If you are curious and want to go deeper into these topics, here are some links

http://hstuart.dk/2007/11/27/microtypography/
http://wiki.lyx.org/Tips/Typography
https://en.wikipedia.org/wiki/Microtypography

7.12 Sub- and superscripts

I mean, sub and superscripts *in the text*, because in math mode it is quite easy: the underscore for subscripts, the caret for superscript and that's it. Now, having subscripts and superscripts in text mode is also necessary, but how to achieve them?

Before continuing, we need to distinguish between *real* and *simulated* sub and superscripts. The real ones are specially crafted characters explicitly designed to act as such while the simulated ones come from reducing the font size and changing the vertical positioning a bit. For example, in

$$2a^{22}$$

the first "2" after the *a* is a real superscript while the second one has been simulated (the 2 before the *a* serves as reference).

As you can see, the real indexes look better than the simulated ones, with less contrast and less detail, which makes them easier to read. That's because scaling the font to simulate the index also reduces the thickness of the strokes, making the indexes look lighter than the base font.

I already told you how to apply OpenType specialties on a font, so enabling true indices with the `+sups`, `+sinf` or `+subs` tags should not be a problem... except for the fact that not all fonts offer these options and those that do, usually do not provide all the characters we might need.

In short, it is generally convenient to use simulated indices. To alleviate this problem, it is convenient to choose a font that offers optical sizes, such as Latin Modern or Antykwa Połtawskiego, since the indexes will be generated from the correct font for that size and will look good.

LaTeX allows working with simulated sub and superscripts thanks to the `\textsuperscript{}` and `\textsubscript{}` instructions. For both LyX offers full support from the graphical interface.

You can insert these instructions in text mode in LyX from the Insert › Formatting › Superscript/Subscript menu.

By default LyX does not offer keyboard shortcuts for these options, but we already saw how to solve this situation (section 4.2), so all you need are the corresponding codes, which for a superscript is:

```
script-insert superscript
```

while for a subscript is:

```
script-insert subscript
```

NOTE: If the font provides all the real real sub- and superscripts needed, there is a package (there is always a package) that takes all `\textsuperscript` and `\textsubscript` commands and automatically replaces them with the corresponding OpenType option.

This package is called `realscripts`, but I won't talk about it here as its documentation is very simple (six pages). Note that when using this package, if you miss a character for which the font does not have a sub/superscript (many fonts have only numbers and almost none offer accents or ñ) you will have strange results.

Now, how can we set the reduction factor and the change in vertical positioning? To achieve this, you need to use pure LaTeX code inside one of those famous "TeX boxes."

The instruction to be used is `\raisebox` and it works as follows

```
\raisebox{<height>}{<text>}
```

where `<height>` indicates the vertical offset and `<text>` is what we need as sub or superscript.

The `<height>` value can be positive (to move up) or negative (to move down) and can be expressed in any unit of measurement. For example

```
\raisebox{6pt}{superscript}
\raisebox{-6pt}{subscript}
```

will give us (the first) a superscript at 6 points above the baseline and a subscript (the second) at 6 points below the baseline.

For the devoted typographer, you can also use the `ex` as a unit. Very basically, an `ex` is a value equal to the height of a lowercase letter without ascending or descending strokes: for example, the height of an x. We could therefore use

```
\raisebox{1ex}{superscript}
```

to make the superscript go up to the height of the "ceiling" of the lower letters.

Now, the `\raisebox` instruction itself does not change the text size so you will have to use an extra instruction. For example

```
\raisebox{1ex}{\scriptsize{superscript}}
```

will give a superscript of size scriptsize (I already told you about font sizes for LaTeX in the section 6.3) at a height of 1ex above the baseline. In principle, to use this code you need two TeX boxes, one before and the other after the text to be indexed, following the following model

base `{\raisebox{1ex}{\scriptsize{` superscript `}}`

you will get the following result

base^{superscript}

I know what you're thinking: writing all this out all the times you need a subscript or superscript? Isn't there a way to abbreviate? Certainly, but you'll have to wait until chapter 9.

Chapter 8
Footnotes and endnotes

Here I will show you different options and tricks to modify the presentation and operation of footnotes in a LyX document and how to use endnotes insets.

For all this you will need the `texlive-yafoot` package. And code in the preamble, of course.

Several of the commands I will point out in this chapter accept configuration parameters, but I won't always go into the detail of what they can give. In most cases doing what is discussed here will be more than enough so it is up to your curiosity to consult the documentation of these packages to understand how to go further.

8.1 Footnotes formatting and their "marks"

By default, in LaTeX and derivatives not only the number identifying the footnote in the text is formatted as superscript, but also the one numbering the footnote itself. One way to change the latter is to add the following in the LaTeX preamble:

```
\makeatletter
\renewcommand\@makefntext[1]{%
   \noindent\makebox[0pt][r]{
   \@thefnmark.\space}#1}
\makeatother
```

Here the footnote mark is redefined, setting it to be presented without indentation (`noindent`), with the number positioned on the right (the `r`) of a box (`makebox`) aligned to 0 pt from the margin (therefore, hanging inside the left margin) while the number itself (`@thefnmark`) is not formatted, so it will appear normal and not as superscript. The `\@thefnmark` instruction is the one that indicates the footnote mark, so if you want to give it a special format just surround it with braces and the necessary instructions.

But this is not the only thing you can do, it is also possible to change the footnotes alignment and the font used by them. See the following code:

```
\renewcommand{\@makefntext}[1]{%
  \setlength{\parindent}{0pt}%
  \begin{list}{}{\setlength{\labelwidth}{6mm}%
    \setlength{\leftmargin}{\labelwidth}%
    \setlength{\labelsep}{3pt}%
    \setlength{\itemsep}{0pt}%
    \setlength{\parsep}{0pt}%
    \setlength{\topsep}{0pt}%
% \setlength{\rightmargin}{0.2\textwidth}%
    \footnotesize}%
  \item[\@thefnmark\hfil]#1%
  \end{list}%
}
```

I'll let you explore it. If you need a hint to start, on the line where it says `\footnotesize` is where you set the footnote font, try writing there something like `\sffamily\footnotesize` to see what you get. I mean, not that I recommend it, in fact, I don't recommend it, but you can try. Okay, a second hint: the commented line allows you to change the right margin of the paragraph of the notes. Now it remains as an exercise to you to understand the rest.

8.2 Footnotes in two columns with main text without columns

Imagine that you have many footnotes, but all of them are very brief, such as "see page such and such," "reference #, on page ##," and so on. In this situation it might be interesting to have the footnotes in two columns even if the body of the text is presented without columns.

For this purpose, write the following in the LaTeX preamble

```
\usepackage{dblfnote}
```

and that's it. The only thing to keep in mind is that on the last page of the chapter, whenever it is half empty, the footnotes will fill the first column before moving on to the second.

8.3 Footnotes without paragraph breaks

If grouping footnotes in two columns is not enough and you want to get footnotes on the same line without paragraph breaks, the `footmisc` package is your friend. For this, in the LaTeX preamble it is enough to type:

```
\usepackage[para]{footmisc}
```

Someone might like it.

8.4 Restart footnote numbering on each page

It is enough to add, in the LaTeX preamble, the following code

```
\usepackage{perpage}
\MakePerPage{footnote}
```

The first line declares the use of the `perpage` package, which is used to restart the numbering of different counters on each page, while the second line indicates which counter we want to restart (the one in the footnotes).

8.5 “Numbering” footnotes with symbols

Imagine that for the footnote numbers you want the following substitution:

1 → * (LaTeX command: `\ast`)
2 → † (LaTeX command: `\dagger`)
3 → ‡ (LaTeX command: `\ddagger`)
4 → § (LaTeX command: `\S`)
5 → **

and so on. It is important to keep in mind that numbering with symbols only makes sense if you restart the numbering of the notes on each page and if you have few notes on the same page, otherwise the document would become chaotic.

I’ll present bellow two methods to achieve this “numbering with symbols.”

The first method is clearly the simplest, but also the less flexible one. It rely in the `footmisc` package.

To use it with the default options, in the LaTeX preamble you need to write the following code:

```
\usepackage[perpage,symbol*]{footmisc}
```

This will give you the default LaTeX symbol sequence, which is like the list above, but with the difference of showing a ¶ at the fifth level and a ‖ at the sixth level instead. If you have more notes, from the seventh onwards you’ll get the previous sequence in the same order, but with duplicated symbols.

`footmisc` offers other character combinations, which you can choose by adding one more line to the above code

```
\setfnsymbol{<option>}
```

where `<option>` can be `bringhurst`, `chicago` or `wiley`. However, I have had problems with those options: if you want to put more than six notes on the same page, you will get an error.

You can find more information in sections 1.6 and 1.7 of the `footmisc` manual, which is installed at

/usr/share/texmf/doc/latex/footmisc/footmisc.pdf

Here is an alternative to the above which, although a bit more complicated, is more flexible since it allows you to choose the sequence of symbols you want.

Continuing with the example at the beginning of this section, in the preamble LaTeX you need to write the following:

```
\def\mysymbols#1{\@mysymbols{\@nameuse{c@#1}}}
\def\@mysymbols#1{\ifcase#1\or $\ast$\or
   $\dagger$\or $\ddagger$\or $\S$\or
   $\ast\ast$\or $\dagger\dagger$\or
   $\ddagger\ddagger$\or $\S\S$
   \else\@ctrerr\fi}
\def\thefootnote{\mysymbols{footnote}}
\usepackage{perpage}
\MakePerPage{footnote}
```

The first line defines a new instruction called `\mysymbols`. The second line set this instruction indicating the substitutions discussed above. The third line redefines the `\thefootnote` instruction to make use of the substitution defined above. The last two lines *must be at the end* and apply the `perpage` package to the footnotes as discussed in the previous section.

It is important to keep in mind that if you try to use more footnotes in a page than the number of substitutions defined, LaTeX will give an error, so better define more symbols than we possible need: if you want to define more symbols, simply add another `\or $\symbol$` to the list in the second line of the code.

The inspiration for this section:
https://groups.google.com/d/msg/comp.text.tex/QZrTF7tzo5k/TvT_RsCyBfsJ

8.6 Return from a footnote

With `hyperref` you get PDF files that let you go from the footnote anchor to the footnote itself, but how do you get back?

Simply: by using a small, old, little know, but undeniably useful package called `footnotebackref`.

After installing the package (it doesn't usually come by default), reconfiguring LyX and setting up all the options you want for `hyperref`, go to the preamble and type in

```
\usepackage{footnotebackref}
```

This will turn the footnote marks into links that will take you back to the footnote anchor, *but only if you have not redefined the footnote mark as described in the first section of this chapter*.

In effect, if you manually change the formatting of the footnote marks you will end up overwriting, and therefore overriding, what the command `footnotebackref` gives you. If you want the marks to be displayed in normal text *and* at the same time be links to go back to the anchor, you will have to use this code:

```
\makeatletter
\renewcommand\@makefntext[1]{%
   \noindent\makebox[0pt][r]
    {\hyperref[\BackrefFootnoteTag]{
\@thefnmark.\space}}#1}
\makeatother
\usepackage{footnotebackref}
```

If you use `hyperref` to define link coloring and such, *this code has to be at the very beginning of the preamble*!

As you can see here we are surrounding `\@thefnmark` with code that indicates that they have to become links. For even more formatting... you know what you have to do.

8.7 Editor notes

You will have seen it more than once, especially in translated books: in addition to the normal author's footnotes there are the editor's footnotes, with their own numbering and more than once clearly separated from the others.

To define a separate set of footnotes you need to use the `manyfoot` and `perpage` packages. Now, in the preamble of the document write the following:

```
\usepackage{manyfoot}
\newcommand{\Afootnoterule}{}
\SelectFootnoteRule{A}[\noindent\footnotesize
                Editor notes:\vspace{2mm}]
\DeclareNewFootnote{A}[alph]
\usepackage{perpage}
\MakePerPage{footnoteA}
```

The first line declares the first package. The second and third lines declare the separator you will see between normal and new footnotes. The fourth line creates a new type of footnote that will be numbered with letters and associated with the LaTeX `\footnoteA` statement. The fifth line declares the second package so that in the sixth line we can say that these new footnotes are numbered by page.

Now, as a foretaste of what I will tell you in more detail in the next chapter (and not to confuse us with endless TeX boxes), you must go to Document » Settings » Local Layout, to create a new "custom inset" where to write these new footnotes:

```
Format 95

Counter ednote
    PrettyFormat "Ednote ##"
End

InsetLayout Flex:Ednote
   LyXType     custom
   LatexName   footnoteA
   LatexType   command
   Counter     ednote
   Decoration  classic
   Font
```

```
    Size Small
  EndFont
  MultiPar    true
  LabelString ednote
  Requires    ednote
End
```

You then validate, accept and that's it: you will now have a new "custom inset" available from Insert ⟩ Custom Insets ⟩ Ednote (or the name you chose while writing the code) with which you can insert these new footnotes. I'll let you test the result.

8.8 Endnotes

The writer does not live by footnotes alone, in some documents it is necessary to use endnotes.

LyX offers graphical support for the `endnotes` package. As *another* introduction of a topic that will be discussed in detail in chapter 9, you should now go to Document ⟩ Setting ⟩ Modules to activate the Endnotes (Basic) module you'll find under Foot- and Endnotes. As of 2.4 you also have the Endnotes (extended) module which uses the `enotez` package, which is much more powerful than `endnotes`.

It is also possible to force all footnotes to become endnotes with the Footnotes as endnotes (Basic) and Footnotes as endnotes (extended) modules.

To insert an endnote in the document it is enough to go to Insert ⟩ Custom Insets ⟩ Endnotes and type the text of the note into the box provided.

Finally, to *display* the notes in the document, since 2.4 you simply go to Insert ⟩ List/Contents/References ⟩ Endnotes. If necessary, you can rename the endnotes section with

```
\renewcommand{\notesname}{Notes at the end}
```

Something interesting with this system is that it is possible to insert these instructions as many times as you want in the document, so it is possible to have endnotes grouped by chapter, for example, instead of all together

at the end. However, if you want to separate the endnotes in this way, it will be convenient to restart the numbering of the endnotes each time, adding the instruction

```
\setcounter{endnote}{0}
```

in a TEX box at the beginning of the new chapter.

8.9 An extra: margin notes

LYX offers native support for inserting margin notes: simply go to Insert > Margin Notes and get one.

Hi!

However, there is not much that can be done to modify the formatting of these margin notes from the program's graphical interface, so you will have to resort to TEX boxes if you want to customize them.

For example, margin notes are placed at the outer margin of the document, but this can be quickly changed by inserting a TEX box with the code

```
\reversemarginpar{}
```

while to return to normal you only have to type, in another TEX box, the following:

```
\normalmarginpar{}
```

LYX uses the default LATEX implementation of margin notes, which has some limitations (it cannot be used in floats, footnotes, and so on). To use the `marginnote` package, which is much more powerful, you can write in the LATEX preamble the following:

```
\usepackage{marginnote}
\let\marginpar\marginnote
```

The `marginnote` documentation is installed in:
/usr/share/texmf/doc/latex/marginnote/marginnote.pdf

It is very brief, so I am sure you will have no problem using it.

At this point I am forced to confess something: I am not particularly enthusiastic about margin notes, you might even say I hate them, so I will not talk about them any further and let you consult the LyX documentation on the subject. Everything there is to know about margin notes in LyX is magnificently explained at Help » Embedded Objects » section 4.3.

Chapter 9
Modules, local format and math macros

Another remarkable feature of LyX is that it can be *extended*. In this chapter I will show you how to add functions, the correct way to define formats not present in the used documenclass and how to prepare some tricks to write mathematical formulas more easily.

9.1 Modules

In chapter 8 I mentioned, in passing and without much detail, the modules: under Document » Settings » Modules (figure 9.1) you have a long list of new functions just waiting to be activated.

These modules simplify the use of several LaTeX packages by adding new tools to the user interface. It is simply impossible to talk about all of them, but in future chapters I will discuss some of them, so for the moment I will let you play with the options. Now, is it possible to add other modules not offered in the default installation?

Of course! (Although it usually won't be necessary...)

On this page of the LyX wiki:

http://wiki.lyx.org/Layouts/Modules

it is possible to find several modules ready to be used, and later on we will see some more. You have modules that add new functionality (comments, name indexes, formatting), integration with the Sage program, mod-

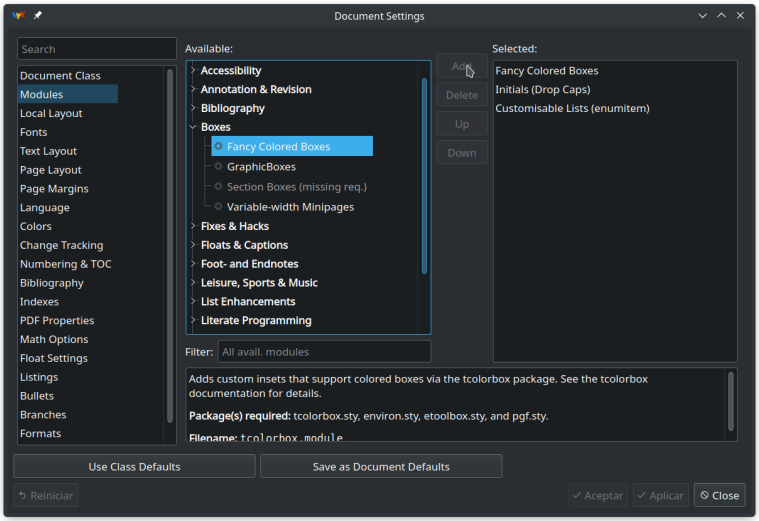


FIGURE 9.1: *Enabling modules*

ifications of the format of different documentclass, and so on.

The way to install these modules is, after downloading the *.module files, to copy them into the layouts folder of the LyX user profile which on Linux is located in

~/.lyx/layouts/

Always remember to *Reconfigure* the program after installing anything!

9.2 Creating new instructions

Almost without realizing it, we already saw how to create instructions when defining new font families in section 7.3. But of course that was just an appetizer: you will now see how to do much more, creating instructions that allow you, for example, to apply formatting, either for paragraphs or characters.

The simplest way to achieve new instructions in LaTeX/LyX is through the command `\newcommand{}[]{}`. This command must be used in the LaTeX preamble as follows:

```
\newcommand{\mycommand}[<number of
  parameters>]{<text and LaTeX instructions>}
```

Returning to the example at the end of chapter 7, imagine that you need to create a new instruction to insert a superscript in text mode using the `raisebox` command. Here, `\mycommand` is the name you will give to the instruction, for example `\goup`, `<number of parameters>` indicates how many parameters the new command will need, in this case 1, whereas in `<text and LaTeX instructions>` you have to write whatever serves to create the desired format, calling each parameter with `#<number>` (`#1` for the first parameter, `#2` for the second, and so on).

The code to be inserted in the LaTeX preamble will then be as follows

```
\newcommand{\goup}[1]{{\raisebox{1ex}{
   \footnotesize{#1}}}}
```

In principle, to use this instruction inside LyX in a "brute force / let's write code" way, you open a first TeX box with Ctrl L, type `\goup{`, exit the box to write the text to be superscripted and finally create a second TeX box where you write a closing `}`. This way, you have the following in the LyX edit window

base `\goup{` superscript `}`

but in the PDF output you will get

base^{superscript}

It is possible to use more parameters, or even none at all. In the latter case, the instruction would have the form

```
\newcommand{\mycommand}{<text and
  LaTeX instructions>}
```

Done! Right? Well, not quite, as it's easy to see this is still impractical, but here we are! I'll finally show you how to do this and much more in the best LyX style.

9.3 Creating character styles in LyX

In LyX you can quickly apply some predefined character styles, such as emphasis (with Ctrl E, by default it gives italics), bold (Ctrl B) or small caps (Alt C, then C), but not much more. By enabling the Logical Markup module you will have a few more, which will be listed in the right click » Text Style or in Edit » Text Style. Now, if you want to create another one? You might need a simpler way to activate a particular OpenType feature —for example, discretionary ligatures— instead of using so many TeX boxes, or prefer to do less typing when using the `hologo` package I told you about earlier.

> Note: Remember that, for the second example, you have to write `\usepackage{hologo}` in the preamble!

Let's go to Document » Settings » Local Layout. There you have to write something like the following:

```
Format 95
   InsetLayout Flex:HistoLig
   LyxType               charstyle
   LabelString           histolig
   LatexType             command
   LatexName             histolig
   Preamble
     \newcommand*{\histolig}[1]{{%
        \addfontfeature{Ligatures=Historic} #1}}
   EndPreamble
   ResetsFont            true
End

   InsetLayout Flex:LogoTipo
   LyxType               charstyle
   LabelString           HoLogo
   LatexType             command
   LatexName             loguito
   Preamble
```

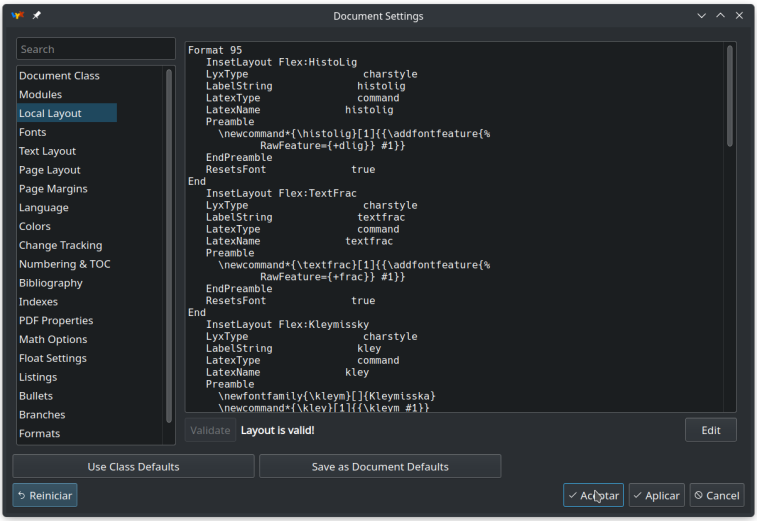


FIGURE 9.2: *Create a "local layout"*

```
    \newcommand*{\loguito}[1]{{\hologo{#1}}}
  EndPreamble
  ResetsFont            true
End
```

NOTES:

- Before anyone asks, the `Format 95` at the beginning indicates the version of the local layout system being used by LyX 2.4 (later versions may have a greater number). This first line is mandatory, *but you only need to use it once* if you define several local layouts.

- After the colon on the second line, there should be NO space. This line defines the name of the new format.

- `LyxType` defines the type of format being created: in the example, a character style (`charstyle`) whereas in the example I dropped in the previous chapter it was a "custom inset" (`custom`).

- `LabelString` is the label you will see in the LyX edit window marking the applied formatting.
- `LatexType` defines that a command is being applied, either base or, as in the example, user-defined.
- `LatexName` is the name (without the leading backslash) of the LaTeX command to be applied.
- If you want your style to be presented differently *while you are editing the document*, you can vary the font type by doing something like the following:

```
Font
   Family     Serif
   Shape      Italic
EndFont
```

- The part between `Preamble` and `EndPreamble` can be used to define code that should go to the LaTeX preamble. Clearly, if that code is already in the preamble, you do not need to rewrite it here!
- `ResetsFont` is used to prevent the applied formatting from extending beyond the selected text.
- Before you can accept, you have to use the VALIDATE button: only if LyX confirms that the code is correctly written can you proceed.
- It is also possible to create environments, , i.e. paragraph styles. This is already explained in detail in the LyX manuals (Help › Customization › section 5.3.7, but if you want an example, see the following section.

Now you just have to select the text that needs the style, and right click › Custom Text Styles › HistoLig. What in the LyX edit window looks more or less like this

this is a non historical text this is an historical standard this another non historical text

histolig

in the output it will look like this:

this is a non historical textthis is an historical standard this another non historical text

whereas

XeTeX AmS
HoLogo HoLogo

will look like this:

XeTeX $\mathcal{AMS}$

To *remove* an already applied text style, it is enough to do a right click > Custom Text Styles > Dissolve Text Style.

In subsection 16.1.2 I will give you another example of local formatting while in the appendix B you will find the definition of all local layouts used to create this book.

9.4 Paragraph environment?

Just a quick example without explanation to create a paragraph environment with a background color and a border. It uses the `mdframed` and `xcolor` packages which of course have to be installed:

```
Format 95
Style MyParagraph
 Category      MainText
 Margin        Static
 LatexType     Environment
 LatexName     myparagraph
 NextNoIndent  1
 LeftMargin    MMM
 RightMargin   MMM
 ParIndent     MM
 ParSkip       0
 ItemSep       0
 TopSep        0.5
 BottomSep     0.5
```

```
ParSep        0
Align         Block
AlignPossible Block, Left, Right, Center
Preamble
  \usepackage{mdframed}
  \usepackage{xcolor}
  \newenvironment{myparagraph}
     {\begin{mdframed}[
       backgroundcolor=lightgray,
       linecolor=lightgray
     ]\quotation}
  {\endquotation\end{mdframed}}
EndPreamble
End
```

Try to understand it! Just play a bit with the parameters. It's fun!

9.5 Math macros

Let's consider two mathematical situations:

SITUATION 1: You could have a complex mathematical expression that is repeated, with minor variations, several times in the same paper. For example, you might be writing an essay on Fourier transforms, which would require you to enter an expression many times where only the function to transform would change, not the rest of the integral.

SITUATION 2: LyX cannot support every existing LaTeX command so sometimes you will have to resort to writing code directly, code that might look quite strange in the edit window.

Tackling any of these situations would mean a lot of manual work of tweaking details and compiling to see how it looks, work that would be repeated *ad nauseam*... or not, or at least not always.

I welcome you to one of the most interesting, but less known, features of LyX: *mathematical macros*.

The idea is to create a macro with the necessary LaTeX instructions (the fixed part of the expression), the arguments needed (the part that will change each time) and give the whole thing a short name. Once the macro is ready, it will only be a matter of calling it from anywhere in the document using its name, the only pending task will be to fill the gaps left by the arguments.

With each new version of LyX, the SITUATION 2 gets increasingly rare, because the remaining unimplemented commands are just the most exotic ones, so I will tell you how to solve SITUATION 1 instead: in the process it will become clear how it all works.

For this, I will use the `mathop` command for which LyX has graphical support. This command is used to convert any sign or text into an operator that accepts limits.

First, you have to go to Insert » Math » Macro, that something similar to what is shown in figure 9.3 will appear.

As you can see, a box will open in the document that will be used to create your macro while underneath you will have, in addition to the traditional toolbars of the equation editor, a new one to create or remove arguments, make them optional, and so on.

> NOTE: This box will not be present in the compiled document, so you do not have to worry about its existence. In fact, it could be placed anywhere in the document, but it is better to group all the macros in a place where you can find them quickly.

Now replace `newmacroname` with something you are more comfortable with. Since in this example we want to simplify the use of the `\mathop` command, we can call it `mop`.

> IMPORTANT: Be careful with the chosen name, you cannot use a valid, preexisting LaTeX command!

In the first of the two small sub-boxes, the one labeled as "TeX," you have to type the LaTeX code while in the second one, the one labeled as "LyX," what you want LyX to display during the use of the macro. Since `\mathop` has complete graphical support, you simply type the same in both boxes.

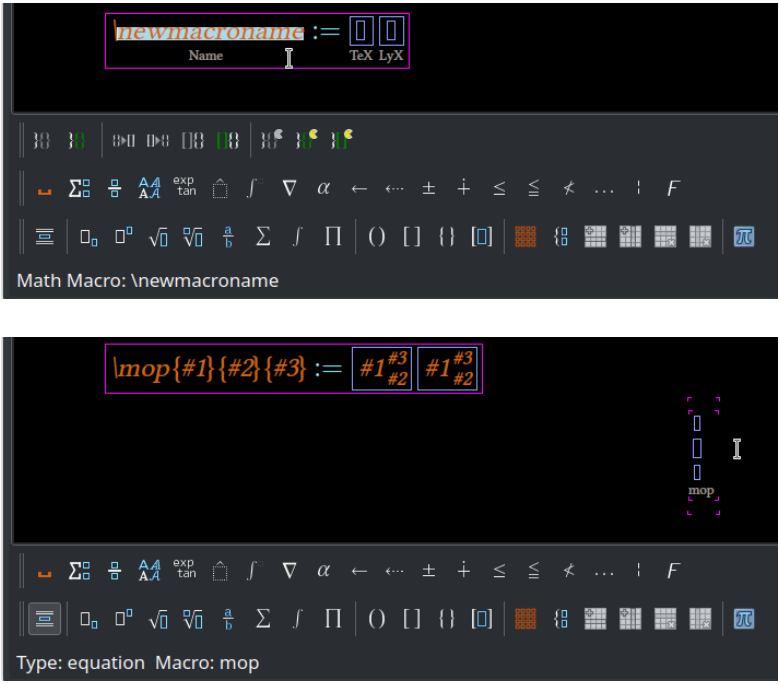


FIGURE 9.3: *Creating a math macro. Above, starting the process; below, how it all looks in the LyX window, with the expression waiting for the three parameters to be completed*

And it is here where you can see how to solve the SITUATION 2: in the second box you do not type the same as in the first one, but something that helps to understand what you will get in the output.

Remember that in the LyX math editor, the easiest way to insert the `{}` delimiters for grouping expressions is by typing `\{` (the closing brace will appear automatically).

Then use the new toolbar to add the number of arguments needed, which in this case are three, and apply the necessary instructions to each of them (`\mathop` to the first parameter, subscript to the second, and superscript to the third), which will give you what is shown in the bottom image of figure 9.3.

There you have it. To use the brand new macro you only need to write inside an equation

```
\mop
```

and then press the space bar once. Now you will have three boxes waiting for content.

Finally, by putting an equals in the first parameter, leaving the second empty and typing def in the third, or by typing the sum for i and j equal to one up to N in the first parameter, $i > j$ in the second and leaving the third empty, in the compiled document (remember that the box defining the macro will not appear) you will obtain

$$f(x) \overset{def}{=} \ln(x)$$

$$\sum_{\substack{i,j=1 \\ i>j}}^{N} A_{ij}$$

You can nest your expressions or, as you see, leave some of the boxes blank: the only limit is your creativity in solving these problems.

And that's it! As you can see, math macros in LyX are an incredibly flexible and simple tool to use, so be sure to take advantage of them!

If you want to see an example of a math macro in action, and don't mind to look at the Spanish interface, you have this short YouTube video:

https://www.youtube.com/watch?v=VG9PHl3ToVk

There I show you how to solve SITUATION 2, defining an instruction for which there is not (yet) graphical support in LyX.

Chapter 10
Numbered lists

As I mentioned in the introduction, modifying the format of numbered lists has its twists and turns. In this chapter I will tell you how to make them numbered with Roman numerals, or with letters. I will also show you how to do this for all the lists in your document or for a particular one.

Just make sure you have the `enumitem` package installed on your system, otherwise nothing will work.

10.1 Modify all lists in the document

The first stop is Documents » Settings » Modules. There select CUSTOMIZABLE LISTS (ENUMITEM) under LIST ENHANCEMENTS and press ADD.

Now you have to go to the preamble, but first think carefully about what you want to achieve for your lists. For example, you could have the first level numbered with capital letters, the second level marked with numbers, the third level with Roman numerals and the fourth level with lowercase italic letters, and each sublevel also showing the previous numbers. So, this would be the code:

```
\setlist[enumerate,1]{label = \Alph*}
\setlist[enumerate,2]{label = \theen-
umi.\arabic*}
\setlist[enumerate,3]{label = \theenu-
mii.\Roman*}
```

```
\setlist[enumerate,4]{label = \theenu-
miii.\textit{\alph*}}
```

How to format numbers and choose styles is something we have already seen in previous chapters. The codes `\theenumi`, `\theenumii` and `\theenumiii` refer to the numbering of levels 1, 2 and 3, respectively and come from ***the enum****eration* <#>, where the number is written in... lower case Roman numerals. And yes, the fourth level would be `\theenumiv`, luckily we don't need it.

Instead of the dots after each `\theenum#` you can use whatever symbol you want, for example a parenthesis or even a mathematical instruction like `$dagger$` to get a †. You can also add a separator after the last number. Just try not to overdo it.

If you don't want to show the previous levels, you can use something like this:

```
\setlist[enumerate,1]{label = \Alph*}
\setlist[enumerate,2]{label = \arabic*}
\setlist[enumerate,3]{label = \Roman*}
\setlist[enumerate,4]{label = \textit{\alph*}}
```

10.2 Modify some numbered lists

In the previous section I showed you how to modify all the numbered lists in the document simultaneously, now you will see how to create a local format to modify selected lists.

So, under Document › Settings › Local Layout

```
Format 95
Counter  legali
         LabelString     "\arabic{legali}."
End
         Counter  legalii
         Within   legali
         LabelString     "\thele-
gali\arabic{legalii}."
```

```
End
Counter  legaliii
         Within          legalii
         LabelString     "\thele-
galii\arabic{legaliii}."
End
Counter  legaliv
         Within          legaliii
         LabelString     "\thele-
galiii\arabic{legaliv}."
End
Style    Legal
         CopyStyle Enumerate
         LatexName legal
         LabelCounter "legal"
         RefPrefix leg
         Requires enumitem
         Preamble
            \newlist{legal}{enumerate}{4}
            \setlist[legal]{label*=\arabic*.}
         EndPreamble
End
```

Validate, accept and you are ready to work: this local formatting will add a list style called "Legal" to the paragraph styles drop-down menu under the File menu. I leave it as an exercise to interpret the code and modify it to obtain other formats.

In the end it was not that difficult.

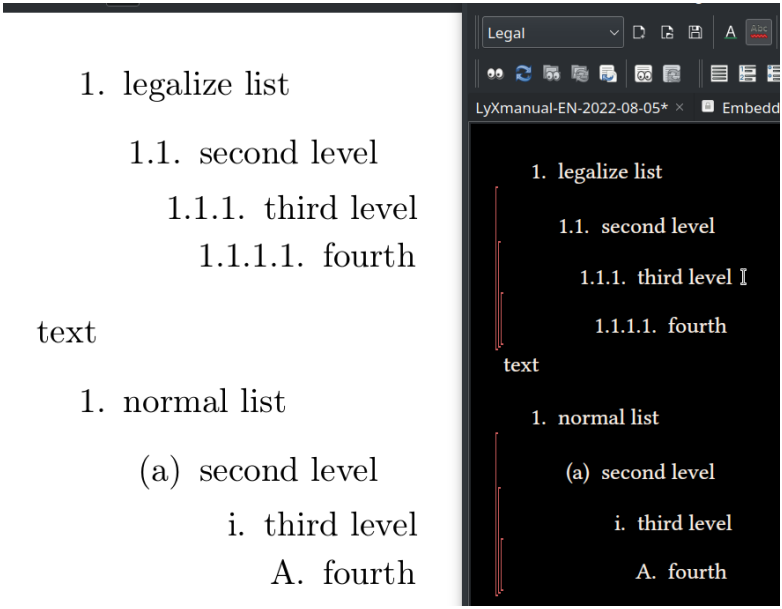


FIGURE 10.1: *Example of modified lists. The first list has been created with the list style introduced in the text, while the second uses the global formatting discussed in the first section of this chapter (albeit with a different format).*

Chapter 11
About the Table of Contents and other Indexes

Setting up indexes is perhaps one of the most obscure points in LaTeX. In this chapter I will give you a brief introduction to some tools for solving specific index-related issues.

11.1 Short titles

Let's start with something simple: what if the title of a chapter or section is too long and you want both the table of contents and the page header to show a shorter version of it?

With the cursor in the title in question: Insert » Short Title, or with the mouse right click » Short Title or perhaps, for the more memory-savvy, Alt A, then 1. The box that appears will allow you to type a shorter version of the title to be used in the table of contents and cross-references, overwriting the actual title.

Now for something more complicated: how to have different text in the title, in the table of contents *and in the page header*?

After inserting the short title, you can open a TeX box with the following code for the chapters

```
\chaptermark{Chapter title for header}
```

or the following for sections

```
\sectionmark{Section title for header}
```

Note that these instructions can be placed *after* the title, not necessarily on the same line. In fact, you can re-enter it later to change again the text displayed in the header to identify the section or chapter. Use it as many times as you need!

11.2 Footnotes in headings

Imagine that for some reason you want to place a footnote in a section title. For example, you have an article with several authors, each in charge of a section, and the editor decides to indicate who wrote each section with a footnote anchored to the title. The problem? That if you insert the footnotes normally and then create a table of contents, the footnotes anchored to the titles will be repeated in the table of contents, all those duplicate footnotes!

The challenge then is to get the footnote to be only in the title that corresponds to it, not in the index. The solution is to use what we saw in the previous section: short titles.

When you insert a short title, by default LyX copies the title already entered in the paragraph, so the trick is the following:

1. Insert the original title.
2. Insert the short title.
3. Return with the cursor to the end of the original title.
4. Insert the footnote.

Done. Since the short title is what goes to the table of contents, the footnote will not be copied.

11.3 Modify the name of the table of contents

Let's say you don't like the sound of a traditional "table of contents" but you want to have a fancier "list of topics" instead. For this you can use Babel.

You don't need to modify Babel's default options here, so without touching anything in the language options of the document configuration, you just need to go to the LaTeX preamble to add the following code:

```
\AtBeginDocument{
   \addto\captionsenglish{
     \renewcommand{\contentsname}{List of Topics}
}}
```

And that's it. If the document you are writing is in Spanish instead of English, simply change to `\captionsspanish` in the second line. The same for other languages.

11.4 Partial tables of contents for each chapter

In the LaTeX preamble you have to write the following:

```
\usepackage{minitoc}
```

(obviously, the `texlive-minitoc` package must be installed). In the document, just before the table of contents, insert a TeX box with the command

```
\dominitoc
```

Finally, for each chapter, just after the chapter name and before any section, insert another TeX box with the command

```
\minitoc
```

11.5 Correct the spacing in lists of figures and tables

When an article document class is used, figures are numbered in sequence without interruption, whereas in book format the numbering is usually <chapter>.<number>.

In LyX it is really simply to activate a numbering scheme by sections, which is presented, in articles, as <section>.<number> while in books it is

<chapter>.<section>.<number>: it is enough to go to Document › Settings › Modules (we talked about modules in chapter 9) to enable NUMBER FIGURES BY SECTION.

The problem with this arises when you have a book document class and you create the figure index: the extra number takes up more space than the default configuration of the index so the numbering clashes with the text that follows it.

To get more space between number and text, go to the LaTeX preamble and type the following:

```
\makeatletter
\renewcommand*\l@figure{\@dottedtocline{1}{
   1.5em}{3em}}
\makeatother
```

To change the distance you just need to play with the `{3em}` value to the right of the middle line of the indicated code.

If you want to do the same change with the table index, add

```
\renewcommand*\l@table{\@dottedtocline{1}{
   1.5em}{3em}}
```

between `\makeatletter` and `\makeatother` and that's it.

To learn more about this topic you can consult the following topics in tex.stackexchange:

http://tex.stackexchange.com/questions/133747/reduce-space-between-table-number-and-title-in-list-of-tables
http://tex.stackexchange.com/questions/87019/list-of-figures-numbers-overlap-image-caption

11.6 Avoid dotted lines in indexes

By default, many document classes use dotted lines as "guides" in the indexes, as a way of connecting the heading with the corresponding page number.

Let's suppose you want to remove those lines. In the LaTeX preamble add the following line:

```
\renewcommand{\@dotsep}{10000}
```

Done. This command will apply to *all* indexes in the document.

What happens here is that the command `\@dotsep` controls the separation between the dots, so if instead of 10000 you use values like 20 or 30, you will simply get a noticeably larger dot separation than the default setting. The value 10000 is special in LaTeX and it is used to indicate "infinity" in any command that needs a numeric parameter, so in this case it ends up disabling the dotted line.

11.7 Adding a line with text to the table of contents

Imagine that you want to add the word **Page** in bold above the page numbers of the table of contents. Well, just below the point where the table of contents has been inserted, it is enough to open a TeX box with Ctrl L to add the following code:

```
\addtocontents{toc}{~\hfill\textbf{Page}\par}
```

Here we are inserting at the beginning of the table of contents a paragraph containing only a "horizontal fill" and the word Page in bold. That's it. And yes, you can also use this for the other types of indexes, just replace `toc` *(table of content)* with `lof` *(list of figures)* for the list of figures or with `lot` *(list of tables)* for the list of tables.

You can use this trick to add any text at any point in the index, just add one of these TeX boxes in the appropriate place. As an exercise, I let you think how to add the text "Here comes the good stuff" just before a given chapter.

11.8 Remove the page number from the entries for Parts

If you use a book class, you might want to group chapters into Parts. And you may also want the table of contents not to show the page number of the parts, after all only chapters matter.

For this, after making sure that the `tocloft` package is installed (and after reconfiguring LyX), write in the LaTeX preamble the following:

```
\usepackage{tocloft}
\cftpagenumbersoff{part}
```

Here we are activating the mentioned package and using one of its options to remove the page number of the parts.

11.9 Some (few) words about alphabetical indexes

In LaTeX there are several ways to generate alphabetical indexes and certainly you can use any of them in LyX.

You can choose between `makeindex`, `texindy` or `xindy`. If you don't know what I'm talking about, don't worry, the problem is not yours: this is a complex subject that requires study in order to take advantage of it. This means that talking in detail about the alphabetical indexes is beyond the general purpose of this book, so I will only say the bare minimum. It is up to your curiosity to delve into both the LyX user's guide (section 6.6) and the various LaTeX manuals available online.

To mark a word or text to be included in an alphabetical index just select it and go to Insert » Index Entry. This will give you a box with the mark just after the marked text. By default the content of the mark will be equal to the selected text, but you can change it without any problems.

The index itself can be entered anywhere in the document with Insert » List/Contents/References » Index List. Unlike the table of contents, nothing happens by clicking on the resulting button, but it is possible to configure the processor to generate the alphabetical index in Document » Settings » Indexes. There, in addition to choosing between `makeindex`, `texindy` or `xindy`, you can pass options to these processors, choose to create more than one index (you could have one for words, another for names, etc.) and manage existing indexes.

Chapter 12
Headings

In this chapter I will show you how to modify the presentation and numbering of the headings of a document, whether they indicate a "part," "chapter," "section," etc.

12.1 Rename the Part and Chapter environments

Imagine you want to write a book with LyX and you select, for example, the document class Book.

This way you will have at your several environments available, among them Chapter (LaTeX command `\chapter{}`) to separate the different topics of you book and Part (LaTeX command `\part{}`) to group chapters.

But suppose that instead of separating into Parts you want to separate into Books and instead of "Chapter 1: Chosen title" you want "Day 1: Chosen title" to appear. How can these environments be renamed?

The inspiration comes from the previous chapter, where I showed you how to rename the table of contents. As usual, in the LaTeX preamble you have to write the following code:

```
\AtBeginDocument{
 \addto\captionsenglish{
  \renewcommand{\partname}{Book}
  \renewcommand{\chaptername}{Day}
 }}
```

Basically you are fooling LaTeX by telling it that instead of using the predefined labels for `Part` and `Chapter`, to use a "translation" you provide instead.

In fact, if you don't want anything there you simply leave the braces empty: `\renewcommand{\partname}{}` will show only the number and title you type, without labels.

In the LyX window nothing will change, but in the PDF output everything will be displayed as you want.

12.2 Number chapters with Roman numerals

To change the style of the chapter numbering into roman numerals, the chapter counter must be modified by applying a formatting to it, and for this purpose the `renewcommand` command can be used:

```
\renewcommand{\thechapter}{\Roman{chapter}}
```

To use uppercase letters it is enough to change `Roman` to `Alph`, as we have seen in chapter 6.

12.3 Make the chapter start at any page

By default in book document classes, chapters always start on a new page which is also a right (odd) page. If you missed the fleeting comment in chapter 5, getting chapters to start on any page, left or right is really simple, in Document › Settings › Document Class › Class Options › Custom is enough to type:

```
openany
```

12.4 Avoid page breaks at the beginning of a chapter

Not that I like it, but you know, for the sake of completeness.

Let's suppose that for any reason you want to completely avoid a page break at the beginning of each chapter so that a chapter starts on the same

page where the previous one ended. To achieve this you have to redefine the chapter environment so in the LaTeX preamble you have to write the following:

```
\makeatletter
\renewcommand\chapter{%
                    \global\@topnum\z@
                    \@afterindentfalse
                    \secdef\@chapter\@schapter}
\makeatother
```

I have seen this code here:
http://compgroups.net/comp.text.tex/latex-report-class-page-break-after-every-chapt/1924905

12.5 Change the format of titles in LyX documents

As we have already seen, the chosen document class determines the general appearance of the document and that certainly includes how the different headings (chapters, sections) look like.

Most document classes use the same font as the body text for the headings. This generally gives good aesthetic results, but what if you want to change this formatting? Or change the color of the text? Or add special effects? For example, you might want to have a box around the title or use a different size.

There are several ways to achieve title formatting changes and some document classes such as Koma Script offer tools to achieve some of these things, but the option I'll present below is perhaps the most general one possible (although it's not completely general since it doesn't work with the Memoir document class).

12.5.1 The packages needed

Generally, they are installed by default, but it is always a good idea to check if the `xcolor` and `titlesec` packages are actually installed in your LaTeX distribution.

The `xcolor` documentation is usually installed, in Linux and with TeXLive, at:

/usr/share/texmf/doc/latex/xcolor/

while `titlesec` documentation is installed in

/usr/share/texmf/doc/latex/titlesec/

I will give here general indications on these packages that will be more than enough in most cases. For more information and other extreme uses, it is always possible to read the magnificent manual.

12.5.1.1 `xcolor`

The `xcolor` package is used to define, yes, you guessed it, colors. The easiest way to use it is to declare it in the LaTeX preamble with, for example

```
\usepackage{xcolor}
\definecolor{PaleBlue}{rgb}{.3,.5,.7}
```

The first line declares the package while the second defines a color: the parameters between braces are, respectively, the color name, the definition method and the values of each basic color in the form of three numbers between zero and one (in this case, how much red, how much green and how much blue), separated by commas.

12.5.1.2 `titlesec`

This package provides several instructions for making changes to titles. It can be used in two modalities, a simplified form and a complete one.

The simplified form can only be used for sections, subsections and lower levels, while for Part and Chapter environments (or to have all the pyrotechnics offered by the package) the full form must be used.

12.5.2 A complete example

In the LaTeX preamble try writing the following:

```
\usepackage{titlesec}
\usepackage{xcolor}
```

```
\definecolor{PaleBlue}{rgb}{.3,.5,.7}
\definecolor{Greeeeen}{rgb}{.3,.7,.3}
\newfontfamily{\coral}{TeX Gyre Chorus}
\titleformat{\chapter}[frame]
   {\LARGE\coral\color{PaleBlue}}{
   \filright\chaptertitlename\
   \thechapter}{20pt}{\Huge}
\titleformat*{\section}{
   \Large\bfseries\sffamily
   \color{Greeeeen}}
```

The first two lines declare the use of the `titlesec` and `xcolor` packages.

Then, in the next two lines, two colors are defined, PaleBlue and Greeeeen (no, that's not a typo).

The fifth line defines a new font family to use in the document. I already told you about that in the section 7.3.

In the example you can see both, the simplified and the complete form of `\titleformat`, part of `titlesec` package.

The starred form is the simplified one and has been used to quickly define the format of the sections: applied to the `\section` environment, it uses the sans font defined in the document (`\sffamily`), in bold and with its size set as `\Large`, assigning to it the color *Greeeeen* that we have defined.

The complete form is used to modify the `\chapter` environment. It starts indicating that a frame should be built around the title (`[frame]`: if you do not want a frame, it is enough to write `[display]` instead). Then, you have the size defined as `\LARGE` for the label (the word "Chapter" followed by the number), the *coral* font family and the *PaleBlue* color. Then come a series of formatting instructions that start with align right, continue to give the structure of the information presented (including a separator space between label and title text), and end by indicating that the title text should be sized `\Huge`, which is definitely larger than the label. You can see the result of these instructions in the screenshot in the figure 12.1.

Clearly, this can also by done for the Part environment, adding for example:

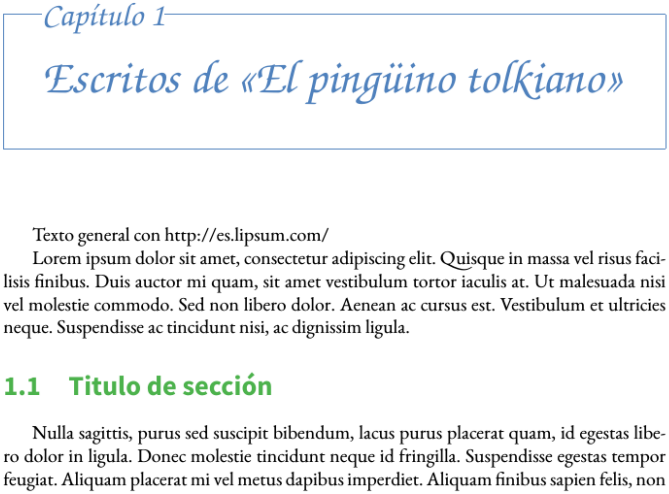


FIGURE 12.1: *An example of titles modified with the use of* `titlesec`

```
\titleformat{\part}[display]
   {\huge\cormorant}{\filright\partname\
   \thepart}{20pt}{\Huge}
```

Here I have not used the frame and chose a larger size for the part label (in addition to using a font family defined elsewhere).

And that's it: all the basic ingredients are in place, so all that remains is to compile the document. The instructions shown here are even more powerful, but I won't go into more details.

12.6 Restart the numbering after a Part

By default, LaTeX does not restart the chapter numbering after a Part environment, giving a continuous numbering:

- Part I
    - Chapter 1

    - Chapter 2

- Part II

    - Chapter 3

While this default option seems logical to me and in fact I use it (in my book on Writer I propose tricks to get it there), it is perfectly possible and therefore completely valid for someone to need the opposite option, i.e:

- Part I

    - Chapter I.1
    - Chapter I.2

- Part II

    - Chapter II.1

The simplest way to achieve this is by using the `chngcntr` package. Since 2018 this package is integrated in LaTeX 2_ε, so if you use a recent version of TeX Live you don't need to call it explicitly, you can simply use the instructions it provides (`counterwithin` and `counterwithout`)

The documentation, at least in Linux, is installed in:

/usr/share/texmf/doc/latex/chngcntr/chngcntr.pdf

It is only two pages, so I recommend you to read it.

To specify that `chapter` is counted within `part`, that is, that it restarts the chapter numbering every time `part` changes, in the LaTeX preamble you must write the following:

```
\counterwithin{chapter}{part}
```

Ready, isn't it? Well, no: when you add the Part to the numbering of chapters (and figures, and tables) you will have problems with the table of contents, since the number will run out of space and overlap with the title text. I already showed you how to solve this for lists of figures and tables in chapter 11, so following that example, after the two lines above you will also have to add in the preamble the following:

```
\makeatletter
\renewcommand\l@chapter{
   \@dottedtocline{2}{1.5em}{3em}}
\makeatother
```

To add space in the index lines corresponding to the sections, duplicate the second instruction shown above, but change `l@chapter` by `l@section`:

```
\makeatletter
\renewcommand\l@chapter{
   \@dottedtocline{2}{1.5em}{3em}}
\renewcommand\l@section{
   \@dottedtocline{2}{1.5em}{3em}}
\makeatother
```

And so on with subsections (`subsection`), etc. OK, now it *is* ready! Although it is not something you will see in this book.

Chapter 13
Headers and footers, and page styles

In LaTeX and derivatives, to modify the default values of headers and footers, the `fancyhdr` package can be used. LyX offers partial support for this package in its graphical interface, but it is so partial that we will have to go to the LaTeX preamble to get everything from it.

Document headers and footers are associated with page styles, a topic that will be covered at the end of this chapter. Before I will tell you how to change the content of the headers and footers of "normal" pages and finally get to the creation and use of special styles.

13.1 Preparations

The `fancyhdr` documentation is installed, in Linux and with TeXLive, in

/usr/share/texmf/doc/latex/fancyhdr/

By default, LaTeX documents are set up as "one-sided" pages, so if you want to use both sides of the paper you have to tell LyX in advance. And while you're at it, enable `fancyhdr` package, of course: it is enough to go to Document > Settings > Page Layout and, as you can see in figure 13.1, for Page style select Fancy and also check the Two-sided document option if you need it.

Now, in this same menu you move on to the LaTeX preamble and start typing... but before that, a brief introduction to *what* can be written.

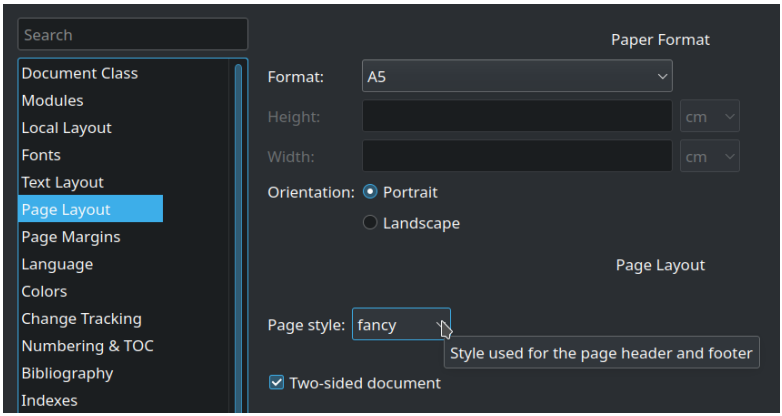


FIGURE 13.1: *Enabling* `fancyhdr`

13.2 The `fancyhdr` package

The `fancyhdr` package allows you to change the content of the headers and footers of the page. Both sectors are divided into three parts: left (l/L), center (c/C) and right (r/R).

There are two ways to use this package. The most straightforward way, useful for "one-sided" documents, is with the instructions

```
\lhead{content of the left header}
\chead{content of the central header}
\rhead{content of the right header}
\lfoot{content of the left footer}
\cfoot{content of the central footer}
\rfoot{content of the right footer}
```

Below you will see examples of the type of content you can use.

The most powerful form of this package is with the `\fancyhead` command for headers and `\fancyfoot` for footers, taking into account the following conventions

R represents "right"

L represents "left"

C represents “center”

O represents odd pages

E represents even pages

The instruction

```
\fancyhead[RO,LE]{CONTENT}
```

will put CONTENT to the right on odd pages and to the left on even pages. Also, if you write

```
\fancyfoot{}
```

the footer will be empty.

Now, what kind of CONTENT can be used?

For sure, simple text works perfectly fine. In fact, if you use XeLaTeX and define a new font family (as we have seen in section 7.3) it is possible to use text as strange as you want (bellow, an example).

You also have instructions for inserting the page number and chapter. For example:

```
\thepage
```

inserts the page number, while

```
\leftmark
\rightmark
```

are used (I’m simplifying a lot here, but for our purposes this idea is enough) to provide information about the chapter / section in use (see example).

> Note: if for some reason the information is not available for any of those “marks” (for example, in a given chapter you have no sections), you will have an empty space on the corresponding pages.

In general, the chapter name will be set to uppercase. To avoid that you need to use the `\nouppercase{}` command (you will see it in action in the example). Care must be taken with this instruction because if the pages are numbered with Roman numerals, these may also appear in lowercase... Yes, LaTeX can be weird.

Using these commands, a separator line will be added. To modify its thickness you can use the following code:

```
\renewcommand{\headrulewidth}{0.4pt}
```

where you can play with the measurement between braces. By typing 0pt (zero points), the line disappears.

13.3 An example

Let's put everything together:

```
\newfontfamily{\coral}{TeX Gyre Chorus}
\renewcommand{\headrulewidth}{0.4pt}
\fancyhead[RO,LE]{\thepage}
\fancyhead[LO]{\nouppercase{\leftmark}}
\fancyhead[RE]{\coral{El pingüino tolkiano}}
\fancyfoot{}
```

Here you have the page number on the right on odd pages and on the left on even pages, the chapter name on the left on odd pages, and the text "El pingüino tolkiano," formatted with the TeX Gyre Chorus font, to the right on even pages.

Now, the command `\leftmark` writes "Chapter #. Chapter name". If you want only the chapter name to appear without the number, also in the preamble you have to type the following:

```
\renewcommand{\chaptermark}[1]{
\markboth{#1}{}}
```

while if you want the section to appear unnumbered you must add this:

```
\renewcommand{\sectionmark}[1]{
\markright{#1}}
```

For the chapter name to also appear in the new font family, it is only necessary to surround `\leftmark` with the corresponding instruction. Continuing with our example, it would be

```
\fancyhead[LO]{\nouppercase{\coral{\leftmark}}}
```

The `fancyhdr` documentation will give you a lot of extra information, such as the right way to include a picture in the headers or footers.

13.4 Create and use page styles

From now on you will see how to define and use different page styles, solving some ~~problems~~ "not always desired features" presented in the default options of most document classes.

Everything will be done with pure LaTeX code. Life is hard.

Imagine that you need to create a document with elaborate page headers and with a structure like this

1. Index with pages numbered with roman numerals
2. A non numbered Chapter with the foreword
3. First part
4. First numbered chapter
5. ...

I already told you about page numbering in indexes (chapter 11), and introduced the tools for modifying headers and footers above. To have Part and Chapter environments you need one of the Book document classes. An unnumbered chapter is created by selecting the Chapter* environment (with the asterisk), so it all seems easy.

The problem with all this, because if there were no problems this section would not exist, has several parts. To begin with, the introduction

created with the unnumbered chapter is not included in the table of contents. To continue, the information available for the header/footer of the introduction will show "Table of contents" instead of "Introduction:" this information does not update' because as the chapter is unnumbered (and therefore *not indexed*) there is no new information available.

To solve this you have to create a page style with the necessary information and tell LaTeX that it should add one more entry to the index.

To understand how this works, I will give you an example to analyze in detail. In the preamble LaTeX writes the following:

```
% The normal page style, with its headers
\pagestyle{fancy}
\fancypagestyle{normal}{%
   \fancyhead{} % clean the headers to
                % let place to what's next
   \fancyhead[RO,LE]{\thepage}
   \fancyhead[LO]{\nouppercase{\leftmark}}
   \fancyhead[RE]{\nouppercase{Our Great Book}}
   \fancyfoot{}
}
% The special page style
\fancypagestyle{special}{%
   \fancyhead{} % clean the headers
   \fancyhead[RO,LE]{\thepage}
   \fancyhead[LO]{\nouppercase{Introduction}}
   \fancyhead[RE]{\nouppercase{Our Great Book}}
   \fancyfoot{}
}
% The index page style
\fancypagestyle{index}{%
   \fancyhead{} % clean the headers
   \fancyhead[RO,LE]{\thepage}
   \fancyhead[LO]{\nouppercase{Index}}
   \fancyhead[RE]{\nouppercase{Index}}
   \fancyfoot{}
}
```

With this I have redefined the normal page style and created two new styles called "special" and "index" to use in the unnumbered chapter and table of contents, respectively, giving everyone the information it serves.

Now, at the beginning of the document, where the table of contents goes, insert a TEX box and in it type this:

```
\frontmatter
\setcounter{page}{1}
\pagestyle{index}
```

(the second line is usually not necessary). With this you tell LATEX that what follows is the space dedicated to the table of contents with the corresponding page style and that it has to number the pages with Roman numerals.

If you want to insist on using uppercase Roman numerals (seriously, people, *from where did you get that weird thing about lowercase roman numerals?* Oh, well) you could add the following:

```
\pagenumbering{Roman}
```

Just to be on the safe side.

Below that box, you need to go to Insert > List/Contents/References > Table of Contents and below it insert another TEX box with the following code:

```
\mainmatter
\setcounter{page}{1}
```

(this second line is usually not necessary either). This code indicates that now the good stuff begins, that we are already in the main topic.

At this point you need to insert the unnumbered chapter using the environment menu and write its title, for example "Introduction."

Use the same title defined for the Special page style!

On the line *immediately below the title* you need to insert another TEX box with the following content:

```
\pagestyle{special}
\addcontentsline{toc}{chapter}{Introduction}
```

The first line assigns the page style "special" to the content that follows, while the second line adds an entry to the table of contents pointing to that page, using the selected level (`chapter`) and the selected text (*Introduction*, here too, consistency must be maintained).

Once you have finished writing the introduction, it is time to move on. After applying (if applicable) the Part environment and then the Chapter environment (without the asterisk), add another TEX box with:

```
\pagestyle{normal}
```

so everything is back to "normality."

The rest of the document is written normally, you don't need more TEX boxes. At least, not because of this problem.

Bellow, some links that I have used to introduce myself to the subject:
http://tex.stackexchange.com/questions/37574/different-headers-for-different-chapters
http://wiki.lyx.org/FAQ/TOC#addcontentsline

13.5 First page of the chapter

By default, in book document classes the first page of each chapter uses a style called "plain." If you want to apply a different style you have to add, in the LATEX preamble, something like the following code:

```
\fancypagestyle{thefirst}{%
   \fancyhead{}
   \fancyhead[R]{\nouppercase{Our Great Book}}
   \fancyfoot[C]{Page \thepage}
}
\makeatletter
   \let\ps@plain\ps@thefirst
\makeatother
```

The first lines define a page style called "thefirst" while the last three lines (OK, only the middle one) change the page style for the first page of the chapters from "plain" to the new "thefirst" style. Thus the first page of each chapter will show, to the right of the header, the text "Our great book" while in the center of the footer you will see "Page" followed by the page number. Needless to say, I recommend that you do *not* copy this as is and adapt the code to your needs.

13.6 Blank pages

In a two-sided document with one of the book classes you will surely get automatically inserted blank pages: chapters start on right (odd) pages, so if the previous chapter ended also on an odd page in the middle you will have a blank page.

By default, for those blank pages LaTeX uses the same page style used in the body of the chapter, so the blank page will not be *really* blank: it would have the same headers and footers of the rest of the chapter.

To make those blank pages "really blank," you can use the `emptypage` package. In TeX Live this package is called

```
texlive-emptypage
```

Once installed, in the LaTeX preamble you need to add this line:

```
\usepackage{emptypage}
```

Now, what if you want a *different*, not fully blank page, but not the same as the body pages of the chapter either? You might need it to say "page intentionally left blank" or something like that.

If you look at the documentation for the `emptypage` package you will notice that all it does is redefine blank pages to use the "empty" style so taking the example you can write:

```
\fancypagestyle{rare}{%
   \fancyhead{} %
   \fancyhead[LE]{\thepage}
```

```
    \fancyhead[RE]{\nouppercase{Empty page}}
    \fancyfoot[CE]{Empty page}
}
\def\emptypage@emptypage{%
    \hbox{}%
    \thispagestyle{rare}%
    \newpage%
    \if@twocolumn%
    \hbox{}%
    \newpage%
    \fi%
}
\def\cleardoublepage{%
    \clearpage%
    \if@twoside%
    \ifodd\c@page%
    \else%
    \emptypage@emptypage%
    \fi%
    \fi%
}
```

I leave as homework the interpretation of this code and its possible modification to adapt it to any circumstance.

13.7 The page style for the Part environment

In a previous section I showed you how to create and apply a page style to change the contents of the header or footer of a block in your document. Now we have a similar problem, but with a more complex solution: changing the page style associated with the Part environment.

The Part environment, used to group chapters in "book" document classes, has a particular page style associated to it, and that page style might have, depending on the document class, a footer with the page number. Let's say you want to remove that page number, is it possible to use what we have seen above? Unfortunately, no.

The problem with the Part environment is that it is also associated with page breaks, so any instruction like `\thispagestyle` used before or after this environment will be applied to the page before or after and not to the one we want. You cannot place the `\thispagestyle` instruction inside the Part environment either, since this instruction cannot be nested. Then? Well, you have to "patch" the Part environment.

I found the following code in this *LaTeX Community* forum topic http://www.latex-community.org/forum/viewtopic.php?f=47&t=5809 What it does is to change the definition of the `\part` environment by assigning it the page style "empty", which has no headers or footers.

So, going to the LaTeX preamble, you need to write the following:

```
% \part pages without footer
\makeatletter
\renewcommand\part{%
  \if@openright
    \cleardoublepage
  \else
    \clearpage
  \fi
  \thispagestyle{empty}%
  \if@twocolumn
    \onecolumn
    \@tempswatrue
  \else
    \@tempswafalse
  \fi
  \null\vfil
  \secdef\@part\@spart}
\makeatother
```

Done. And yes, LaTeX can be *really* strange.

> Note: There is a simpler way to achieve this, but it does not always work. You have to patch the *Part* environment using the `etoolbox` command as follows:

```
\usepackage{etoolbox}
\patchcmd{\part}{\thispagestyle{plain}}{
   \thispagestyle{empty}}
   {}{\errmessage{Cannot patch! \string\part}}
```

But as I said: it doesn't always work. You will probably see the error message defined in the third line. That's the way it is.

Now, how to change the page style that follows the Part environment? In this book we have there different and inspired phrases...

```
\makeatletter
\def\@endpart{\vfil\newpage
              \if@twoside
               \if@openright
                \null
                \thispagestyle{empty}%
\vspace*{3cm}%
\begin{quote}%
\partnote
\end{quote}%
              \fi
              \fi
              \if@tempswa
                \twocolumn
              \fi}
\newcommand\partnote{}
\makeatother
```

Here you have several things. On the one hand you redefine the "endpart," the end of the part, on the other hand you define a "quote" environment with all and leading space and assign a page style to all this.

You can see how I created the character style for the sentence in the appendices B and C. At this point, you should be able to understand all that code.

Chapter 14
Figures and other floats

The concept of "float" has already been introduced in subsection 5.1.7: an object inside which figures or tables can be placed and whose position will be controlled by LaTeX to place it in the correct position when the document is compiled.

In this chapter I will show you some of the options that exist to modify the formatting of these floats as well as their content. Although for simplicity of writing I will speak mostly of figures, what I will comment here serves in principle for all types of floats, so every time you see "figure" what you have to read is "figures/tables/etc."

14.1 Size of inserted images

When you insert an image in LyX, that image will be displayed in its actual size when the document is compiled, and that means that if the image is larger than the page, it will go through the margin. In fact, some images will not only "go through", but will indeed *demolish* the margins. Let's see how to solve this and other problems.

14.1.1 Adjusting the image size in the compiled document

Either while inserting a picture or later with a click on it, the menu shown in figure 14.1 will open.

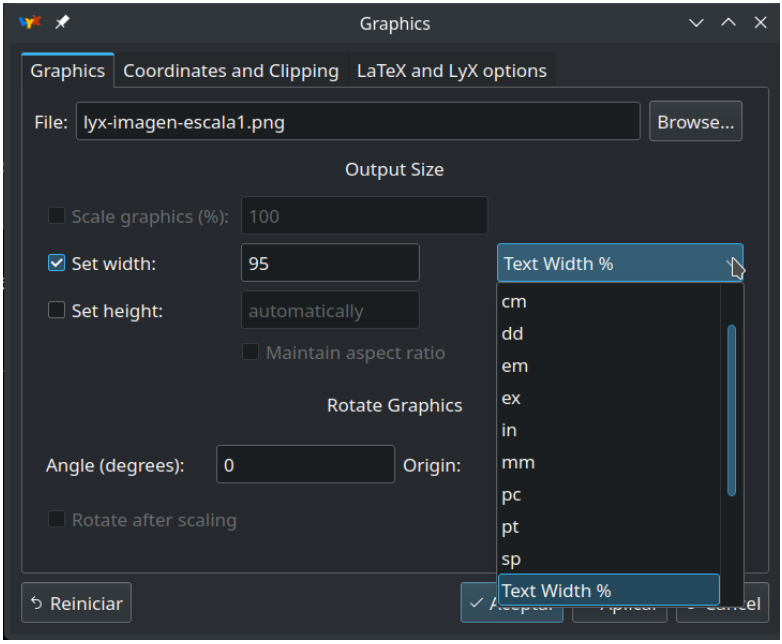


FIGURE 14.1: *Modifying the width when inserting an image*

As you can see, in the GRAPHICS tab you have several options, such as giving a percentage with respect to the image itself, or setting the width / height. For the latter you can choose an absolute size or a percentage with respect to the page, or the column, whatever works for you. The option selected in the screenshot will allow you to choose to scale the image to the selected percentage, for example, so that it occupies 80% of the width of the text. You could also choose the width of the page, or its height, in short, I'll let you explore the options.

Of course, it is important to note here that this will force *any* image to occupy *exactly* the percentage of the chosen text area: larger images will be automatically scaled down, which is perfectly fine, *but the smaller ones will be scaled up*, which is very bad, so I recommend you pay close attention and use this option only when you know that the image you are inserting is too large for the target page.

Since version 2.4, it is possible to choose the alignment of the *content* inside the floats. To do this globally in the document it is enough to go to Document » Settings » Float Settings to make, for example, the image centered in the float. It is possible also to modify this option for a particular image from the properties of the float itself (right click on the float label » Settings).

14.1.2 Size of the image in the editing window

As can be seen in figure 14.2, in the tab with the LaTeX / LyX options you can choose how the image is displayed (or if it should not be displayed) in the LyX window while editing the document: while working it is not necessary to see the image in its full resolution so it might be convenient to reduce it quite a lot or even decide not to display it.

The image shows something else, but we discuss that in the next subsection.

14.1.3 To avoid repetition: graphic groups

Now, do we have to do what is described in the previous sections *every single time*?

No!

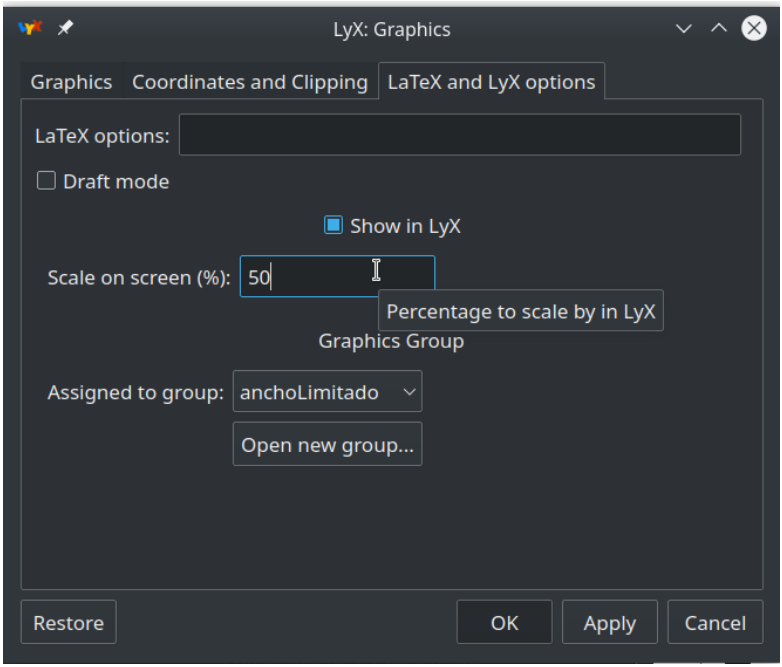


FIGURE 14.2: *How to display images during editing*

As can *also* be seen in figure 14.2, there is the option GRAPHIC GROUP. With this option, you can assign a name to a particular graphic configuration, so you can use it without effort in the next picture: with OPEN NEW GROUP it is possible to assign a name to the particular configuration you want to save, while with the menu ASSIGNED TO GROUP it is possible to later apply that setting to any picture.

Easier than that, impossible.

14.2 Wrapped figures

In this section I will talk about figures surrounded by text, like the one that you can see right here.



FIGURE 14.3: *A cute penguin I adopted online (to the anonymous author of this and other penguins, thank you)*

But before continuing, I must warn that this is one of the few points that LaTeX does not manage to handle very well: `wrapfig`, the package I discuss here, can be quite unstable and provide unpredictable results. For example, with the default options, if when compiling the document the figure falls near the bottom margin it could "break" the page footer and even leave a gap on top of the next page (I let you do some tests, it can be even fun). The picture may even *disappear* from the document without a single warning. The package does not verify if there is enough space on the page to put the picture there.

I'll tell you how to fix that. More or less.

LyX offers full support for this package, so to use it just go to Insert > Float > Figure wrap float (you can also insert wrapped tables from there). It is important to keep in mind that if you place the wrapped float between two paragraphs, the figure will be surrounded by the second one, while if you place it between two words within the same paragraph, it will be sur-

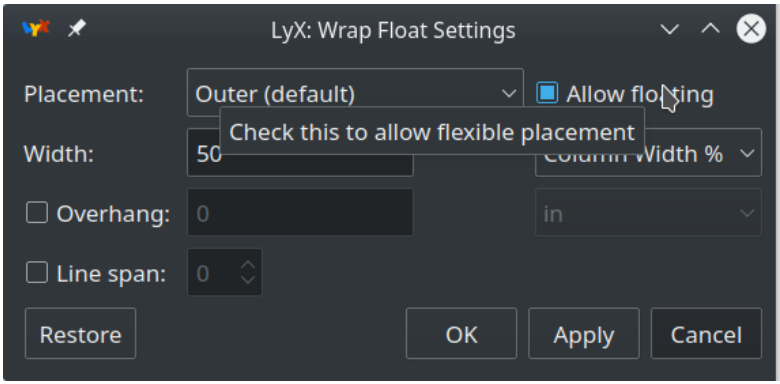


FIGURE 14.4: *Setting up the wrap float*

rounded by the lines of text starting from the one in which those words are located.

This implies the need for a lot of care and manual control of the position of the wrapped figures, so the question arises whether it is possible to make them float like normal figures. And furthermore, how to choose on which side of the page the figure will appear or the fraction of the page width it will occupy?

With a right click on the wrap float label, you can access a configuration menu (figure 14.4). The placement options are OUTER (to the right on odd pages and to the left on even pages of two-sided documents), INNER (the other way round), LEFT and RIGHT. The option ALLOW FLOATING is the one that correct the previously commented problem.

Done? Well, not quite:

- Wrapped floats cannot be used in combination with numbered or bulleted lists.
- If the wrapped float is in the last paragraph and the text is not enough to wrap it, depending on whether float has been activated or not you could have a figure that scrolls after the text and does not respect the chosen side (with float) or a "half-wrapped" figure (without float).
- If you have two wrapped figures close together, the result can be bad

(again, you can test).

- Although the numbering is integrated with that of the normal figures, it could happen that they do not appear in order, giving us, for example, the normal figure number 5 at the top of the page and the wrapped figure number 4 further down.

- ...

In short, it is a package whose results can bring some headaches, requiring a lot of manual control of the document before it can be considered "ready."

Now you know!

14.3 Setting the caption text formatting for all floats

The easiest way to set up the captions of figures and tables in a LaTeX document is to use the `caption` package.

On Linux and with TeXLive (you need to install `texlive-caption` and `texlive-caption-doc`) the documentation is installed at

/usr/share/texmf/doc/latex/caption/caption-eng.pdf

The documentation is quite simple so in this short section I will only give an everyday example to see how the different parameters of this package are used.

The idea of this example is to set the figure and table captions to be:

- justified but with the last line centered,

- have a label formatted in Small Caps,

- set the caption text in *italics*.

To achieve this, in the LaTeX preamble you must write the following:

```
\usepackage{caption}
\captionsetup{justification=centerlast,%
    labelfont=sc,textfont=it}
```

The first line declares the `caption` package, while the second line set it up with the three options discussed above.

Other options offered by `caption`: change margins, the size or the family of the font used, line spacing, text color, modify or even remove the label name (so that only the number is displayed), insert a break between the label and the caption text, the distance between the picture and the caption (the `skip` option), etc.

It is interesting to note that the `\captionsetup` command can be used not only on the preamble: if you place it in a particular float, it will allow you to set *that* caption.

Another example. We might want a separator line to be displayed below each caption. For this you can use the `\hrulefill` instruction as follows:

```
\usepackage{caption}
\DeclareCaptionFormat{myformat}{#1#2#3\hrulefill}
\captionsetup[figure]{format=myformat}
```

Imagine now that you want to modify this line to be a particular color and thicker:

```
\usepackage{xcolor}
\definecolor{rojito}{rgb}{.8,.2,.1}
\usepackage{caption}
\DeclareCaptionFormat{miformato}{#1#2#3\color{rojito}
    \def\hrulefill{\leavevmode\leaders\hrule
        height 2pt\hfill\kern\z@}\hrulefill}
\captionsetup[figure]{format=miformato}
```

I'll leave you with the details to see how to combine this with the above.

There is one last trick related to the setting of floats: make the caption appear *on the side* instead of above/bellow. But that trick will require working with custom modules and local layouts with grand style, so I'll leave it for the section 16.3.

14.4 Choose page style for a particular float

We all appreciate LaTeX for its enormous ability to place floats in our documents. Whether they are figures or tables, you drop them in the editor and when the document is compiled they will go to the best possible place: if there is no other float already there, to the top of the page where they were inserted, if the top is occupied to the end or to the next page, and if the float is very large on a page of its own, but always close to the insertion point.

Now, imagine one of those large floats on a page of its own, is there any way to make the page of that float have a different style? For example, you might want that page to not show the header or footer, just the float. That is, make only the page of the float we are considering to have the `empty` page style.

As it turns out, it's very easy. First you have to make sure you have installed the `sttools` package set, which provides the `floatpag` package. Once installed, you must remember to reconfigure LyX. Finally, in the preamble of the document type the following:

```
\usepackage{floatpag}
```

while in the float itself add a TeX box with this instruction:

```
\thisfloatpagestyle{empty}
```

If you want another page style, instead of `empty` you use the one you want and that's it, I already explained those things in the chapter 13.

14.5 Rename floats

LaTeX offers by default two kinds of floats: Figure and Table. LyX adds a third one: Algorithm.

Now, can we change those names? Maybe we need to use *Diagram* instead of Figure or *Script* instead of Algorithm.

Yes, of course we can! But beware, that changing the name to the label is not enough: floats may have their own indexes that must be changed as well to avoid confusion of the readers.

For this you can use Babel instructions to simulate a translation of the tags. Since Babel is already called by LyX you don't have to call it again, you just have to configure it properly, so let's get to the preamble:

```
\AtBeginDocument{
  \addto\captionsenglish{
  \def\figurename{Diagram}
  \def\listfigurename{Index of diagrams}
}}
```

to change Figure by Diagram. If you want to change the name used by the table floats, you have to use `\tablename{new-name}` for the label, while for the index the command is `\listtablename{Index of new-name(s)}`.

For a list of the names that can be modified, see:
https://texfaq.org/FAQ-fixnam

14.6 Change the numbering scheme of figures and tables

This one is easy, in fact I already showed you something similar in chapter 12. To number the Figures with Roman numerals (in uppercase, please!), you have to write in the LaTeX preamble:

```
\renewcommand{\thefigure}{\Roman{figure}}
```

For tables simply change both instances of `figure` to `table`. And to use alphabetical characters instead, well, you know: chapter 6.

Chapter 15
Drop Caps and similar topics



Surely you have found documents where you see what happens in this particular paragraph: the first letter of the first word in the first line extends over several lines, perhaps showing (as here) a special, ornate font, with the rest of that first word written in small caps.

This style is what is known as "Initials" or "Drop Caps." To get this effect in LaTeX/LyX you need to use the `lettrine` package.

15.1 Something traditional



In LyX the `Lettrine` package is supported from the graphical interface and in section 6.3 of the manual that you find in the menu Help ⟩ Embedded Objects one of the ways to use it is explained in detail. Because as I will show you below there is *another* way, which is much simpler. The manual gives you the maximum flexibility to choose the shape of the initial letter every time you want to use it, but what if you plan to use Initial several times in the document (for example, in the first paragraph of each chapter) and want to be sure they always look the same? Doing it in the way indicated in the LyX manual could become a bit cumbersome, so it is convenient to find a way to automate the process.

To begin with, as always, you must be sure that the package to be used is correctly installed in LaTeX and if not, install it and then reconfigure LyX

to recognize it. The documentation for the `Lettrine` package on Linux is installed at

/usr/share/texmf/doc/latex/lettrine/lettrine.pdf
/usr/share/texmf/doc/latex/lettrine/demo.pdf

Now go to Document > Settings > Modules (I already told you about modules in chapter 9) to enable the module called Initials (Drop Caps).

Click on Apply, not OK, there is still some work to do in this menu! As it usually happens in these cases, you have to go to the LaTeX Preamble to add some code.

This document is configured to use XeTeX with EB Garamond as the base font and defines an alias to a font called Pali,[1] using the stylistics alternates feature (+salt) to serve as the capital letter.

This is what I have written in the preamble:

```
\newfontfamily{\Iniciales}[
    RawFeature={+salt}]{Pali}
\renewcommand{\LettrineFontHook}{\Iniciales}
\setcounter{DefaultLines}{4}
```

The first instruction defines a new font family (we have already discussed this topic in detail in the section 7.3), a definition that is used in the second instruction to tell the `Lettrine` package which font to use for the Initial letters. The third instruction tells how many lines the Initial letter should occupy.

It is possible to control the vertical positioning of the Initial character by adding a fourth line:

```
\renewcommand{\DefaultLraise}{0.35}
```

where the value 0.35 can be changed to anything needed. Generally it is not necessary to use this instruction.

Done all that, when you want to add a new Initial select the first character of the paragraph and right click > Custom Text Style > Initial. To have the rest of the first word in small caps you must enclose it in braces and put everything (rest of word and braces) in a TeX box (just select everything and

1. http://www.softerviews.org/Fonts.html#Pali

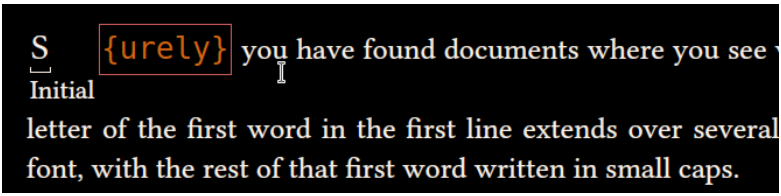


FIGURE 15.1: *Initials during document edition in LyX*

press Ctrl L). In the editing window this will look like what is shown in the figure 15.1, in the output... well, you have already seen it.

The `Lettrine` package documentation is short, so I'm sure you will have no problem finding more options to adjust everything quickly and easily.

15.2 Be careful with the number of lines!

PRECISELY, you must be careful!

Something very important to keep in mind is that if the paragraph on which you activate the Initial letter is short (for example, one or two lines) and the initial letter is large (for example, four lines) *the initial letter will overlap the following paragraph*.

Something that certainly did not happen here. The trick? Simply join the paragraph with the initial to the following paragraph by simulating the paragraph break with a line break.

In LaTeXline breaks are entered with two consecutive backslashes "\\" (without quotes), but in LyX it is much easier: just press Ctrl ↵. In the LyX window we will see an "angled arrow" in red.

Being everything in the same paragraph, the nasty overlap is avoided, *but the second "paragraph" does not show the first line indent.* But we have indent here. Trick? Here there is a little a problem, because the normal LaTeX codes for inserting horizontal spaces do not work at the beginning of a paragraph, then? Insert » Formatting » Phantom » Horizontal Phantom and in the resulting box write, for example, a couple of "m." Those "m" will be shown during edition between angular brackets and crossed out, but don't worry: the result will give you what is shown at the beginning of this

subsection.

15.3 A variant: a “raised” initial

Here the trick is a small variation of the previously shown code.

In the LaTeX preamble (of course, after enabling the Initials module and everything else) you have to write the following:

```
\newfontfamily{\Iniciales}[
   Scale=2,RawFeature={+salt}]{Pali}
\renewcommand{\LettrineFontHook}{\Iniciales}
\setcounter{DefaultLines}{1}
```

As you can see everything is being defined almost in the same way as in the previous section, but adding an instruction to make the font defined in the new family `\Iniciales` to be displayed as twice its size (clearly, you can always play with the `Scale` parameter) while the number of lines has been changed to 1.

This is enough to ensure that the initial letter is not lowered but is displayed, larger, on the same line of text.

15.4 A challenge

Right at this point I leave you with a small challenge: to combine the previous sections to achieve an Initial that takes several lines (say, two), but that also goes up enough to achieve what you see in this paragraph.

C’mon, with what you’ve seen, it should be easy!

And no, you will not see the solution in the appendix with the LaTeX preamble code for this document since this initial letter was created “in

place" using the Initial environment with custom instructions inserted *just for this paragraph*.

You cannot cheat!

15.5 An alternative: first line of paragraph in small caps

An alternative to the use of Initials/drop caps is to make the first line of the first paragraph be written in small caps, something that can be seen in some magazines (and in this paragraph, of course).

The only way I have found to get this effect in an (almost) automatic way with LaTeX is to use the `magaz` package.

The documentation for this package is installed, on Linux and with TeXLive, in

/usr/share/texmf/doc/latex/magaz/magaz.pdf

However, this package is somewhat capricious in that it not only gets along poorly with the `lettrine` package, it also does not accept equations, particular formatting or even remove the first line indent it introduces. But in its simplest form it is very easy to use.

Hyphenation can be a problem: sometimes a syllable may disappear, but do not ask me what triggers that.

In its favor, we can say that is quite easy to use.

To use it, in the LaTeX preamble write

```
\usepackage{magaz}
```

and then, in the paragraph you want to modify, open two TeX boxes, one at the beginning of the paragraph with the instruction

```
\FirstLine{
```

and the other later (not necessarily at the end of the paragraph, just enough text should be taken to cover the first line) with an

```
}
```

to close the instruction.

The package will automatically change the format of the first line without any further work on the part of the writer, even taking care (though not always: you have to check the result!) of cases where the last word of the first line is broken.

It is important to note that the `\FirstLine` command *must* be at the beginning of the paragraph, otherwise it will give you an error.

This package offers more options. For example, it is possible to select a different format for the font used in the first line with the instruction

```
\renewcommand\FirstLineFont{\
   <formatting instructions>}
```

but as the brief documentation of the package is clear enough I will not go into those details.

Chapter 16
Extras

In this chapter I will present to you some details that are, perhaps, a bit *too* particular.

On the one hand, you will come across tricks that I have used in this document, such as inserting menu addresses in a way that they look like actual menu paths or giving keyboard shortcuts that look like actual keys (I give you two methods for this).

On the other hand you will also see a few things that I have not done for this book, like adding more floats, using lateral captions for figures and tables, how to transform a module into local format or even how to surround a paragraph with a "color box."

Finally I'll give you a brief overview of the huge variety of things I haven't gotten to in this book. Remember: LaTeX is *huge*.

16.1 Describe keys and menus: the **menukeys** package

Throughout this book I have shown many times keyboard shortcuts and menus, such as Ctrl + ⇆ or Document » Settings » LaTeX Preamble.

There is no official way to get these effects, so I will show you a method to do both with the `menukeys` package. In the next section I will show another method to do the first one.

The `menukeys` manual is generally installed in
/usr/share/texmf/doc/latex/menukeys/menukeys.pdf

It is really complete, but it presupposes a minimum level of knowledge on the part of the reader, so although all the options are clearly written, many times it is not clearly stated how to use them. Let's go into it step by step.

16.1.1 What is **menukeys**

This package offers three macros:

- A macro to show keyboard shortcuts as keys.
- A macro to show menus in highlighted form.
- A macro to show file path in highlighted form.

The easiest way to use it is to enable the use of non-TEX fonts. I mean, sure, it is *possible* to use this package with normal LATEX fonts *just* by changing the file's encoding and all that comes with that... but let's avoid that topic.

After checking that the package is installed on the system and asking LYX to note that the package is available (you always have to reconfigure!), you have to go to the LATEX preamble to write the following:

```
\usepackage{menukeys}
```

which will activate menukeys... with their default values, which include using Mac keyboard keys instead of PC keyboard keys. That is, unless you use the apple system, keys like Enter, Tab, Caps Lock and others will look very strange to you.

To use the PC-like keys you have to pass a parameter to the package to tell it that the operating system you are interested in is (ouch!) *win*, modifying the instruction shown above as follows:

```
\usepackage[os=win]{menukeys}
```

The menukeys macro offers several key "shapes" and different presentations for menus to choose from. For example, if you want to use the shaded keys and make the menus look more rounded you have to add two lines in the LATEX preamble to leave it like this:

```
\usepackage[os=win]{menukeys}
\renewmenumacro{\keys}{shadowedroundedkeys}
\renewmenumacro{\menu}{roundedmenus}
```

A list with all the available styles can be found on section 4.2 of the package manual.

16.1.2 Using **menukeys** in LyX

The are two ways to use this package in LyX: a "brute force" method using a TeX box or a "local layout" (section 9.3).

However, before moving on to the second (and more convenient) of the two options, it will be useful to know the first one, this way you will learn which code you need to write.

So, inside a TeX box write, for a directory:

```
\directory{/usr/share/texmf/doc/latex/
   menukeys/menukeys.pdf}
```

while for a menu you have two options, either use the default separator (the "greater than" symbol >):

```
\menu{Document > Settings > LaTeX Preamble}
```

or explicitly indicate *each time* which separator you want to use to indicate each step:

```
\menu[,]{Document , Settings , LaTeX Preamble}
```

It is important to note that for words marked by this package no hyphenation will occur. Even if you have a phrase as one of the levels of a menu, it will remain complete, without breaking, so that a particularly long menu may pierce the right margin. When using this package it is always necessary to check the output of the compiled document to avoid unpleasant surprises.

To represent a key you have to type the following:

```
\keys{\tab}
```

which will give you ⇤⇥... which is not the ↹ you have seen so far through out the whole book, but I will come back to that technicality in the next section.

On section 4.5 of the menukeys manual you will find a complete list with all available keys. It is important to keep in mind that if you type a key sequence without the slash, for example

```
\keys{Meta}
```

this sequence will be considered as a single key Meta.

For more keys, more \keys commands or, if you want a separator, just one command with a "plus." For example,

```
\keys{\ctrl}\keys{N}
```

will give you Ctrl N, while

```
\keys{\ctrl+N}
```

will give you Ctrl + N.

Now that you know how everything works, go to Document › Settings › Local Layout in order to create a character style to apply the "menu formatting." You just need to write the following code there:

```
Format 95
  InsetLayout Flex:MenuKeys
  LyxType           charstyle
  LabelString       menukey
  LatexType         command
  LatexName         menu
  Preamble
    \usepackage[os=win]{menukeys}
  EndPreamble
  ResetsFont true
End
```

Now in the document you just have to write the menu path needed separating each step with a ">" then select everything and with a right click > Custom Text Styles select "MenuKey." Doing so, in the LyX editing window you will see

text Document > Settings > LaTeX Preamble more text
menukey

while in the output you will see:

text Document > Settings > LaTeX Preamble more text

I leave it to you as a homework to figure out how to create a local layout for keyboard shortcuts, after all that's not what I have done for this book.

And that is all about this topic on my part. The package offers many more options, including the ability to define your own styles, but I will not go into those details: with all what we have said, the package documentation should be clear now.

That and the fact that I quite like the default options and I have no intention to change them. I mean, other than the Mac keys. Writing "win" in my documents... that was tough.

16.2 Another way to represent keys

The method presented in the previous section to represent keys has a small problem: it does not work in headings. That is, if you want to represent a keyboard shortcut in a section heading using `menukeys`, sorry, you cannot, the document does not compile.

The solution I use in this book to represent keys is to create a character style that applies the Libertinus Keyboard font, which offers several ligatures that automatically change the *name* of a key to the *key representation*.

Configured the document to use non-TEX fonts and having installed the Libertinus fonts[1], in the preamble you need to create a font family with the following code:

```
\newfontfamily{\libertinusKB}{Libertinus Key-
board}
```

while in the local layout you need to type something like this:

1. https://github.com/alerque/libertinus

```
Format 95
  InsetLayout Flex:LibKB
  LyxType       charstyle
  LabelString   LibKB
  LatexType     command
  LatexName     libkb
  Preamble
    \newcommand*{\libkb}[1]{{\libertinusKB #1}}
  EndPreamble
  ResetsFont    true
  Font
    Family      typewriter
  EndFont
End
```

That's it, you now have the new style in right click > Custom Text Styles > LibKB. The text "Ctrl" will give you Ctrl, Alt will give you Alt, AltGr AltGr, Space [ ], Enter ↵, Esc Esc, Back ←, Del Del, as well as you can type separate characters such as A, b, →, and so on.

16.3 Side captions for figures/tables

We have already seen in section 9.1 how to add modules to LyX, now I will show you how to use, *and modify*, one that is quite interesting: `sidecap`.

This module (actually, two modules) adds more floats that place the captions on the side instead of at the top or bottom.

The `sidecap` module uses the LaTeX package of the same name to perform this task, integrating it into the LyX workflow to make everything easier.

From this page:

http://theorie.physik.uni-konstanz.de/jrioux/lyx/

it is possible to download the following files (you must pick the LyX 2 versions!):

- `sidecap.module`: it is the main version. It creates floats whose numbering integrates without a glitch with normal floats and with a for-

matting that follows the settings established for them.

- `sidecap-starred.module`: like the previous one, it places the caption at the side of the image, but eliminates the numbering.

With the module installed and after LyX gets reconfigured, you can now go to Document > Settings > Modules to activate the SIDECAP module. After that, just go to Insert > Float > ... to insert the floats labeled as "side caption" you need.

Now, this module uses the default options for the `sidecap` package, which consist of placing the caption on the outside of the page (to the right on odd pages and to the left on even pages for two-sided documents) with the caption aligned to the bottom of the figure. Changing the latter is very simple. In the LaTeX preamble you can write the following:

```
\sidecaptionvpos{figure}{c}
```

to make the caption vertically centered. If you change the `c` to a `t` the caption will be aligned to the top of the image. To set the vertical alignment of table captions it is enough to change `figure` for `table` in the previous code.

Unfortunately changing the side on which the caption is placed is not so simple. If the default option is appropriate this module is very useful, but if you want something else (for example, to place the caption on the inner side of the page, or always on the right/left) what you can do is to open the module file in a text editor, copy its content from the Format statement to the end and paste it as a new local format (section 9.3). You can now modify the line

```
\usepackage{sidecap}
```

to something like

```
\usepackage[innercaption]{sidecap}
```

(or `leftcaption` or `rightcaption`, whatever you need). Now all you have to do is click on the CONVERT TO THE NEW FORMAT button (the module is a bit old), then click on VALIDATE and finally on OK.

Done!

Not only have I shown you how to use an external module, you now also know how to convert it into a local format and change it according to your needs. You should not even find it difficult to use all this to understand how to create a new module.

Not bad for an *extra* section!

16.4 Adding a new float

An example without explanation on how to add a feature to LyX: add a new float using the `photo` package.

In the local layout, you have to type the following:

```
Format 95
AddToPreamble
  \usepackage[]{photo}
  \usepackage[english]{babel}
  \addto\captionsenglish{
  \def\photoname{Photograph}
  \def\listphotoname{Index of Photographs}
 }
EndPreamble
Float
 Type photo
 GuiName "Photo"
 Placement tbp
 Extension loscf
 NumberWithin none
 Style plain
 ListCommand listofphotos
 ListName "Index of Photographs"
 RefPrefix Photo
 IsPredefined 1
 UsesFloatPkg 0
End
```

Now you can go to Insert › Float. Also check Insert › List/Contents/References. Enjoy.

16.5 Colored boxes

Imagine you want to get the following effect in your document:

This is the colored box title

This is the content of the little colored box

That is, a box with colored borders, possibly with a title and other kinds of pyrotechnics.

For this and much more there is the LaTeX package `tcolorbox`.

LyX offers partial graphical support for this package. To use it, go to Document › Settings › Modules to activate the Fancy Colored Boxes module. When you are ready go to Insert › Custom Insets (without the module we are talking about here, that menu entry would be empty) to choose one of the available options.

Color Box provides a box that takes the full width of the text area, Fit Color Box provides a box that adapts to the content, and so on.

If you choose for example the option Color Box you will find in the LyX editing window one of those insets like the one for the footnotes or the floats: a label on the left that indicates what kind of "box" it is and a space to the right to write in.

Writing some content in the box and compiling the document you will find in the output a gray box without title, rather disappointing if you ask me. Of course, that is because you need to pass extra parameters to the package to obtain the needed effect.

With the cursor within the box, either with Insert › Color Box Options or through a right click › Color Box Options you will get a box inside the box where to write the necessary code to set the parameters. For example, to get the box opening this section I have written

```
colback=red!5!white,colframe=red!75!black,title=This
```

```
is the colored box title
```

What all that mean, you may ask? Well, the `tcolorbox` documentation, which in Linux can be found in

/usr/share/texmf/doc/latex/tcolorbox/tcolorbox.pdf

takes more than 530 pages, so it is completely beyond the scope of this brief section to discuss all the available options. Even a full chapter would not be enough: the package offers macros to create indexes, theorems, explanatory tables... for examples showing the power of this package it is possible to consult the following document:

/usr/share/texmf/doc/latex/tcolorbox/tcolorbox-example.pdf

It is also possible to read the manual for this module that comes with LyX and can be found under Help » Specific Manuals » Colored boxes.

Now that you know the tool exists, you have some material to keep you entertained.

16.6 Things I have not talked about

LyX offers a huge arrange of possibilities to take advantage of the immeasurable list of LaTeX & co. capabilities.

In this misleadingly short section I will give a cursory list of some of the topics that, although interesting, I have decided not to touch on in order to keep the book from growing out of proportion.

16.6.1 BibLaTeX support

As we discussed in section 5.3, LyX offers support for BibLaTeX.

This topic is mainly of interest to academics and requires some work to understand how it works. You know, the package documentation and Internet are your friends.

16.6.2 Presenting code with `listings`

LyX offers support for the `listings` package, which provides the ability to format code in documents.

With Insert ⟩ Program Listing you get a box that allows you to type the code you want and with a right click on that box you can quickly select the programming language, its dialect, font, tab length, and so on. In this menu you can also pass parameters to the package to color the instructions and anything else that is useful.

In chapter 8 of the manual you find in Help ⟩ Embedded Objects there is more information about this topic.

16.6.3 PDF forms

It turns out that you can also use LyX to create "for the office" documents. Under Help ⟩ Specific Manuals ⟩ PDF forms you can find a detailed description of this tool.

16.6.4 **Sweave** and **knitr**

Under Help ⟩ Specific Manuals ⟩ Sweave / Knitr you can find two brief manuals that explain how to use these interesting LaTeX packages that allow you to work with the R statistical program from LyX.

16.6.5 Computer Algebra Systems

If you right-click inside the LyX equation editor, you will see the option Use Computer Algebra System (CAS) and there several possibilities like Octave, Maxima, etcetera, to quickly evaluate the inserted expression. For example, selecting Maxima in the expression[2] $\int_a^b x^2 dx$ you will get $\int_a^b x^2 dx = \frac{b^3}{3} - \frac{a^3}{3}$.

16.6.6 And much, much more

I'd better stop here, because adding a list of things I haven't talked about would *also* be too long.

Creating Feynman diagrams, writing musical scores with LilyPond, linguistics, Beamer presentations, the Xy-pic[3] package for making all kinds of graphics *from within* LaTeX, even writing in Braille find direct support in the graphical interface of LyX.

2. Of course, you need Maxima to be installed in the system!
3. https://ctan.org/pkg/xypic

And if you think that what I have presented to you in this third part of the book has been a LaTeX code mashup, I have shown you only a small fraction of what is possible. Things like generating paragraphs with arbitrary shapes or even using LaTeX to perform mathematical calculations is within the reach of the enthusiastic programmer: don't forget that TeX is a Turing-complete programming language!

Do not forget that TeX is a *Turing complete* programming language!

No one has said that all these things are easy and in the vast majority of cases they are not even worth the extra work. For example, controlling the shapes of the paragraphs involves a lot of manual work, but it is certainly possible.

My intention with this book was to open a road. It is up to you to decide how far you want to go.

As they say, *happy LyXing!*

Part IV

Appendixes

Where several details about the format of this book are presented

The grown-ups are very strange

Appendix A
All about typeface

Choosing the right typeface for a text is one of those topics where you have more answers than people answering.

This is not a book about design, but an explanation of how to use LyX to design. I have no background in graphic design so don't expect advice. But I can, however, comment on what I use.

A.1 Fonts used in this book

As already mentioned, the body text font used is EB Garamond:

http://www.georgduffner.at/ebgaramond/index.html

However, for this edition I have used Octavio Pardo's version, which offers several advantages (e.g., bold) along with some disadvantages (no optical sizes):

https://github.com/octaviopardo/EBGaramond12

Splendid recreation of Claude Garamond's mid-16th century designs, this font has (at least to my eyes) unparalleled beauty. In order to have the long-tailed *Q* I have activated the OpenType `+ss06` option. I have overcome the lack of optical sizes with what was discussed in the section 7.6

For headings, I have also used another garamond, Cormorant:

https://github.com/CatharsisFonts/Cormorant

This font is too detailed and with a too high contrast to be used as the main font, but for large sizes it is just perfect.

Combining typefaces harmoniously is a difficult art. While some projects make things (relatively) easy by providing "families" of fonts (Libertinus, Source pro, PT, Alegreya, etc.), which sans font can we use that match a garamond? From the people who created Cormorant comes an interesting proposal: Ysabeau

https://github.com/CatharsisFonts/Ysabeau

This font is in the middle of development, but I had no problems using it in this text.

I don't know of any typewriter font designed to go side by side with a garamond, so in the end I decided to opt for a classic like DejaVu Sans Mono:

https://dejavu-fonts.github.io/

For the mathematical formulae I used a new project called Garamond-Math. I have talked about that font in chapter 7.

In the examples used throughout this book, several other fonts where used.

A.2 More about typefaces

For more information about typefaces with free licenses, including (more or less) detailed analysis of their advanced features, you can take a look at the "font review" category in my personal blog:

https://frommindtotype.wordpress.com/category/software/font_review/

In particular I would like to highlight a series of articles whose objectives are too general to be included in this book. The first one is here:
https://frommindtotype.wordpress.com/2017/11/13/typographical-concepts-part-1/

Appendix B

Local Layout

To develop this text, I have created several character styles to simplify the writing process. Below, the content of Document » Settings » Local Layout.

It is important to note that some of the formatting defined here depends on instructions already existing in the preamble, which I will detail in the appendix C. And of course, *in a normal document it is not necessary to define so many styles:* being this document an example of the topics it deals with make such a list necessary. In fact, I think I have only used the textfrac style on two occasions... or was it one? Dammit!

Always keep your styles under control! Formatting helps you to better express your ideas, but only up to a certain limit: if the presentation of the document is too varied, it will end up distracting the reader from what you want to convey. While it is true that going too far for the sake of minimalism can result in an unclear presentation, you should not forget that complexity is not synonymous with elegance.

```
Format 95
   InsetLayout Flex:DiscrLig
   LyxType                 charstyle
   LabelString             discrlig
   LatexType               command
   LatexName               discrlig
   Preamble
     \newcommand*{\discrlig}[1]{{
```

```
            \addfontfeature{%
            RawFeature={+dlig}} #1}}
   EndPreamble
   ResetsFont             true
End
   InsetLayout Flex:TextFrac
   LyxType                charstyle
   LabelString            textfrac
   LatexType              command
   LatexName              textfrac
   Preamble
     \newcommand*{\textfrac}[1]{{
            \addfontfeature{%
            RawFeature={+frac}} #1}}
   EndPreamble
   ResetsFont              true
End
   InsetLayout Flex:Kleymissky
   LyxType                 charstyle
   LabelString             kley
   LatexType               command
   LatexName               kley
   Preamble
     \newfontfamily{\kleym}[]{Kleymissky}
     \newcommand*{\kley}[1]{{\kleym #1}}
   EndPreamble
   ResetsFont              true
End
   InsetLayout Flex:MenuKeys
   LyxType                 charstyle
   LabelString             menukey
   LatexType               command
   LatexName               menu
   ResetsFont              true
End
```

```
    InsetLayout Flex:KeysKeys
    LyxType                 charstyle
    LabelString             okey
    LatexType               command
    LatexName               keys
    ResetsFont              true
  End
    InsetLayout Flex:sinserif
    LyxType                 charstyle
    LabelString             sinserif
    LatexType               command
    LatexName               sinserif
    Preamble
      \newcommand*{\sinserif}[1]{{\sffamily #1}}
    EndPreamble
    Font
      Family                sans
    EndFont
    ResetsFont              true
  End
    InsetLayout Flex:ttmono
    LyxType                 charstyle
    LabelString             ttmono
    LatexType               command
    LatexName               ttmono
    Preamble
      \newcommand*{\ttmono}[1]{{\ttfamily #1}}
    EndPreamble
    Font
      Family                Typewriter
    EndFont
    ResetsFont              true
  End
    InsetLayout Flex:LibKB
    LyxType       charstyle
```

```
  LabelString   LibKB
  LatexType     command
  LatexName     libkb
  Preamble
    \newfontfam-
ily{\libertinusKB}{Libertinus Keyboard}
    \newcommand*{\libkb}[1]{{\libertinusKB #1}}
  EndPreamble
  ResetsFont    true
  Font
    Family      typewriter
  EndFont
   ResetsFont              true
End
   InsetLayout Flex:LogoTipo
   LyxType                 charstyle
   LabelString             HoLogo
   LatexType               command
   LatexName               loguito
   Preamble
     \newcommand*{\loguito}[1]{{\hologo{#1}}}
   EndPreamble
   ResetsFont              true
End
   InsetLayout Flex:PartNote
   LyxType                 charstyle
   LabelString             PartNote
   LatexType               command
   LatexName               notitas
   Preamble
     \newcommand*{\notitas}[1]{{
        \partnote\itshape #1}}
   EndPreamble
   ResetsFont              true
End
```

Appendix C

LaTeX preamble

Below you will find what I have written in the preamble of the document. In the first lines I define the main font of the text. This means that I have set the corresponding option in the document settings to *default*, so don't forget that! Also, do not forget that I have selected Non-TeX default fonts so that I can set the mathematical font and that I have activated the use of microtypography in the document fonts menu.

```
\makeatletter
\renewcommand\@makefntext[1]{%
   \noindent\makebox[0pt][r]
    {\hyperref[\BackrefFootnoteTag]{
\@thefnmark.\space}}#1}
\makeatother
\usepackage{footnotebackref}
%------------
\setmainfont{EB Garamond}[
   Mapping=tex-text,
   RawFeature={+ss06},
UprightFont = EB Garamond Regular,
BoldFont = EB Garamond SemiBold,
ItalicFont = EB Garamond Italic,
BoldItalicFont = EB Garamond SemiBold Italic,
UprightFeatures = { SizeFeatures = {
```

```
 {Size = {-10}, Font = EB Garamond Medium},
 {Size = {10-}, Font = EB Garamond}}},
BoldFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Bold},
 {Size = {10-}, Font = EB Garamond SemiBold}}},
ItalicFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Medium Italic},
 {Size = {10-}, Font = EB Garamond Italic}}},
BoldItalicFeatures = { SizeFeatures = {
 {Size = {-10}, Font = EB Garamond Bold Italic},
 {Size = {10-}, Font = EB Garamond SemiBold
                                Italic}}},
]
\setmathfont{Garamond-Math}

\usepackage{hologo} % XeTeX & co. logos
\usepackage[os=win]{menukeys} % keys and menus
\newfontfamily{\voll}[]{Vollkorn} % examples
\newfontfamily{\genplus}[]{Gentium Plus}
% for the initials chapter --
\newfontfamily{\Iniciales}[RawFeature={+salt}]{Pali}
\renewcommand{\LettrineFontHook}{\Iniciales}
\setcounter{DefaultLines}{4}
\usepackage{magaz}

% captions -------------------------------
\usepackage{caption}
\captionsetup{justification=centerlast,%
    labelfont=sc,textfont=it}
% all about page styles ------------
\renewcommand{\headrulewidth}{0.4pt}

% not show the chapter number
% nor section in headers -------------------
\renewcommand{\chaptermark}[1]{
```

```
\markboth{#1}{}}
\renewcommand{\sectionmark}[1]{
\markright{#1}}

% the normal page style, with its headers
\pagestyle{fancy}
\fancypagestyle{normal}{%
   \fancyhead{}
   \fancyhead[RO,LE]{\thepage}
   \fancyhead[LO]{\nouppercase{\leftmark}}
   \fancyhead[RE]{\nouppercase{\rightmark}}
   \fancyfoot{}
}
% the index page style
\fancypagestyle{index}{%
   \fancyhead{}
   \fancyhead[RO,LE]{\thepage}
   \fancyhead[LO]{\nouppercase{Index}}
   \fancyhead[RE]{\nouppercase{Index}}
   \fancyfoot{}
}
% the section about headings -----------------
\usepackage{titlesec}
\newfontfamily{\cormorant}{Cormorant}
\titleformat{\part}[display]
   {\huge\cormorant}{\filright%\thepart\
   }{20pt}{\Huge}
\titleformat{\chapter}[frame]
   {\LARGE\cormorant}{\filright
     \chaptertitlename\
     \thechapter}{20pt}{\huge}
\titleformat*{\section}{\Large\bfseries\cormorant}
\titleformat*{\subsection}{\large\bfseries\cormorant}

% to make \part pages without footer
```

```
\makeatletter
\renewcommand\part{%
  \if@openright
    \cleardoublepage
  \else
    \clearpage
  \fi
  \thispagestyle{empty}%
  \if@twocolumn
    \onecolumn
    \@tempswatrue
  \else
    \@tempswafalse
  \fi
  \null\vfil
  \secdef\@part\@spart}
\makeatother
\makeatletter
\def\@endpart{\vfil\newpage
              \if@twoside
               \if@openright
                \null
                \thispagestyle{empty}%
\vspace*{3cm}%
\begin{quote}%
\partnote
\end{quote}%
               \fi
              \fi
              \if@tempswa
                \twocolumn
              \fi}
\newcommand\partnote{}
\makeatother
```

Appendix D
Other formats... and a greeting

A short list of the *other* things I have set up to achieve this format.

D.1 Other formats used in this book

For this book I have activated the following modules:

- Fancy Colored Boxes
- Initials (Drop Caps)
- Customisable Lists (enumitem)

The page layout uses an A5 page with fancy headers and a "two-sided" layout.

Margins are

Top	2,5 cm
Bottom	1,5 cm
Inner	2,5 cm
Outer	1,5 cm

I have activated the `Hyperref` support, allowing the links to break at the end of the line. I have removed the frames around links and selected to color links with

```
linkcolor=cyan,urlcolor=blue
```

In the BOOKMARS tab I have selected NUMBERED BOOKMARKS, so that the PDF index also show the numbers of the entries.

Output Format PDF (XeTeX)...

I have already said everything else.

D.2 And yes, also direct formatting

But only on one page!

I am referring, of course, to the cover page: defining a style to be used only there does not make sense, so I performed a few right clicks to change the paragraph formatting (bring them to right alignment) and the text formatting (its size), together with a few page breaks (Insert » Formatting). The short text with the font Kleymisska uses a style... albeit with a manually modified size.

What took more time was placing the cover image.

The penguin silhouette in front of the gates of ~~Moria~~ LyX was created in LibreOffice Draw over the outline of certain image that I used as an example in chapter 14. The LaTeX and Co. logos in that image were created with LyX using the PDF (cropped) template I talked about at the time, converting them to SVG and inserting them into the Draw drawing along with some rounded rectangles and gradients.

Since Draw does not allow exporting only a selection when creating a PDF, the slide of the document was modified to have no margins and to be of the exact dimension needed for the image.

It is important, when exporting to PDF from Draw, to select "loose less compression" to not ruin the image... but OK, I am digressing, after all this is a book about LyX.

From the Draw document I then created a PDF file that was inserted in the page dedicated to the cover of this document.

And here comes the interesting part. Creating a page style with its margins only for the cover make no sense, but if I use only the width of the text area for the image it would be too small so explicitly indicating the size of the image as I discussed in chapter 14 "sounds like a solution," right? Not quite: making the picture start *before* the left margin needs some extra work.

But everything can be solved! Here's the direct formatting sequence used.

First, a TEX box with the following code:

```
\begingroup
\leftskip=-3em
```

The second instruction causes the paragraph below to take a negative indentation of 3 em... and also the following one and all the paragraphs after it: that's why it was necessary to create a group with the first instruction in order to isolate the effects of `\leftskip` from the rest of the document.

Below that TEX box I inserted the PDF picture commented before, setting it to take 81% of the full page width.

Finally, below the image another TEX box with a

```
\endgroup
```

to close the group and finish the process.

For the back cover, I also inserted some vertical spaces with the help of Insert » Formatting » Vertical Space, but they are easy to understand.

And yes, I had to compile the document several times until I managed to adjust everything and get it right. Oh, well.

D.3 Final greetings

If you've made it this far, reader, thank you for your patience!

I hope you enjoyed the book. I certainly enjoyed writing it. Trying to explain is that I have learned more than I thought I would ever know. The moment I decided to write this book now joins the moment you fulfilled your resolution to read it to the end. The journey has been long, but I think it was worth it.

Until the next *Typographical Adventure*!